CHƯƠNG 3
TẦNG VẬN CHUYỂN

NHẬP MÔN MẠNG MÁY TÍNH

A note on the use of these PowerPoint slides:

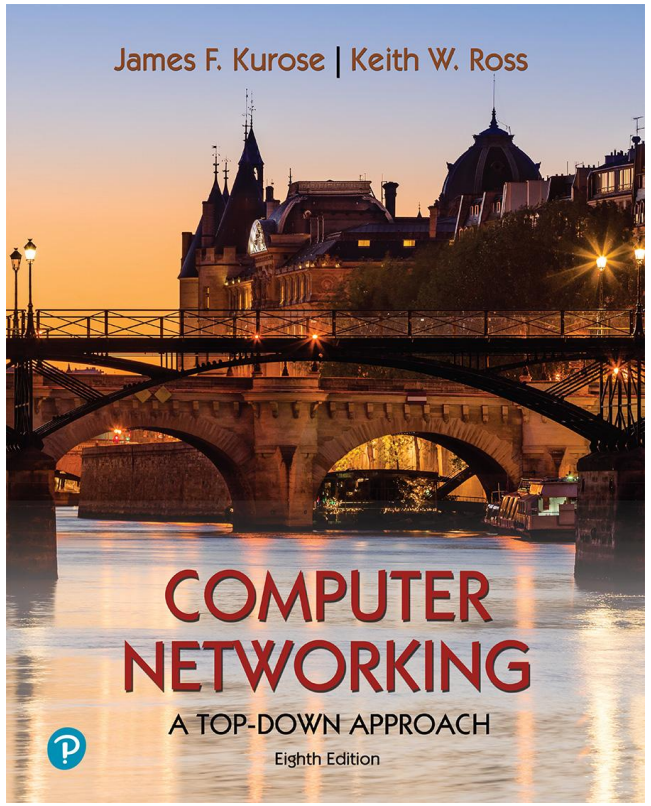
We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

For a revision history, see the slide note for this page.

Thanks and enjoy! JFK/KWR

All material copyright 1996-2020
J.F Kurose and K.W. Ross, All Rights Reserved



Computer Networking: A Top-Down Approach

8th edition
Jim Kurose, Keith Ross
Pearson, 2020

Tổng quan

Mục đích:

- Hiểu được các nguyên tắc/cơ chế hoạt động:
  - multiplexing, demultiplexing
  - reliable data transfer
  - flow control
  - congestion control

- Hiểu các giao thức:
  - UDP: connectionless transport
  - TCP: connection-oriented reliable transport
  - TCP congestion control

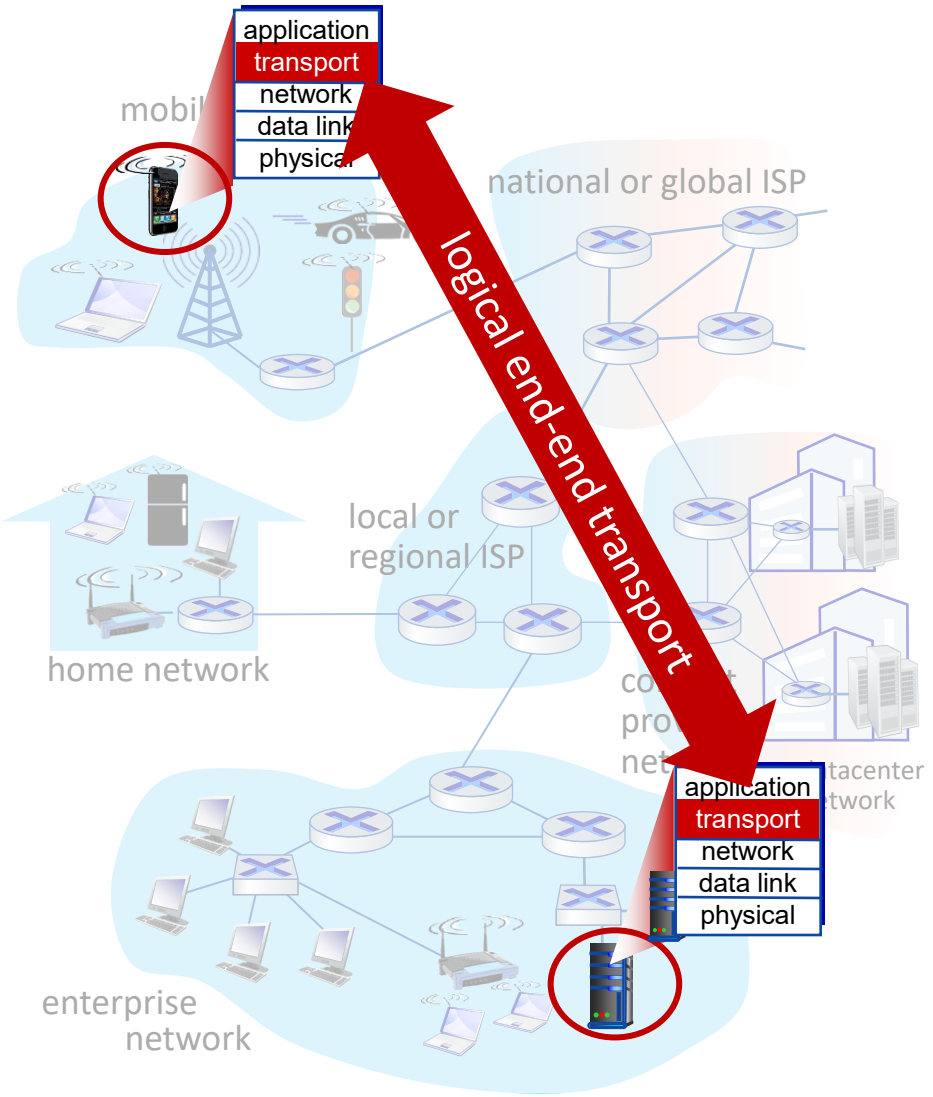
Nội dung

- Các dịch vụ tầng vận chuyển
- Multiplexing and demultiplexing
- UDP
- Nguyên lý truyền tin cậy
- TCP
- TCP - Điều khiển tắc nghẽn
- Sự phát triển của các tính năng của tầng vận chuyển

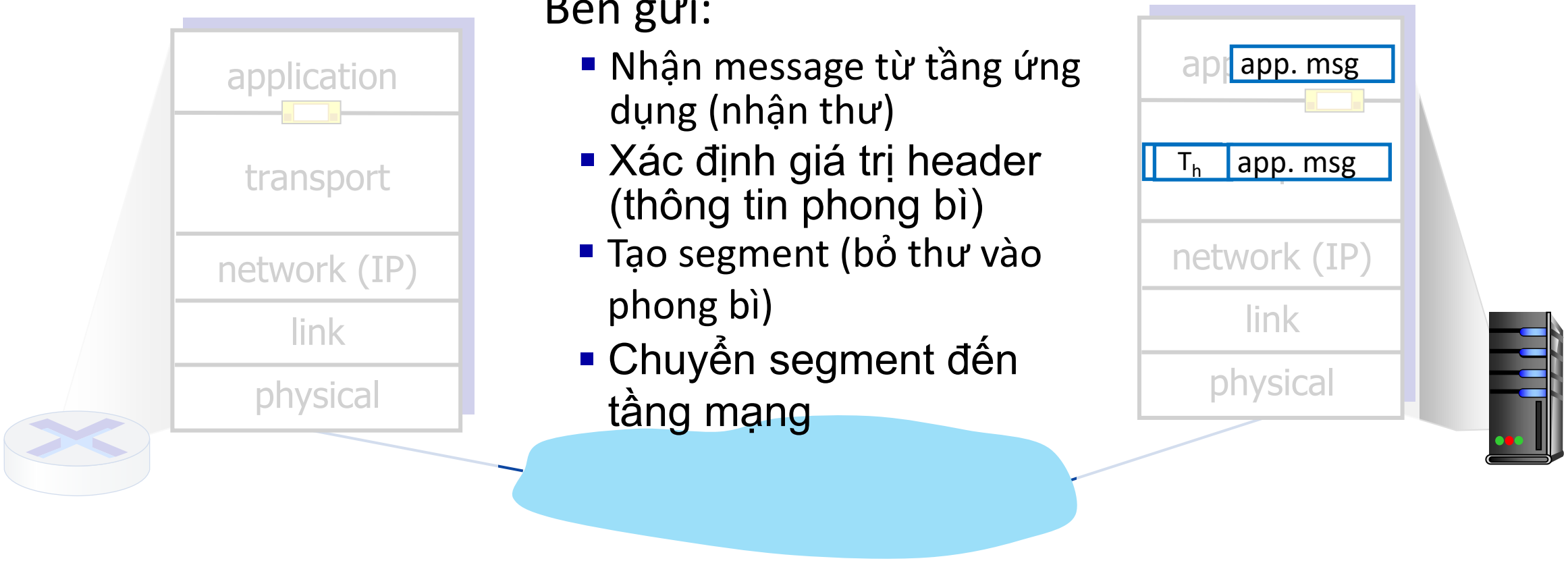


Giao thức và dịch vụ

- Cung cấp cơ chế "*truyền thông luận lý (logical communication)*" giữa các tiến trình trên các "*host*" khác nhau
- Các giao thức vận chuyển hoạt động trên các thiết bị đầu cuối:
  - Bên gửi: chia các "messages – thông điệp" thành các "segments" và chuyển xuống tầng mạng.
  - Bên nhận: ghép các segments thành messages, chuyển đến tầng ứng dụng.
- 2 giao thức chính: TCP và UDP



Hoạt động của tầng vận chuyển



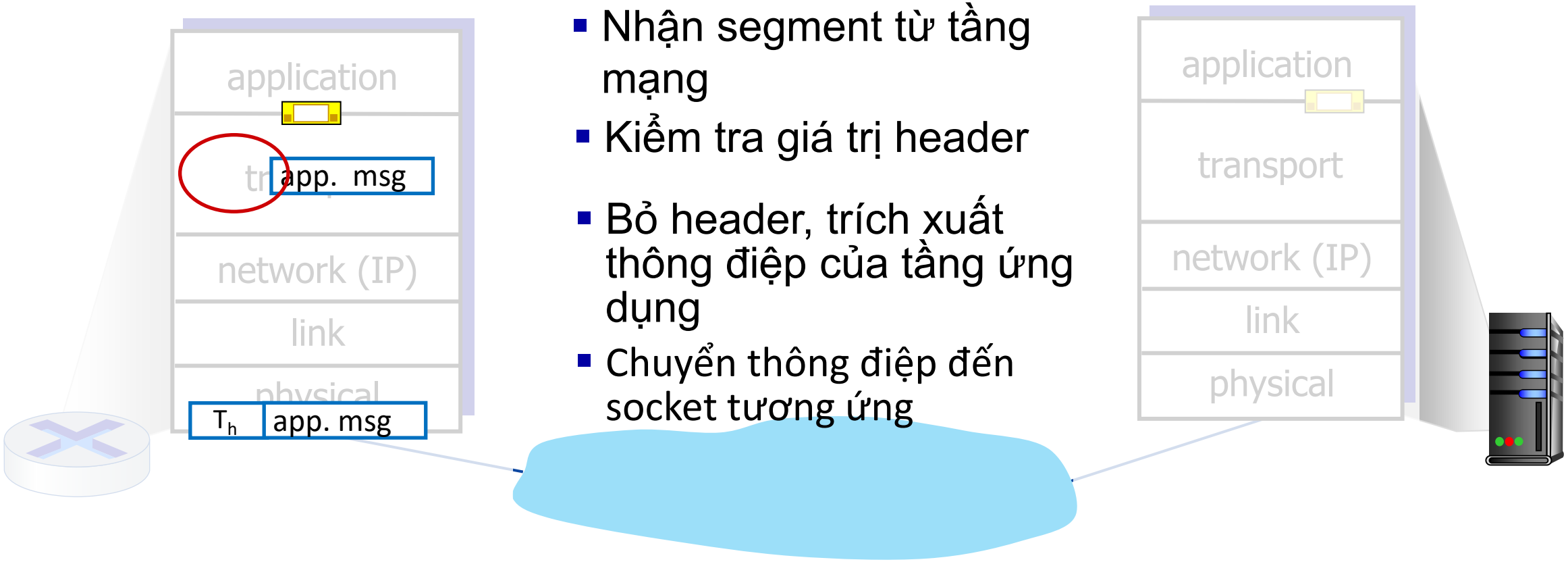
Bên gửi:

- Nhận message từ tầng ứng dụng (nhận thư)
- Xác định giá trị header (thông tin phong bì)
- Tạo segment (bỏ thư vào phong bì)
- Chuyển segment đến tầng mạng

Transport Layer Actions

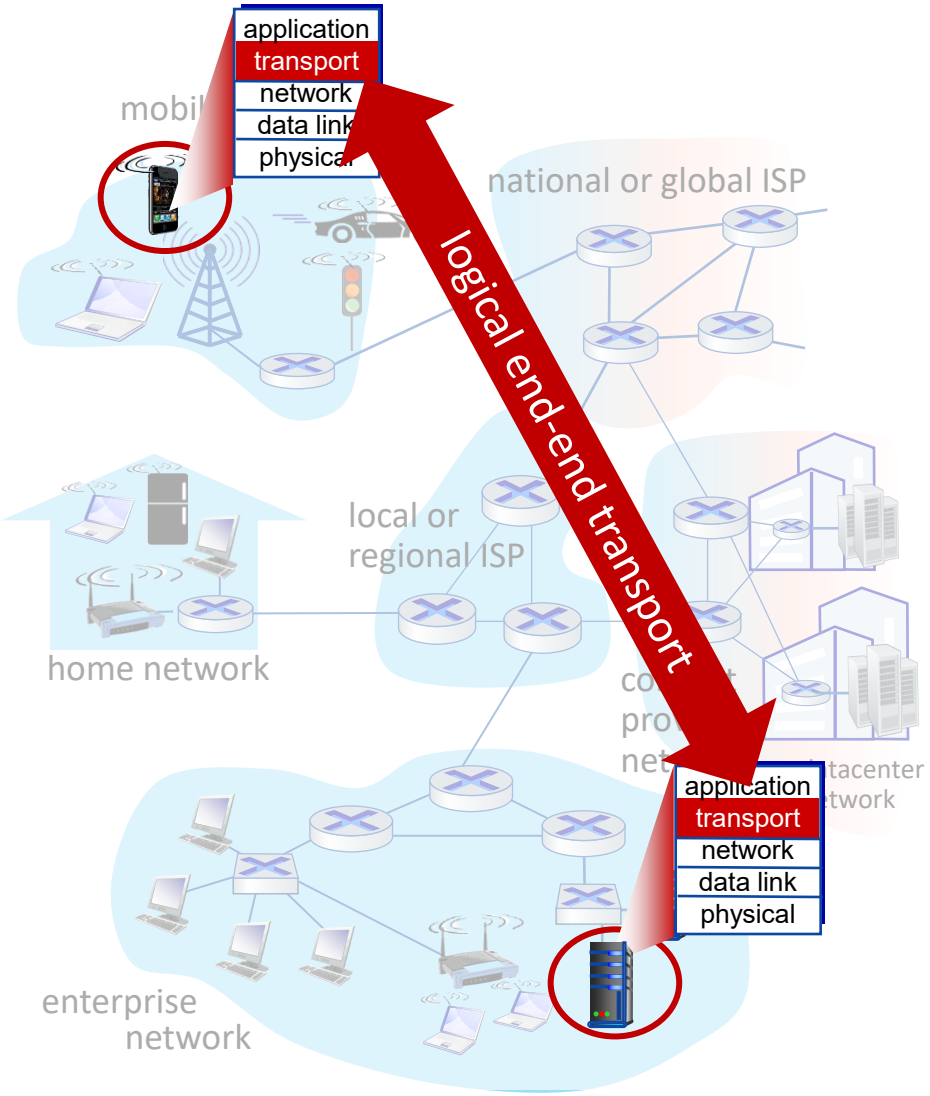
Bên nhận:

- Nhận segment từ tầng mạng
- Kiểm tra giá trị header
- Bỏ header, trích xuất thông điệp của tầng ứng dụng
- Chuyển thông điệp đến socket tương ứng



Giao thức tầng vận chuyển

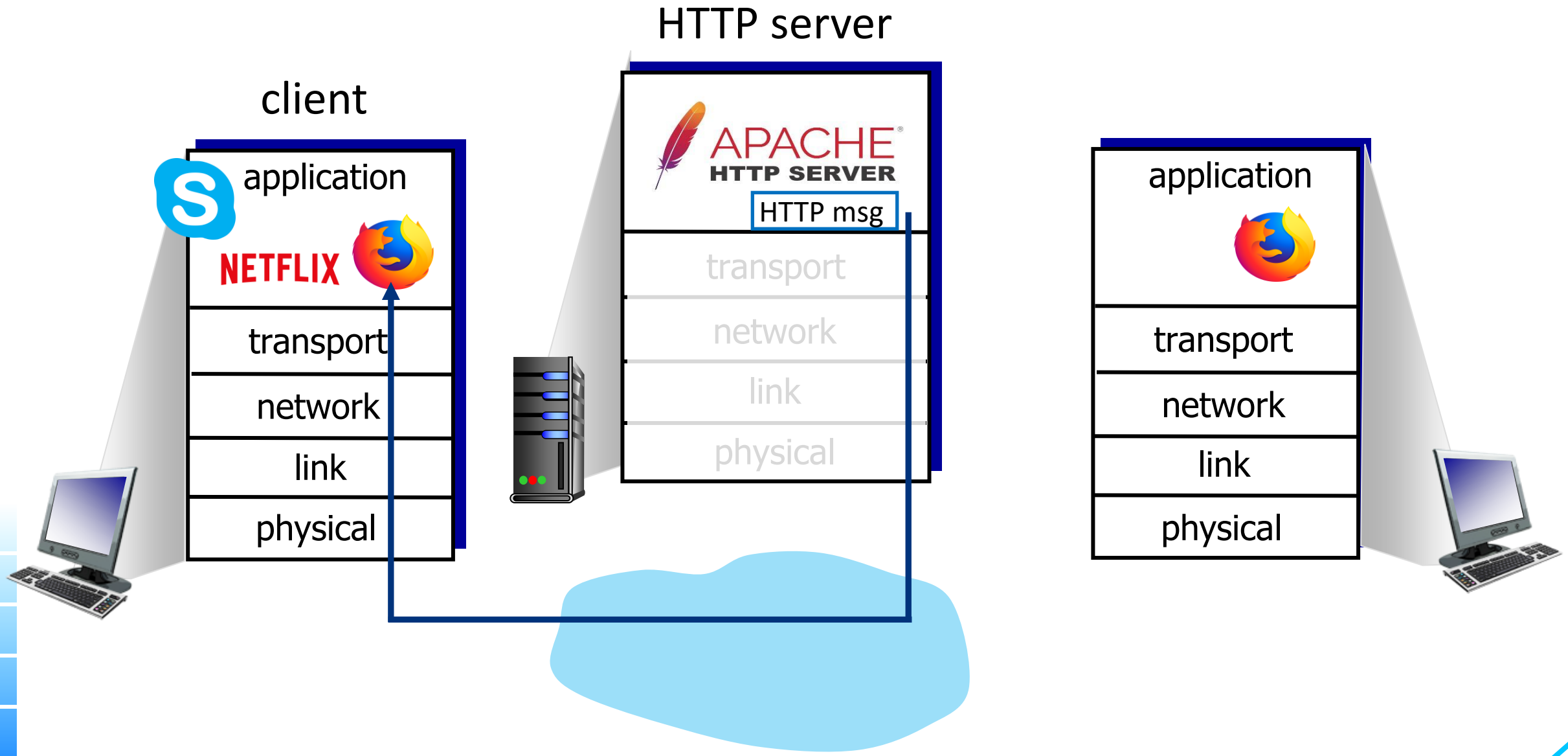
- **TCP:** Transmission Control Protocol
  - Tin cậy, vận chuyển đúng thứ tự
  - Điều khiển tắc nghẽn
  - Điều khiển luồng
  - Thiết lập kết nối
- **UDP:** User Datagram Protocol
  - Không đảm bảo độ tin cậy và không đảm bảo thứ tự nhận dữ liệu
  - Cung cấp dịch vụ vận chuyển theo nguyên tắc “best effort” trên IP
- Không cung cấp các dịch vụ sau:
  - Đảm bảo độ trễ
  - Đảm bảo băng thông

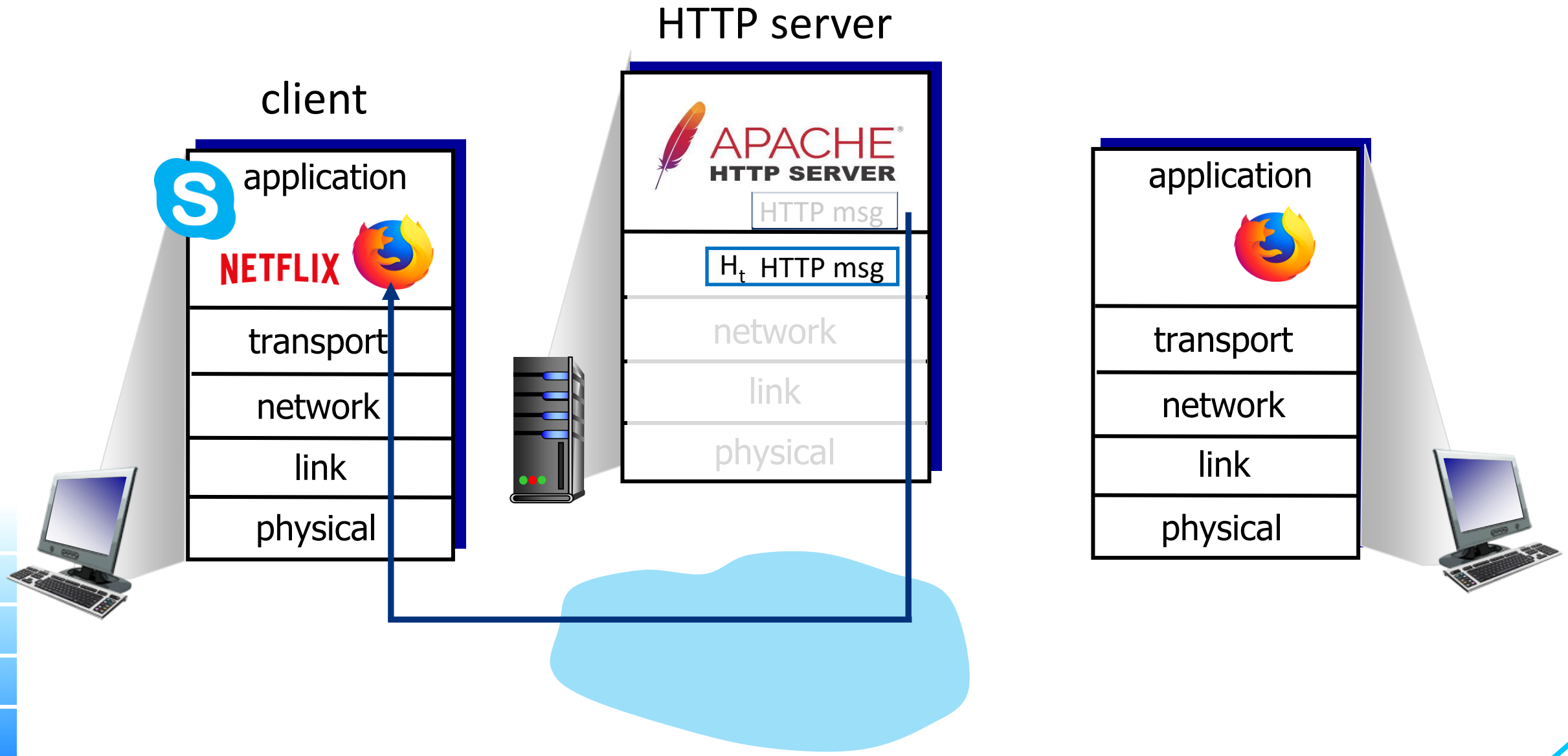


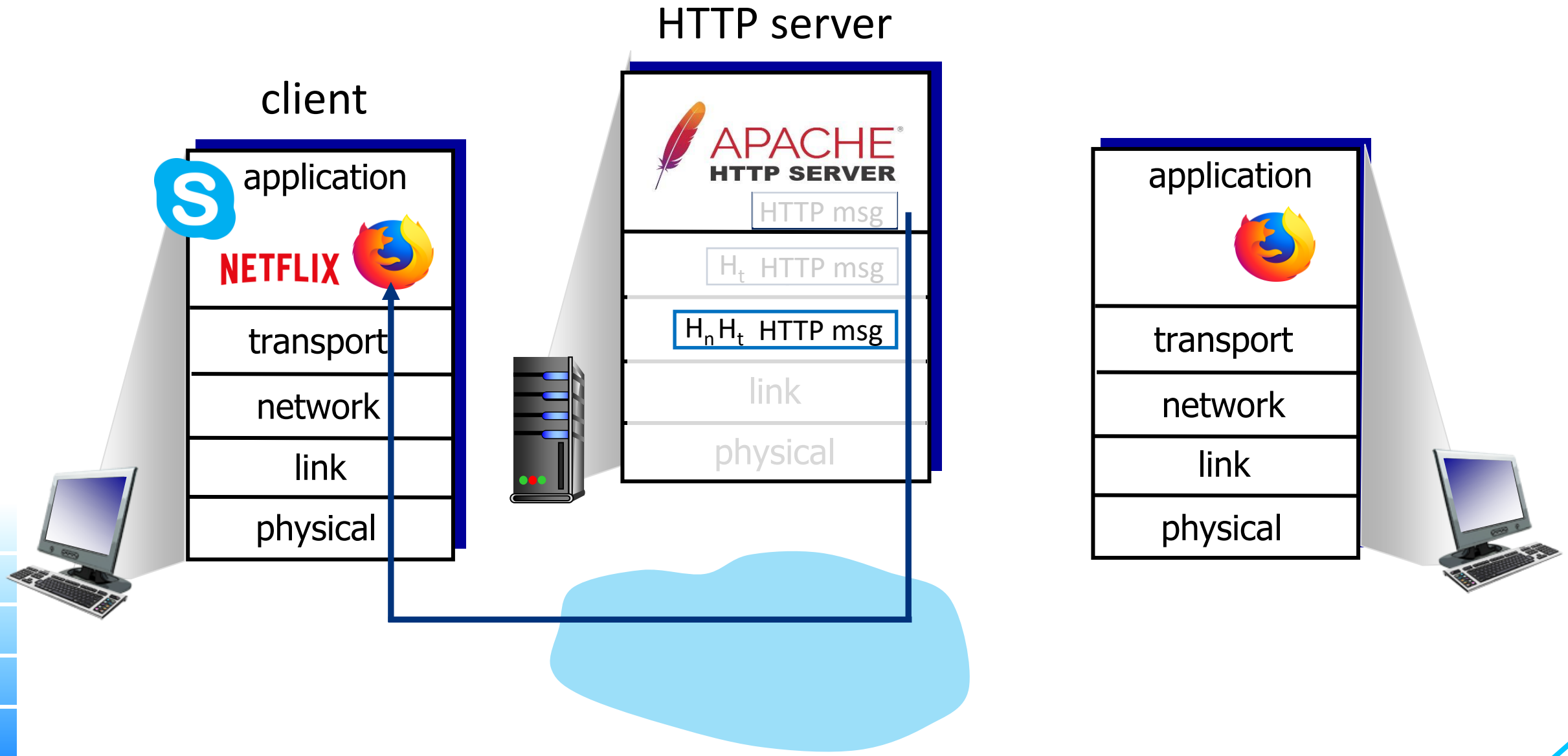
Nội dung

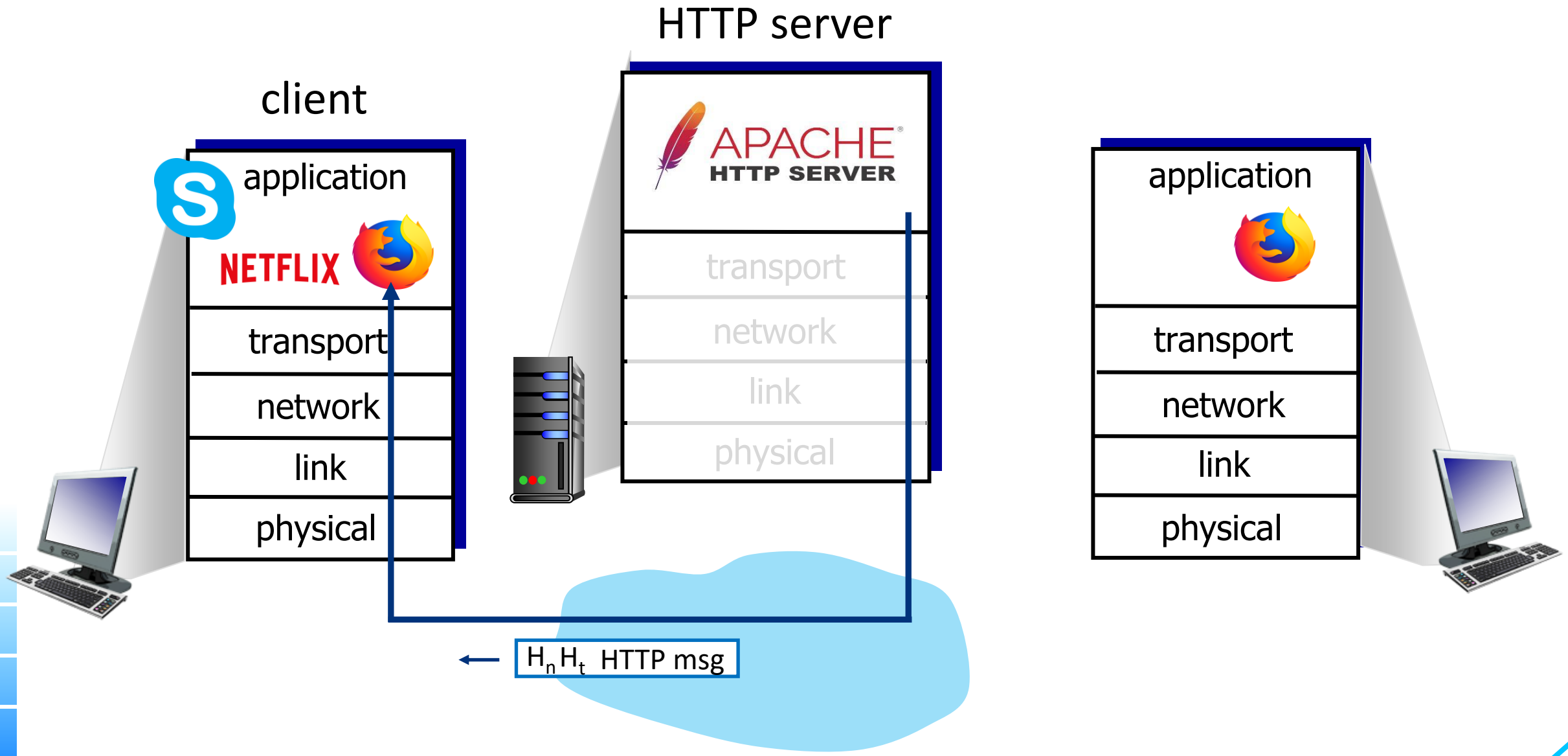
- Các dịch vụ tầng vận chuyển
- Multiplexing and demultiplexing
- UDP
- Nguyên lý truyền tin cậy
- TCP
- TCP - Điều khiển tắc nghẽn
- Sự phát triển của các tính năng của tầng vận chuyển

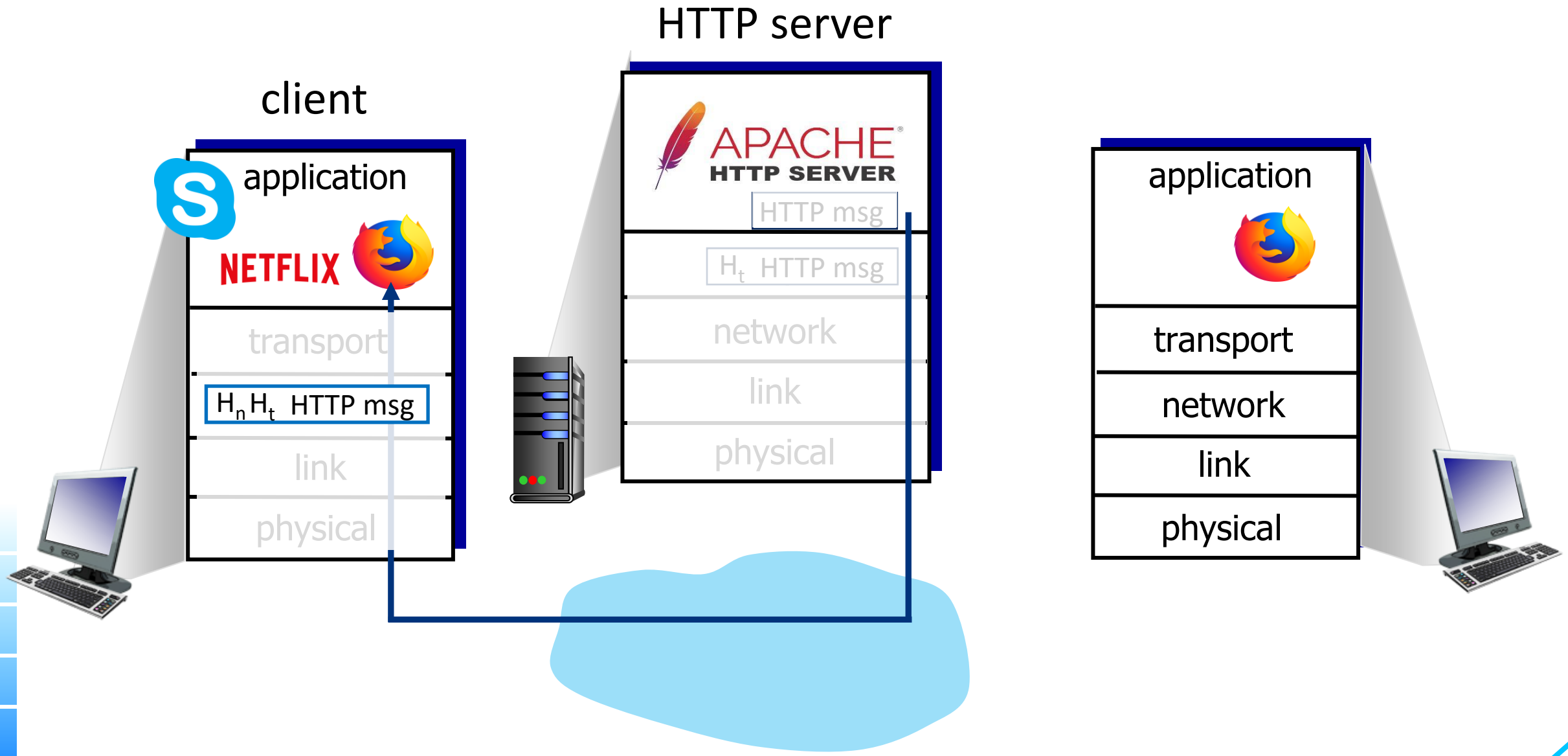


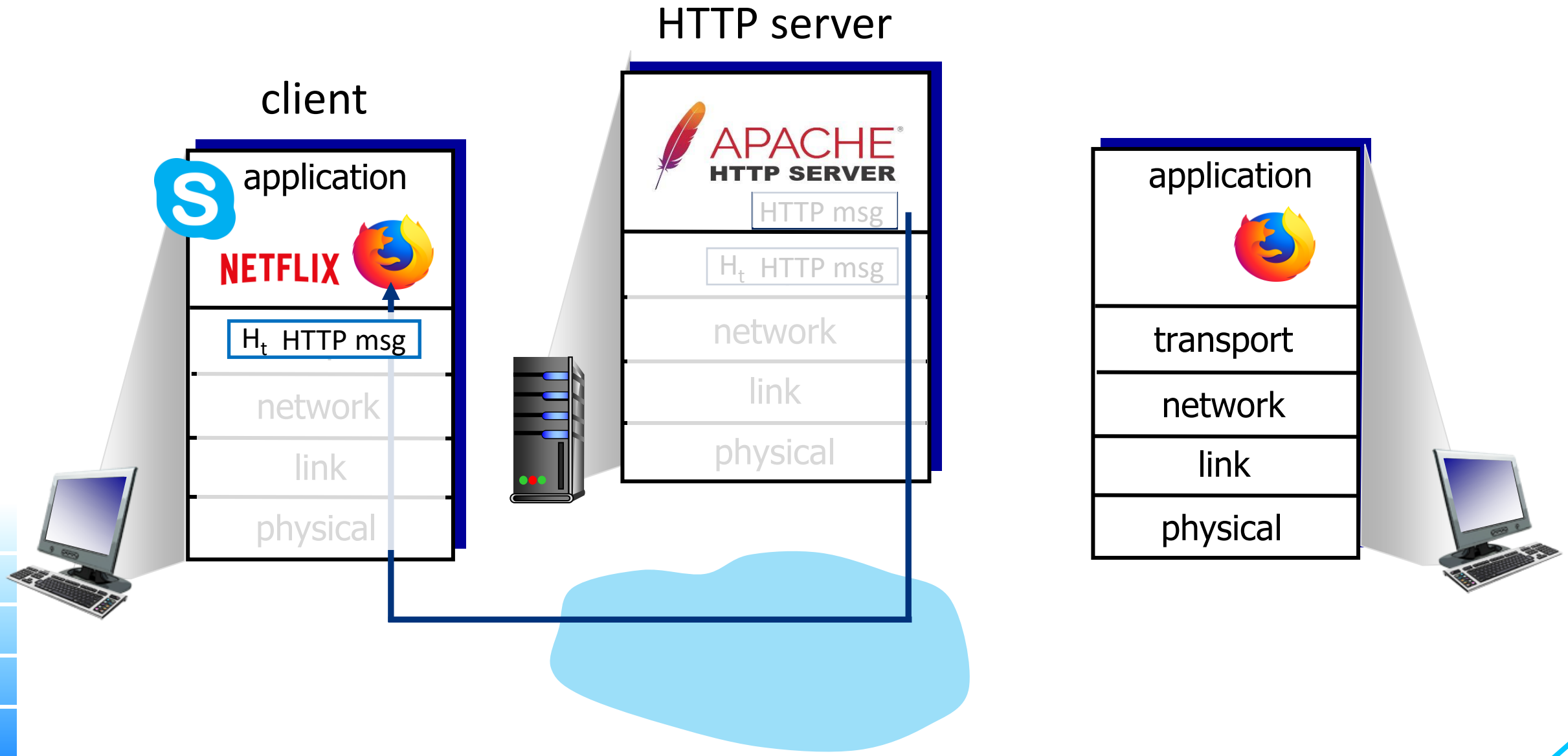

HTTP server
client
application
NETFLIX
transport
network
link
physical
APACHE
HTTP SERVER
HTTP msg
transport
network
link
physical
application
transport
network
link
physical


HTTP server
client
application
NETFLIX
transport
network
link
physical
APACHE
HTTP SERVER
HTTP msg
H_t HTTP msg
network
link
physical
application
transport
network
link
physical

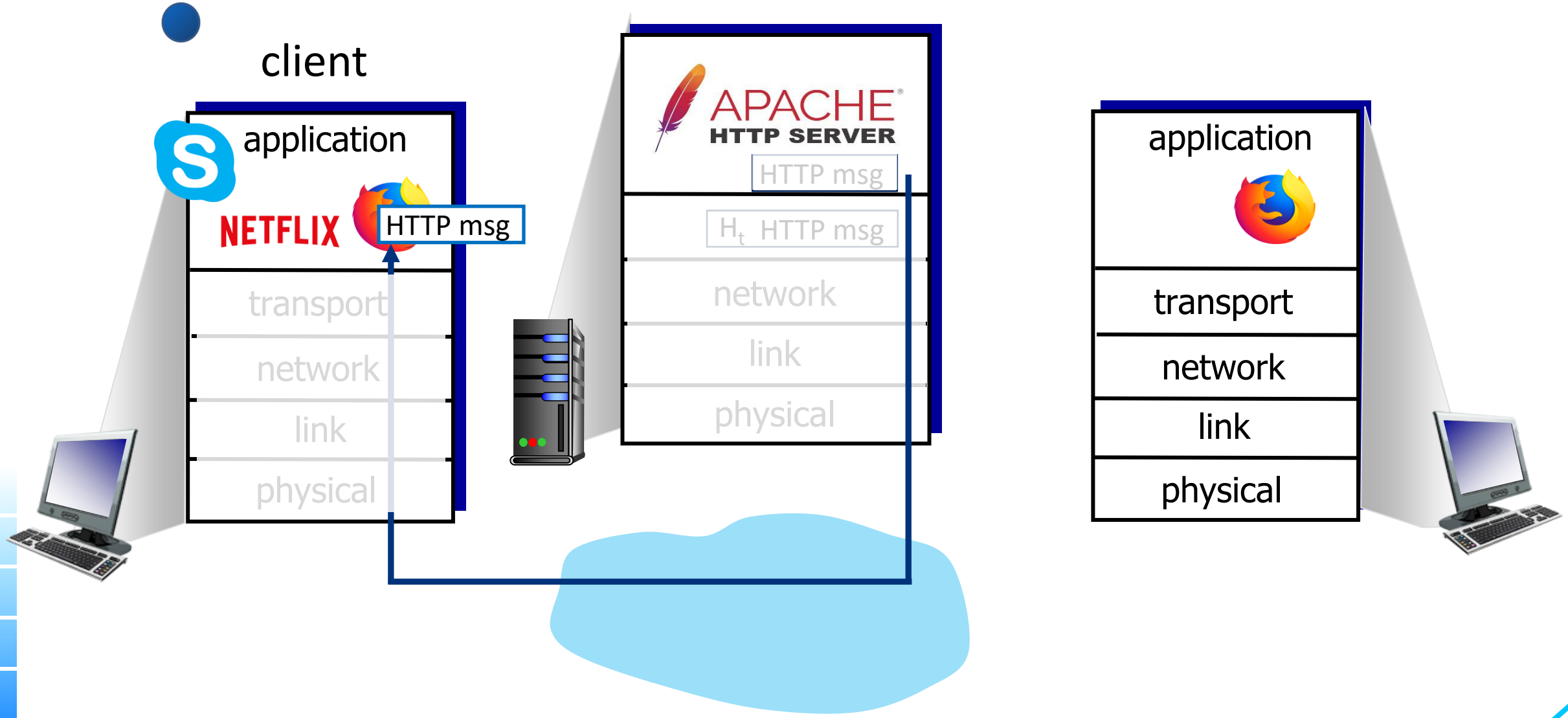

HTTP server
client
application
NETFLIX
transport
network
link
physical
APACHE
HTTP SERVER
HTTP msg
H_t HTTP msg
$H_n H_t$ HTTP msg
link
physical
application
transport
network
link
physical


HTTP server
client
application
NETFLIX
transport
network
link
physical
APACHE
HTTP SERVER
transport
network
link
physical
application
transport
network
link
physical
$H_n H_t$ HTTP msg


HTTP server
client
application
NETFLIX
transport
H_n H_t HTTP msg
link
physical
APACHE
HTTP SERVER
HTTP msg
H_t HTTP msg
network
link
physical
application
transport
network
link
physical


HTTP server
client
application
NETFLIX
H_t HTTP msg
network
link
physical
APACHE
HTTP SERVER
HTTP msg
H_t HTTP msg
network
link
physical
application
transport
network
link
physical

Q: Làm thế nào mà tầng vận chuyển biết gửi đúng message tới trình duyệt Firefox thay vì Netflix hoặc Skype?



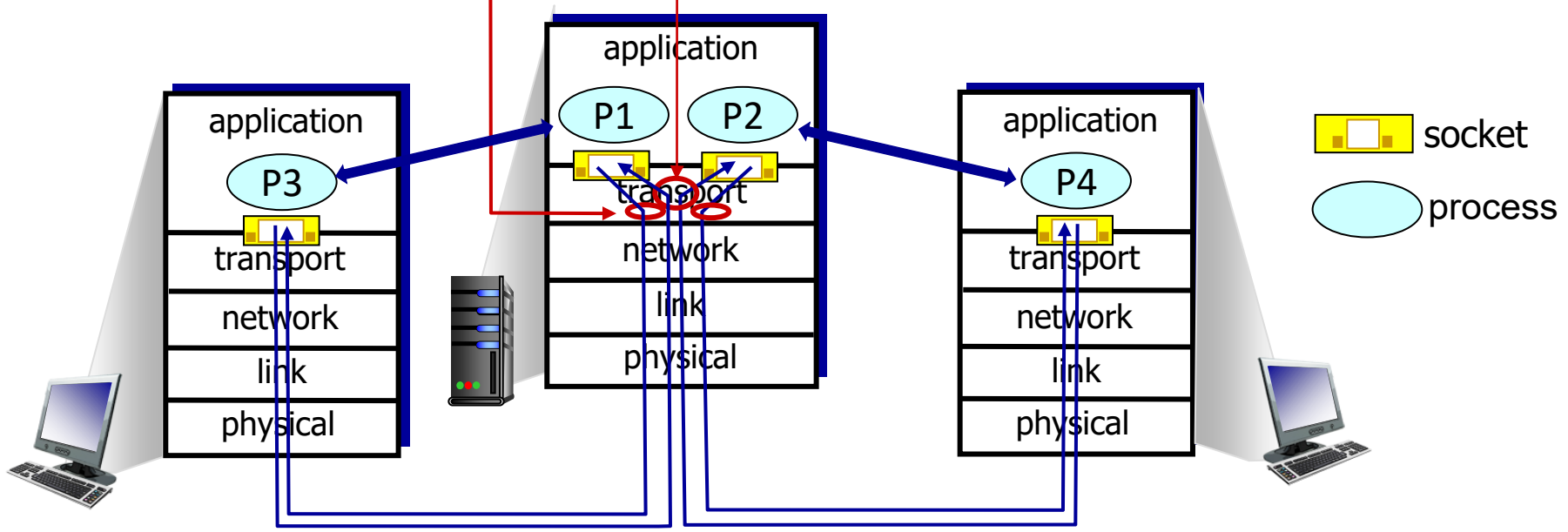
Multiplexing/demultiplexing

multiplexing tại bên gửi:

Nhận dữ liệu từ socket, thêm header của tầng vận chuyển

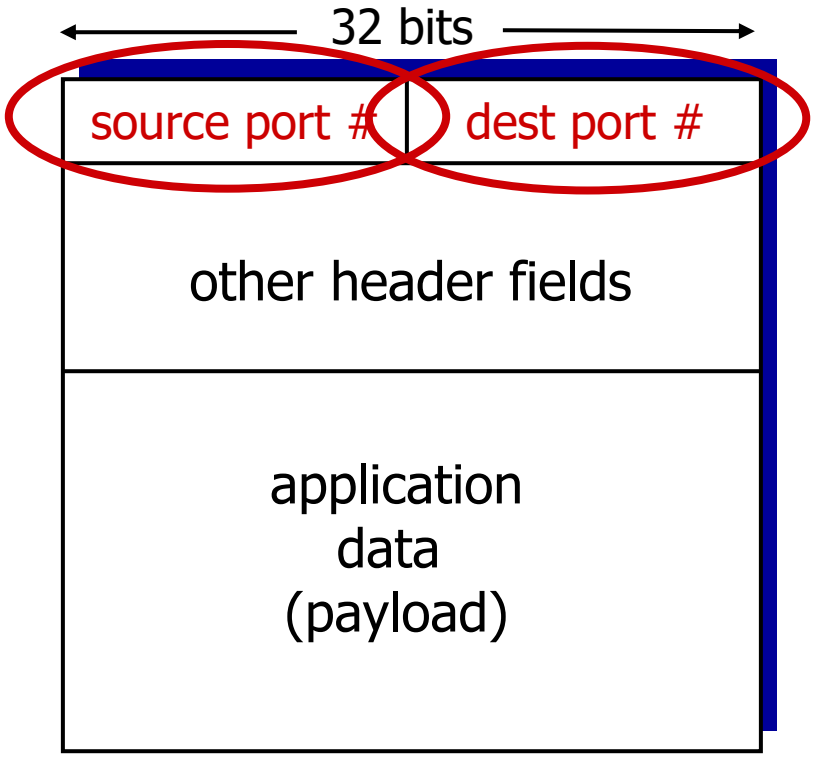
demultiplexing tại bên nhận:

Sử dụng thông tin trong header để chuyển segment nhận được đến đúng socket.



Demultiplexing làm việc thế nào?

- Khi host nhận được một IP "datagram"
  - Mỗi datagram có địa chỉ IP nguồn và IP đích
  - Mỗi datagram này chứa 1 segment (đơn vị dữ liệu của tầng vận chuyển)
- Mỗi segment có port nguồn và port đích
  - "Host" dùng địa chỉ IP & port numbers để chuyển segment đến đúng socket tương ứng



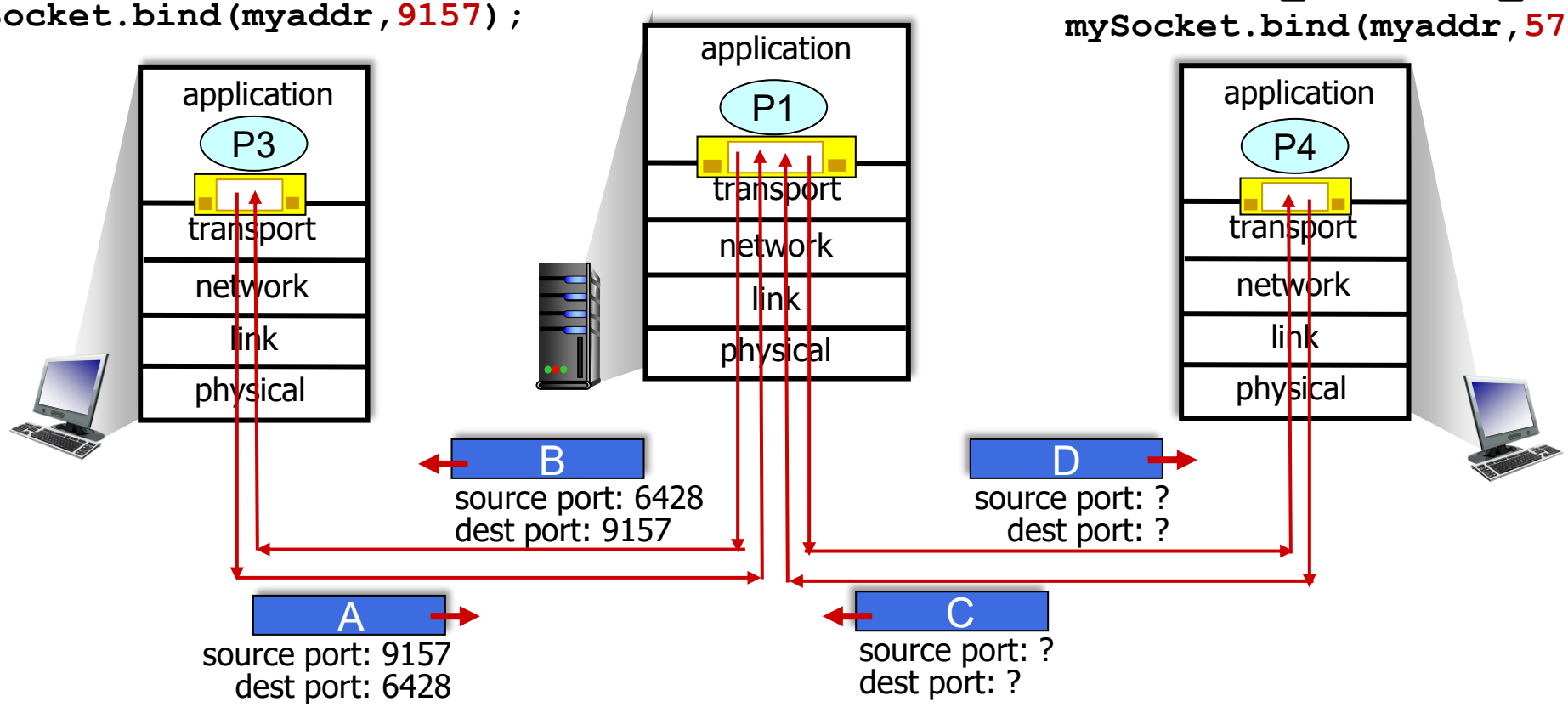
TCP/UDP segment format

Ví dụ về "connectionless - không kết nối"

```
mySocket =
  socket(AF_INET,SOCK_DGRAM)
mySocket.bind(myaddr,6428);
```

```
mySocket =
  socket(AF_INET,SOCK_STREAM)
mySocket.bind(myaddr,9157);
```

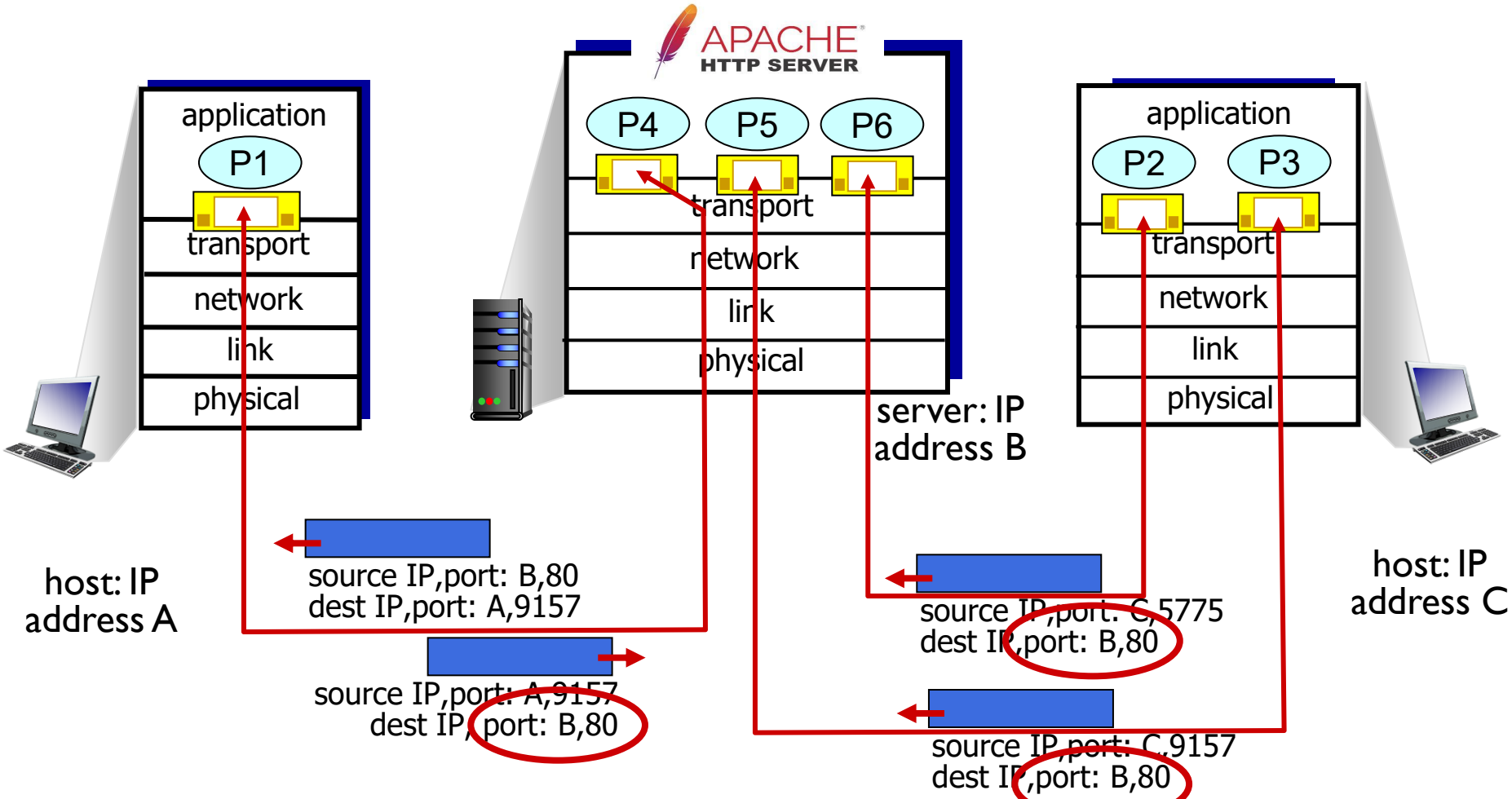
```
mySocket =
  socket(AF_INET,SOCK_STREAM)
mySocket.bind(myaddr,5775);
```



Hướng kết nối

- TCP socket được xác định bởi 4-tuple (4 thông tin chính):
  - source IP address
  - source port number
  - dest IP address
  - dest port number
- demux: bên nhận sử dụng cả 4 thông tin chính để chuyển segment đến đúng socket
- Server có thể hỗ trợ nhiều TCP socket cùng lúc:
  - Mỗi socket được xác định bởi 4 thông tin
  - Mỗi socket tương ứng với một client đang kết nối với nó

Ví dụ về "hướng kết nối"



Cả ba segment đều có địa chỉ IP đích B và cổng đích 80, nhưng được phân kênh đến các socket khác nhau dựa trên bộ bốn thông tin của kết nối.

Tổng kết về Multiplexing, demultiplexing

- Multiplexing, demultiplexing: dựa trên giá trị các trường thông tin trong header của segment, datagram
  - **UDP**: demultiplexing chỉ sử dụng destination port number
  - **TCP**: demultiplexing sử dụng 4 thông tin: source and destination IP addresses, and port numbers
- Multiplexing/demultiplexing hoạt động ở tất cả các tầng

Nội dung

- Các dịch vụ tầng vận chuyển
- Multiplexing and demultiplexing
- UDP
- Nguyên lý truyền tin cậy
- TCP
- TCP - Điều khiển tắc nghẽn
- Sự phát triển của các tính năng của tầng vận chuyển



UDP: User Datagram Protocol

- Một giao thức đơn giản
- Cung cấp dịch vụ "best effort"
  - Segment có thể mất
  - Segment có thể đến không đúng thứ tự
- "connectionless":
  - Không có bước thiết lập kết nối
  - Mỗi segment được xử lý độc lập

Tại sao vẫn dùng UDP?

- Không có bước thiết lập kết nối (bước này làm tăng độ trễ)
- Đơn giản: không lưu trữ trạng thái kết nối
- Header nhỏ gọn
- Không điều khiển tắc nghẽn
  - UDP gửi segment ngay khi có thể, không điều chỉnh tốc độ theo tình trạng tắc nghẽn

UDP: User Datagram Protocol

- UDP được sử dụng bởi:
  - streaming multimedia apps (loss tolerant, rate sensitive)
  - DNS
  - SNMP
  - HTTP/3
- Nếu cần truyền tin cậy qua UDP (e.g., HTTP/3):
  - Thêm xử lý tin cậy ở tầng ứng dụng
  - Thêm điều khiển tắc nghẽn ở tầng ứng dụng

UDP: User Datagram Protocol [RFC 768]



INTERNET STANDARD

RFC 768

J. Postel
ISI
28 August 1980

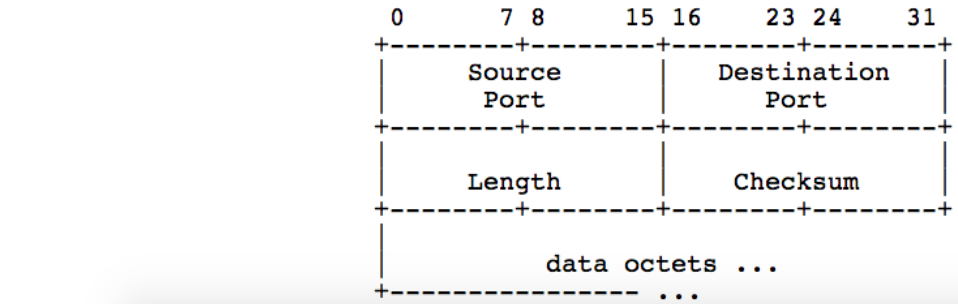
User Datagram Protocol

Introduction

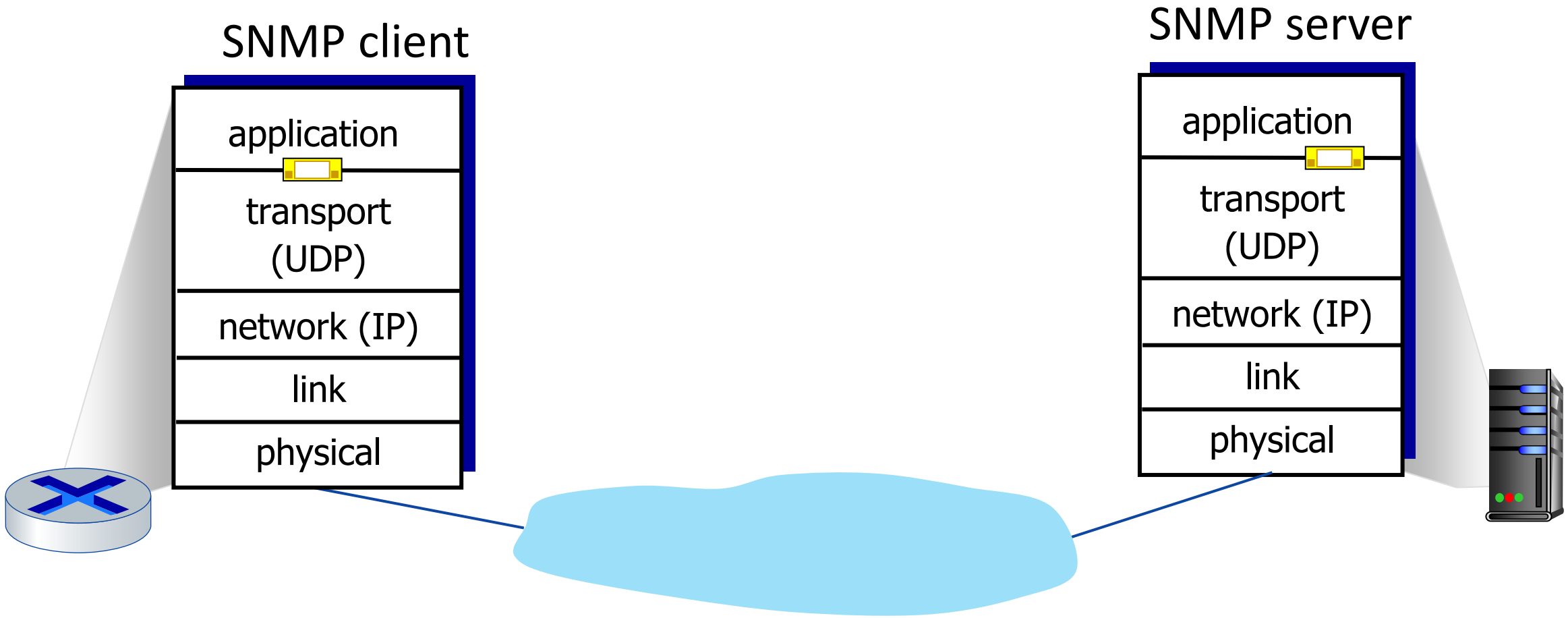
This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

Format



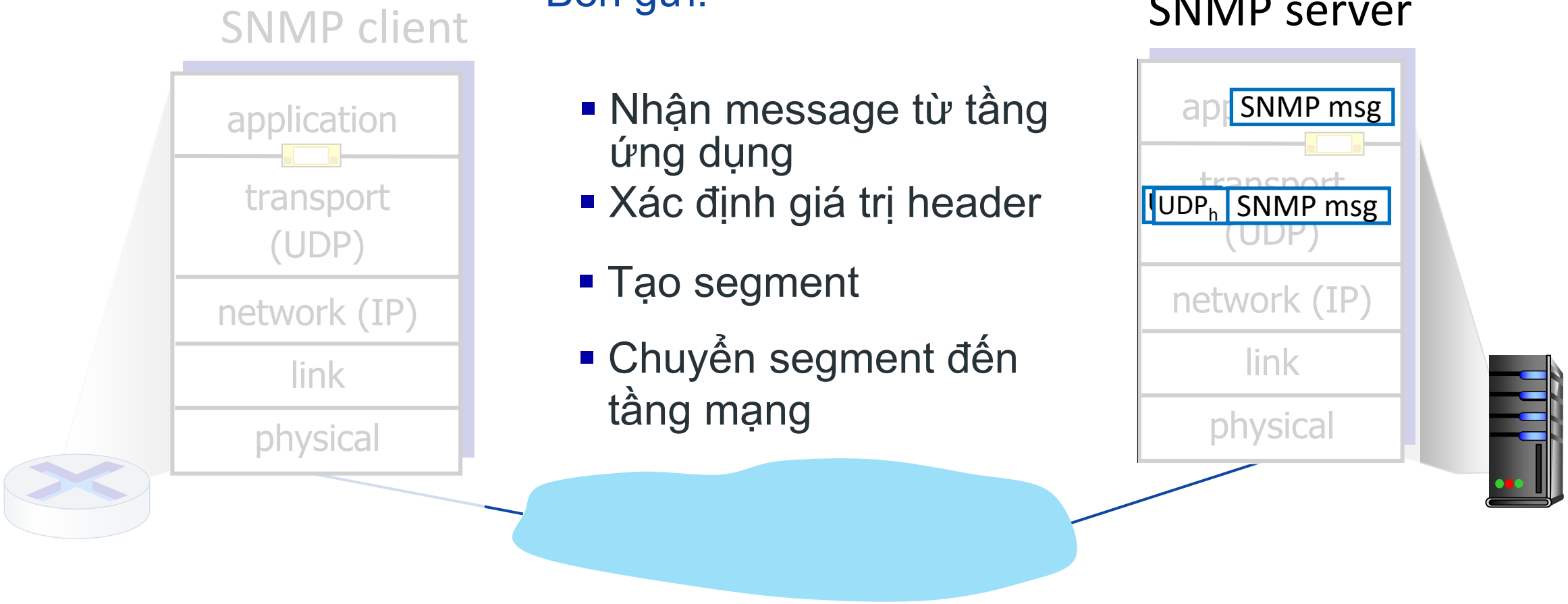
UDP: Hoạt động



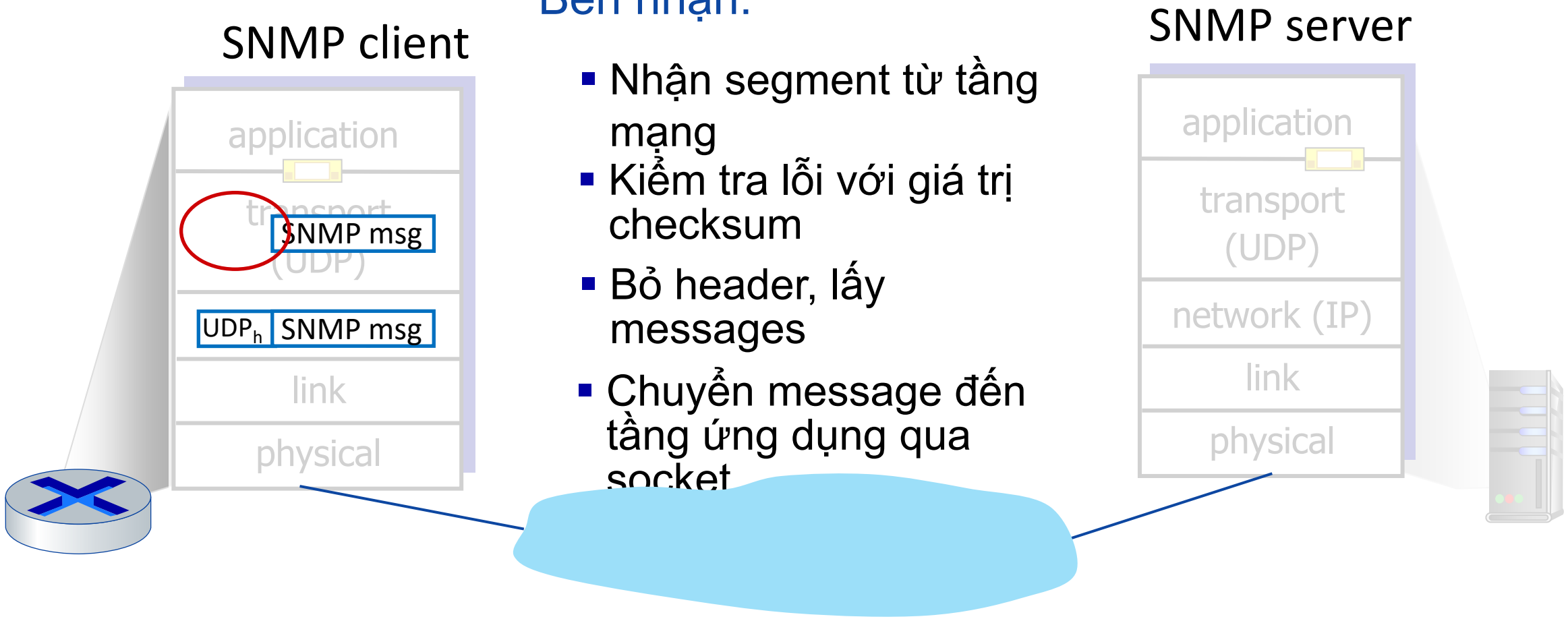
UDP: Hoạt động

Bên gửi:

- Nhận message từ tầng ứng dụng
- Xác định giá trị header
- Tạo segment
- Chuyển segment đến tầng mạng



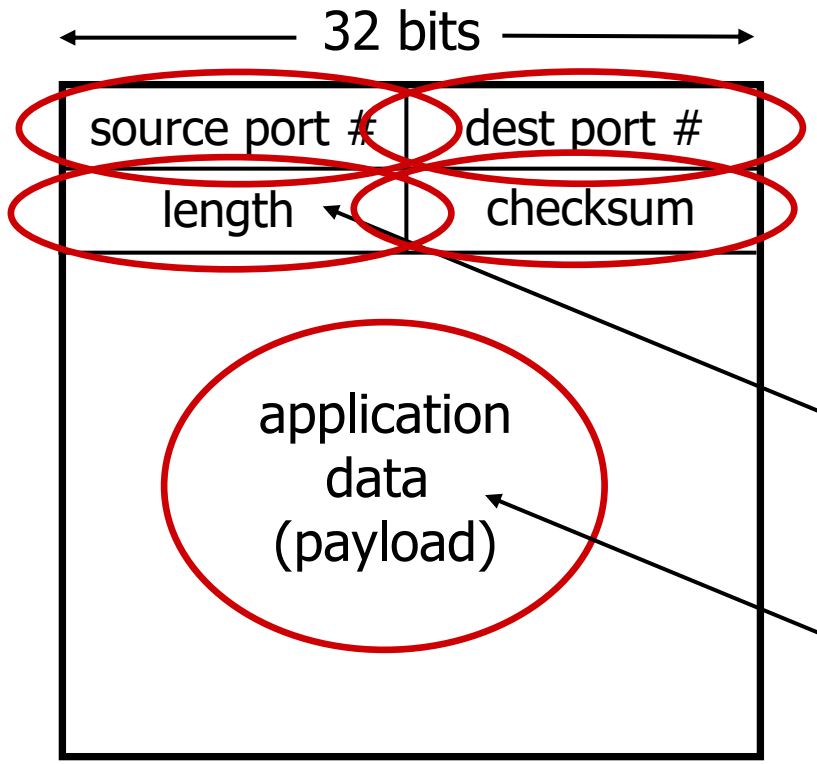
UDP: Hoạt động



Bên nhận:

- Nhận segment từ tầng mạng
- Kiểm tra lỗi với giá trị checksum
- Bỏ header, lấy messages
- Chuyển message đến tầng ứng dụng qua socket

UDP header



UDP segment format

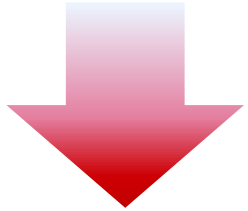
Length: chiều dài tính theo byte của UDP segment, bao gồm cả header

Dữ liệu tầng ứng dụng

UDP checksum

Mục tiêu: phát hiện lỗi trong quá trình truyền (các bit bị lỗi, từ 0 thành 1 hoặc ngược lại)

	1st number	2nd number	sum
Gửi:	5	6	11



Nhận:	4	6	11

receiver-computed checksum ≠ sender-computed checksum (as received)



Internet checksum

- *Mục tiêu: phát hiện lỗi trong quá trình truyền (các bit bị lỗi, từ 0 thành 1 hoặc ngược lại)*

sender:

- Chia nội dung của segment (bao gồm các trường tiêu đề UDP và địa chỉ IP) dưới dạng chuỗi 16 bit
- checksum: Tính tổng bù 1 (one's complement sum)
- Đặt kết quả vào phần UDP checksum trong header

receiver:

- Tính checksum của segment nhận được
- So sánh kết quả tính được và nội dung phần checksum
  - Không bằng nhau – có lỗi
  - Bằng nhau – không phát hiện lỗi. *Tuy nhiên*….

Internet checksum: ví dụ

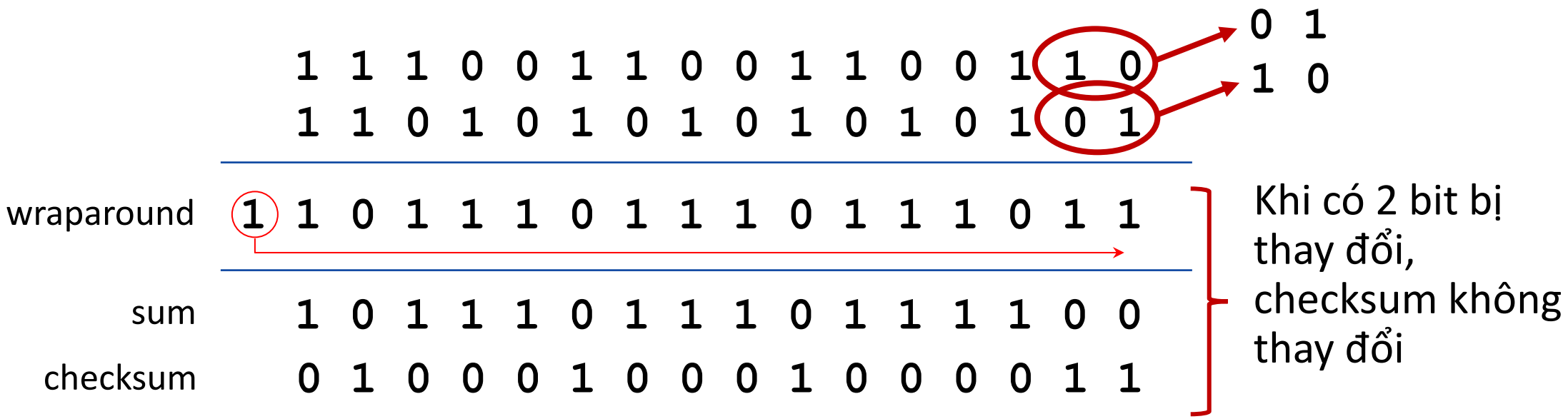
- Ví dụ: cộng 2 chuỗi 16-bit

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	(1)	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1
sum		1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum		0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

Note: nếu kết quả tạo ra bit nhớ thứ 17, cộng bit nhớ này trở lại vào 16 bit thấp của kết quả.

Internet checksum: điểm yếu!

- Ví dụ: cộng 2 chuỗi 16-bit



Tổng kết: UDP

- Giao thức đơn giản:
  - Segment có thể bị mất, sai thứ tự
  - “Best-effort”: gửi và hy vọng có kết quả tốt nhất
- UDP vẫn có điểm cộng:
  - Không có quá trình thiết lập kết nối
  - Có checksum để kiểm tra lỗi
- Các tính năng bổ sung thì đưa lên tầng ứng dụng (e.g., HTTP/3)

Nội dung

- Các dịch vụ tầng vận chuyển
- Multiplexing and demultiplexing
- UDP
- Nguyên lý truyền tin cậy
- TCP
- TCP - Điều khiển tắc nghẽn
- Sự phát triển của các tính năng của tầng vận chuyển



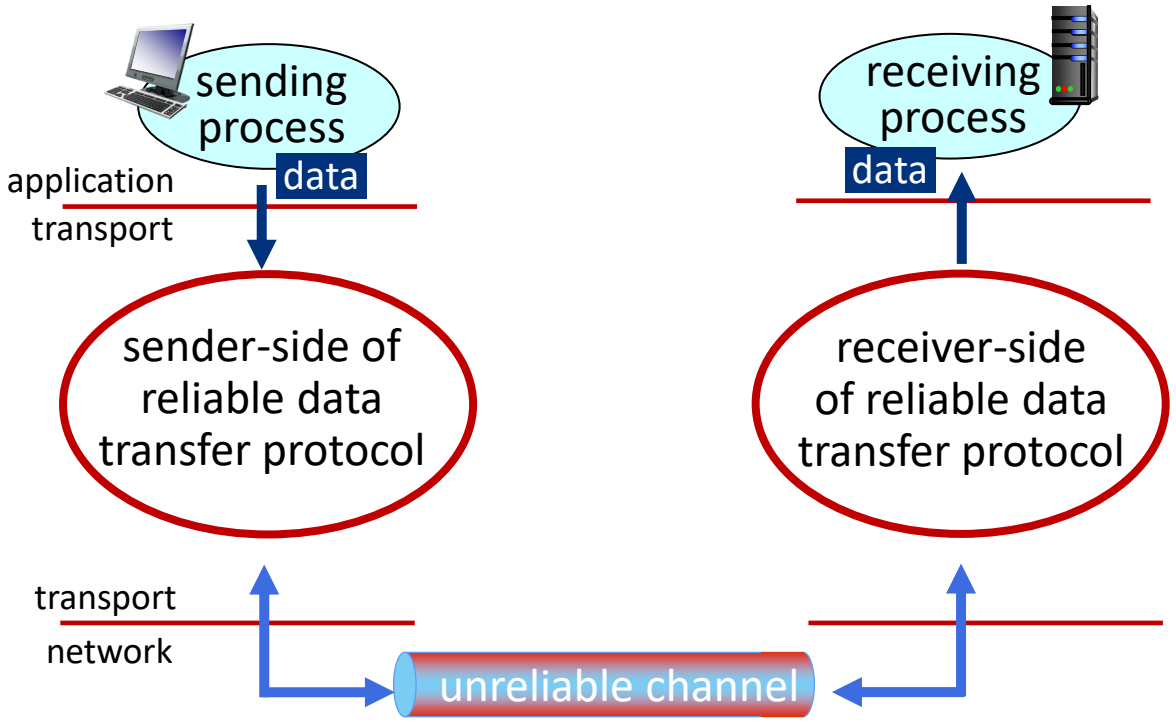
Nguyên lý truyền tin cậy



Tổng quát dịch vụ truyền tin cậy

Principles of reliable data transfer

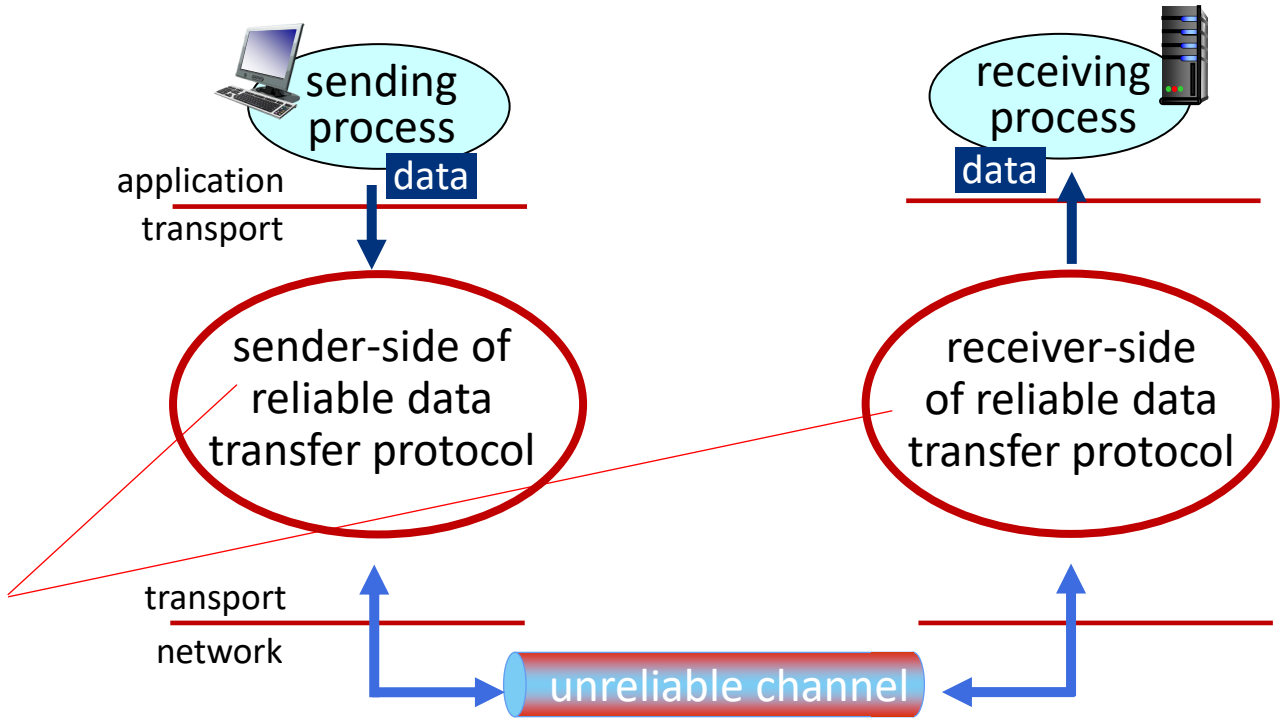
reliable service *abstraction*



Triển khai cụ thể của dịch vụ truyền tin cậy

Principles of reliable data transfer

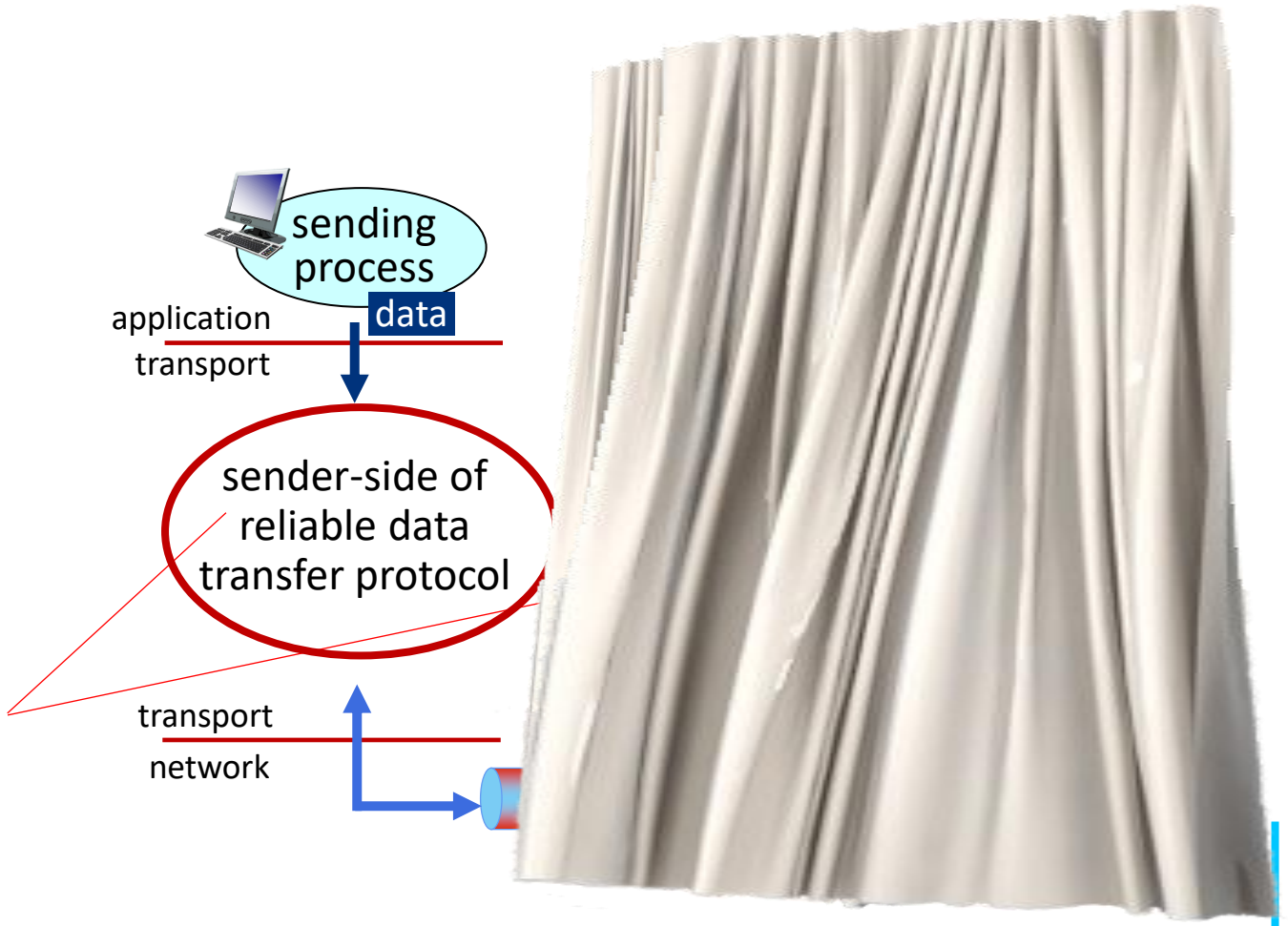
Độ phức tạp của giao thức truyền dữ liệu tin cậy sẽ phụ thuộc nhiều vào đặc điểm của kênh không tin cậy (mất gói, lỗi bit, đảo thứ tự gói)



Triển khai cụ thể của dịch vụ truyền tin cậy

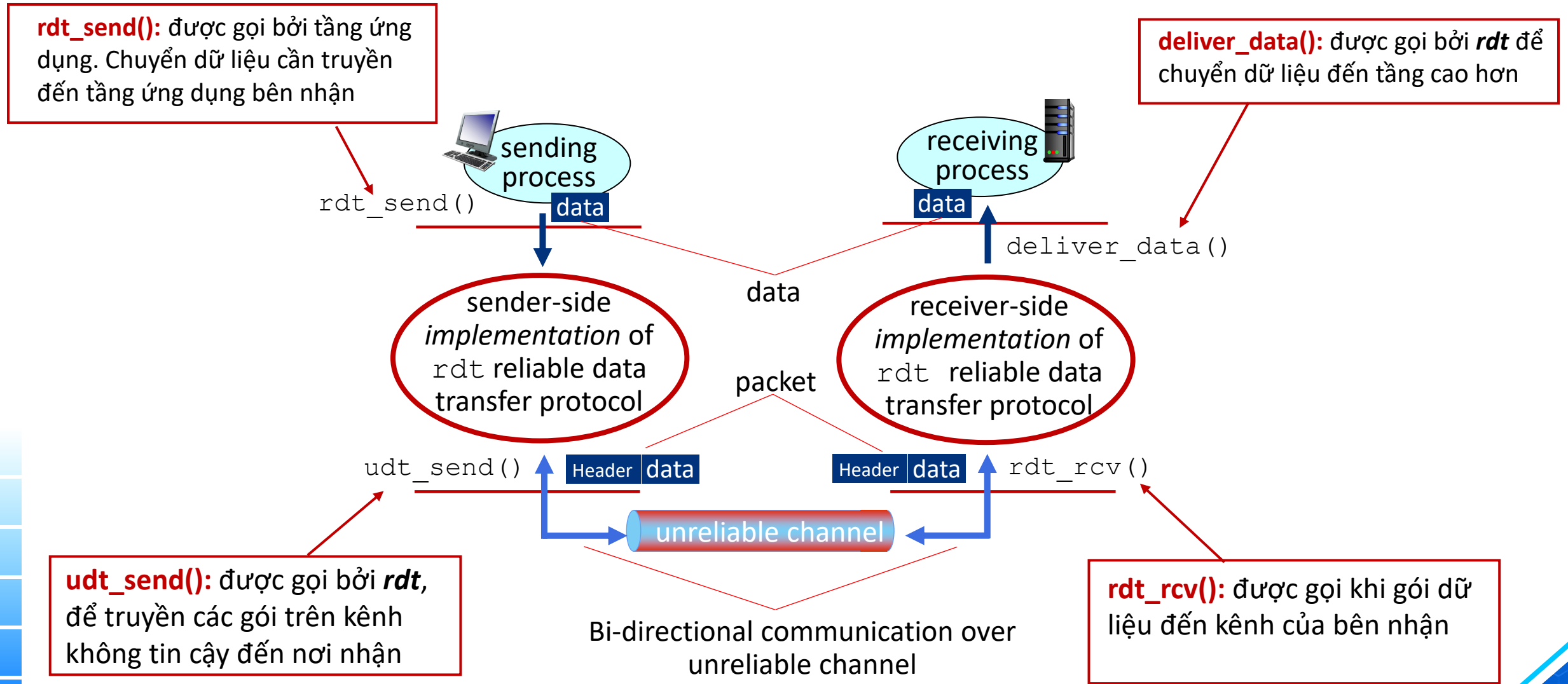
Principles of reliable data transfer

- Bên gửi, bên nhận không biết "trạng thái" của nhau, ví dụ: đã nhận được thông điệp chưa?
  - trừ khi được liên lạc qua các thông điệp



Triển khai cụ thể của dịch vụ truyền tin cậy

Reliable data transfer protocol (rdt): interfaces

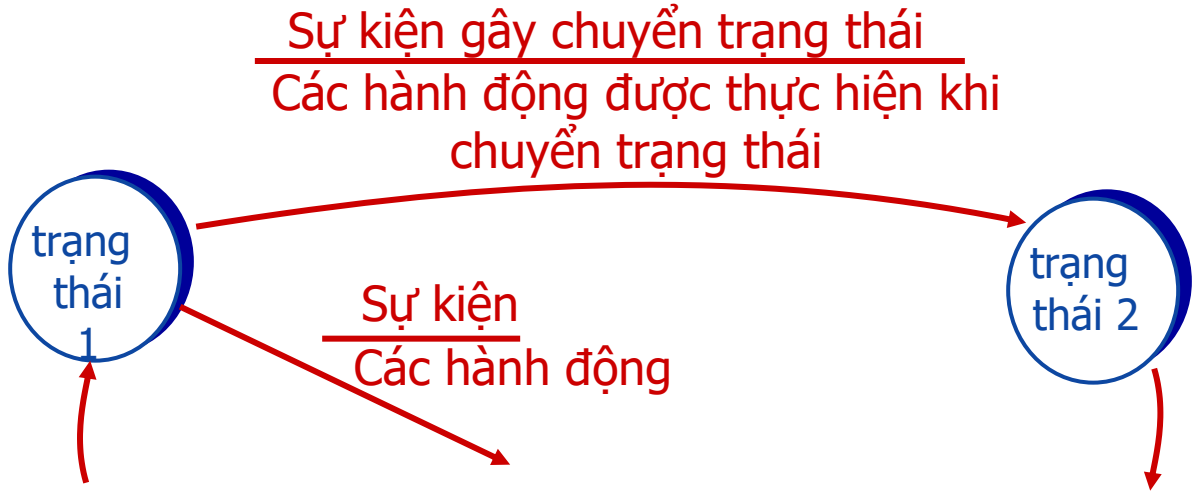


Truyền dữ liệu tin cậy: bắt đầu

Chúng ta sẽ:

- Từng bước phát triển truyền dữ liệu tin cậy (rdt) bên phía người gửi và nhận
- Chỉ xem xét việc truyền dữ liệu theo một chiều (bên gửi đến bên nhận)
- Sử dụng finite state machines (FSM) để xác định bên gửi và nhận

Trạng thái: khi ở "trạng thái" này thì trạng thái kế tiếp được xác định duy nhất bởi sự kiện kế tiếp



Tổng quan

RDT1.0

RDT2.0

RDT2.1

RDT2.2

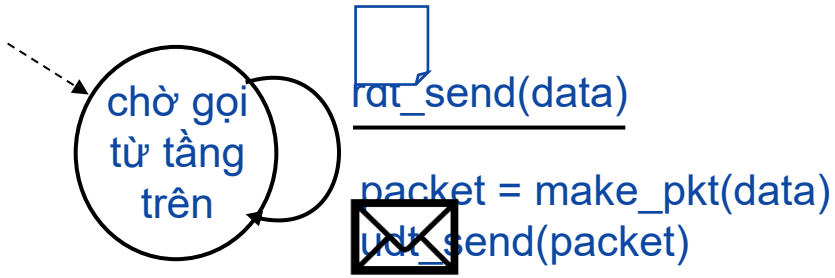
RDT3.0

Truyền dữ liệu tin cậy

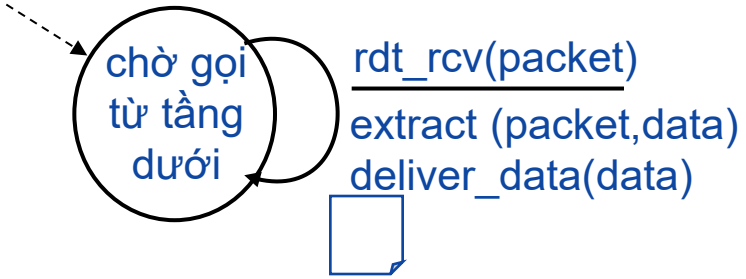
RDT1.0

- Kênh truyền tin cậy hoàn toàn

rdt1.0 - Truyền dữ liệu tin cậy hoàn toàn



bên gửi



bên nhận

Truyền dữ liệu tin cậy

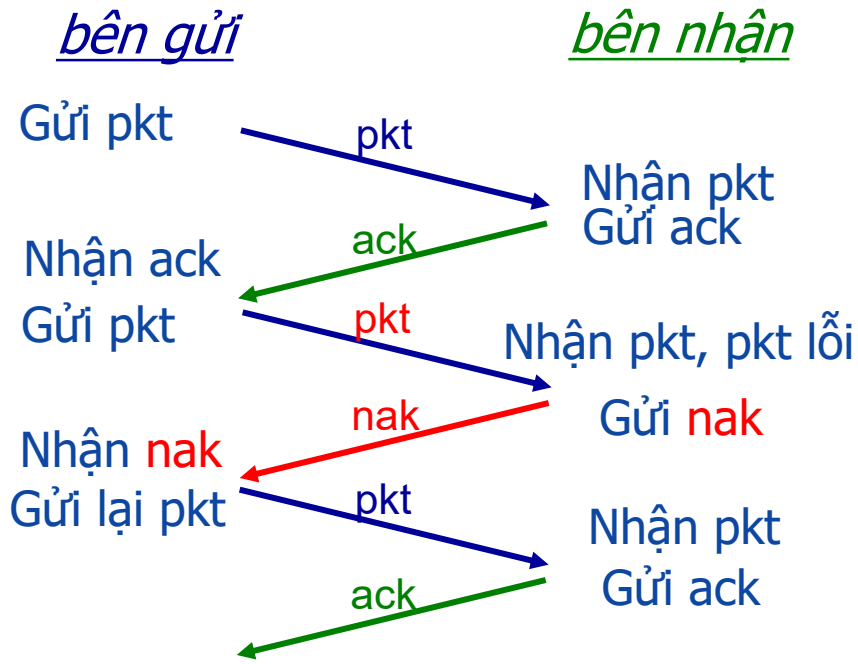
RDT1.0	• Kênh truyền tin cậy hoàn toàn
RDT2.0	• Vấn đề: Kênh truyền có lỗi

rdt2.0: Kênh truyền xảy ra lỗi

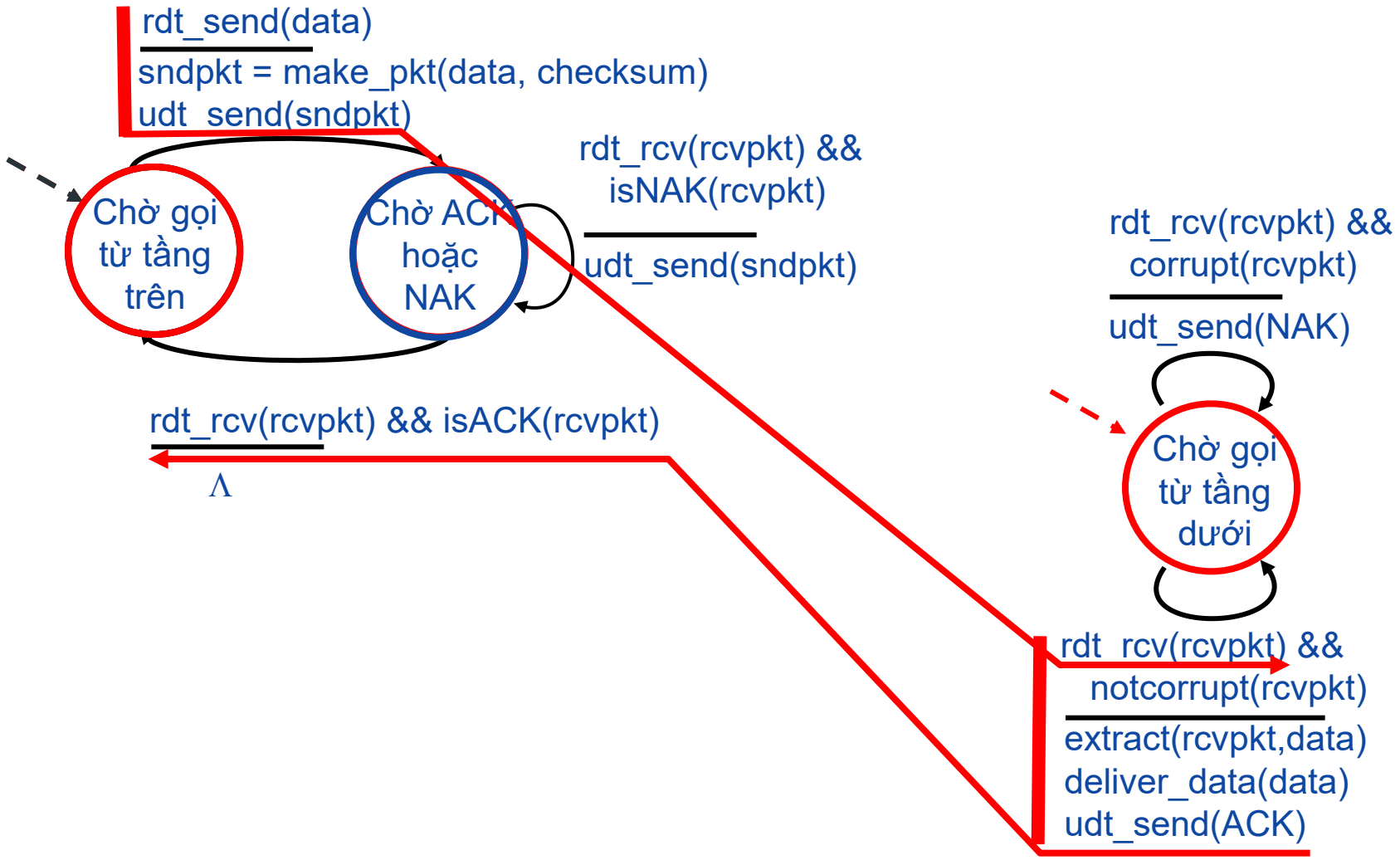
- Kênh cơ bản có thể đảo các bit trong packet
  - checksum để kiểm tra các lỗi
- *Câu hỏi*: làm thế nào để phát hiện và khắc phục lỗi?
  - *acknowledgements (ACKs):* bên nhận báo bên gửi là gói tin đã được nhận thành công.
  - *negative acknowledgements (NAKs):* bên nhận báo bên gửi là gói tin đã bị lỗi
  - Bên gửi sẽ gửi lại gói tin nếu nhận được NAK
- Cơ chế mới trong rdt2.0:
  - Phát hiện lỗi
  - Phản nhận phản hồi cho bên gửi bằng ACK hoặc NAK

Làm thế nào để con người phục hồi "lỗi" trong cuộc trò chuyện?

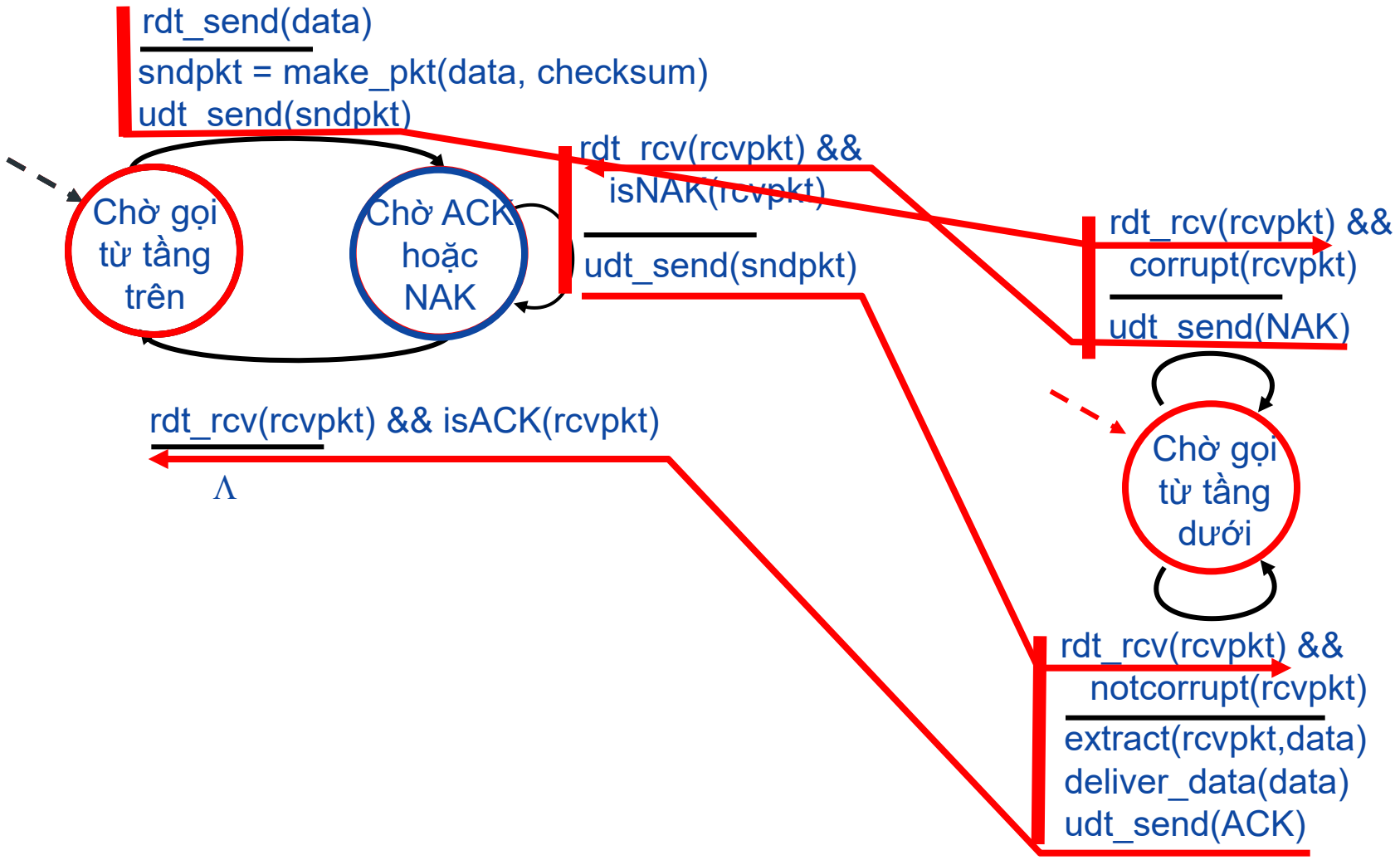
rdt2.0: Kênh truyền xảy ra lỗi



rdt2.0: hoạt động khi không lỗi



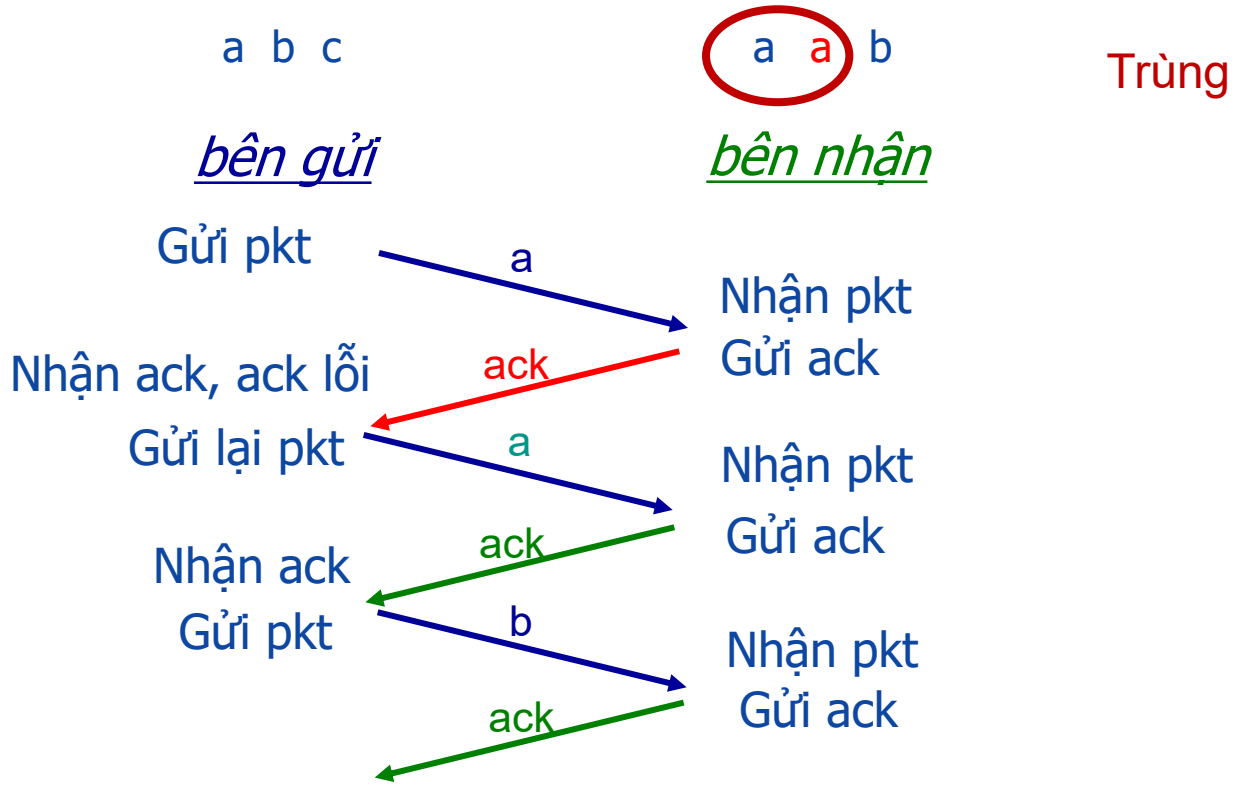
rdt2.0: hoạt động khi có lỗi



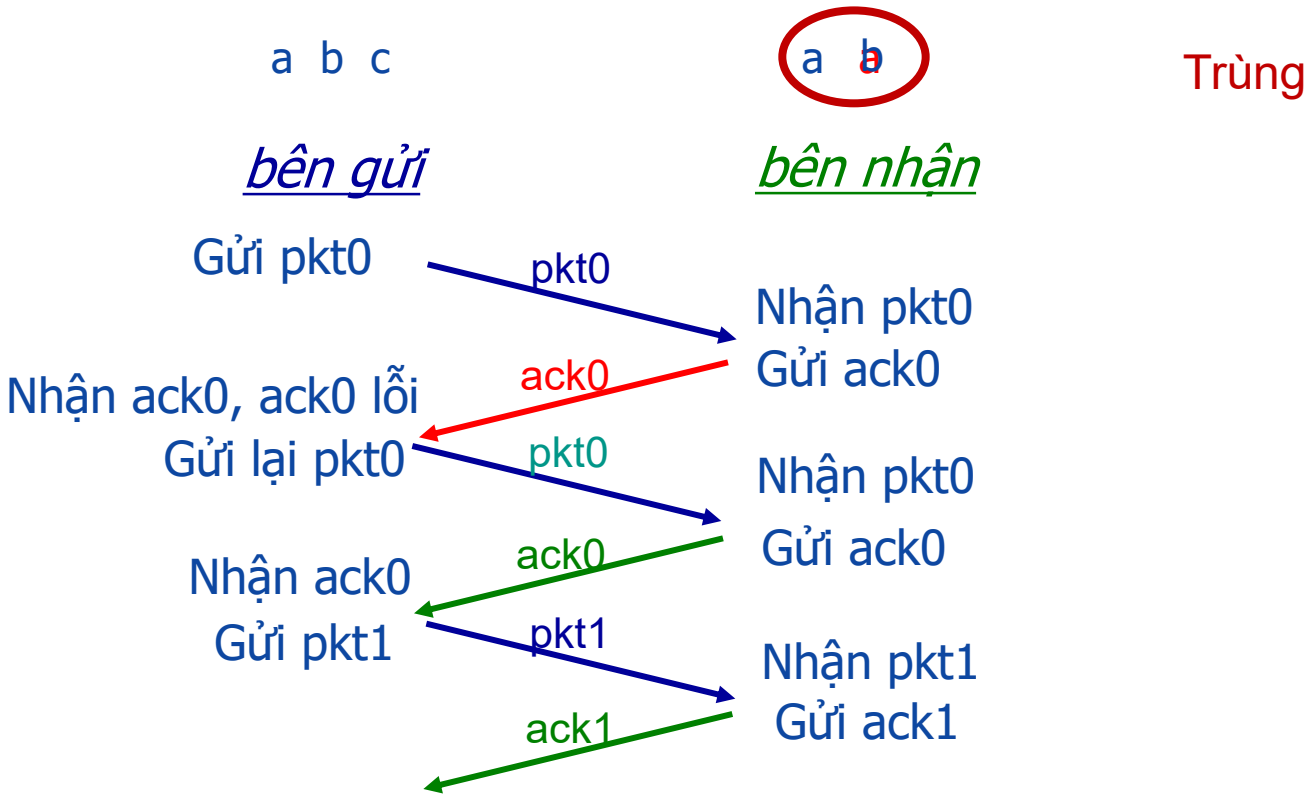
Truyền dữ liệu tin cậy

RDT1.0	• Kênh truyền tin cậy hoàn toàn
RDT2.0	• Vấn đề: Kênh truyền có lỗi • Giải quyết: ACK và NAK
RDT2.1	• Vấn đề: ACK/NAK lỗi

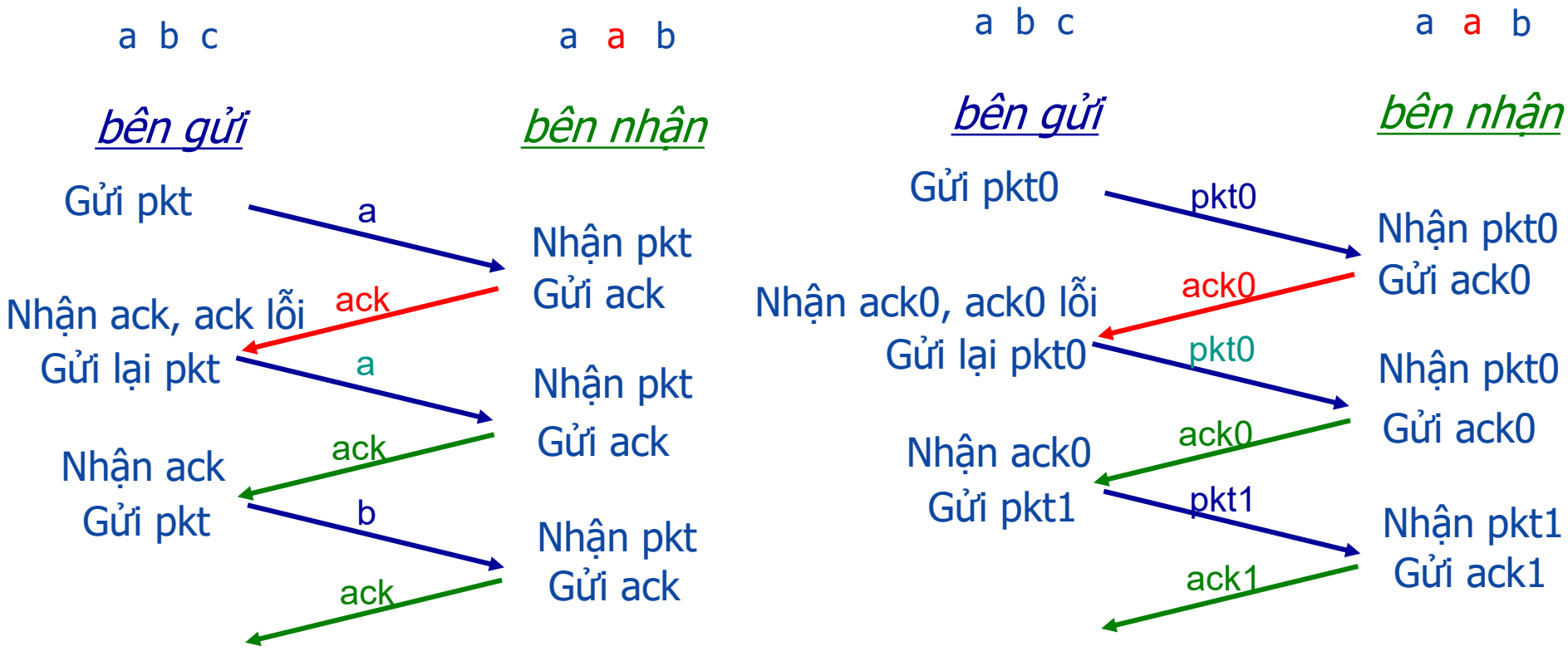
rdt2.0 có lỗ hổng nghiêm trọng!



rdt2.1 bên gửi, xử lý các ACK/NAK bị hỏng



rdt2.1 bên gửi, xử lý các ACK/NAK bị hỏng



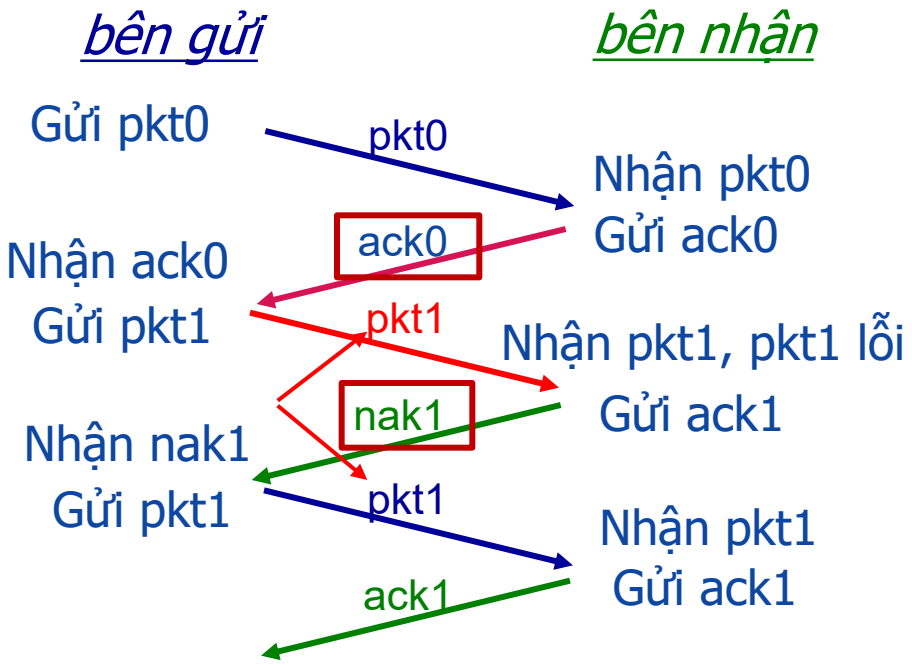
Truyền dữ liệu tin cậy

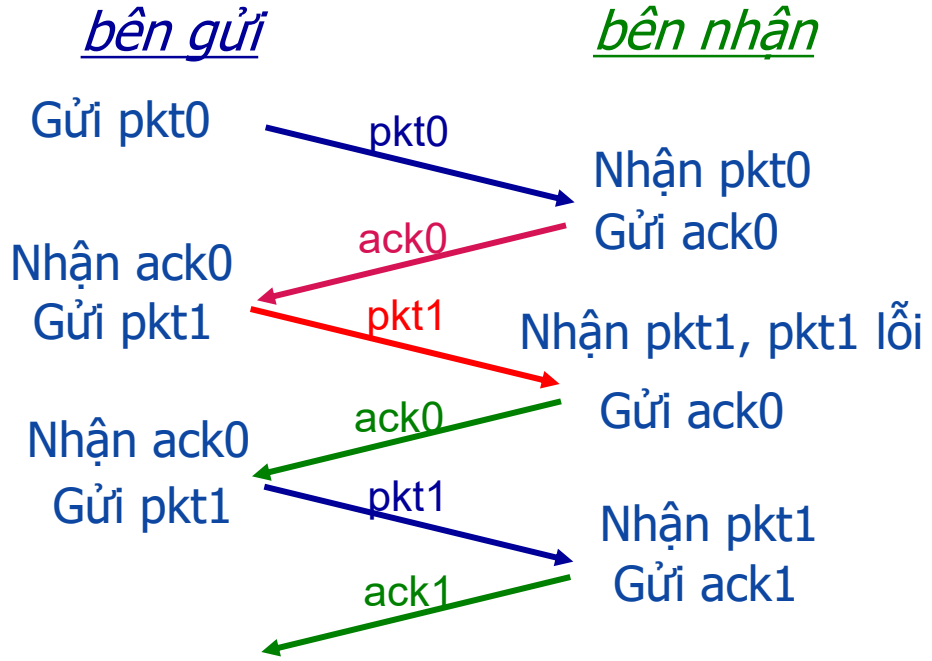
RDT1.0	• Kênh truyền tin cậy hoàn toàn
RDT2.0	• Vấn đề: Kênh truyền có lỗi • Giải quyết: ACK và NAK
RDT2.1	• Vấn đề: ACK/NAK lỗi • Giải quyết: Đánh số gói tin {0,1}
RDT2.2	

rdt2.2: một giao thức không cần NAK

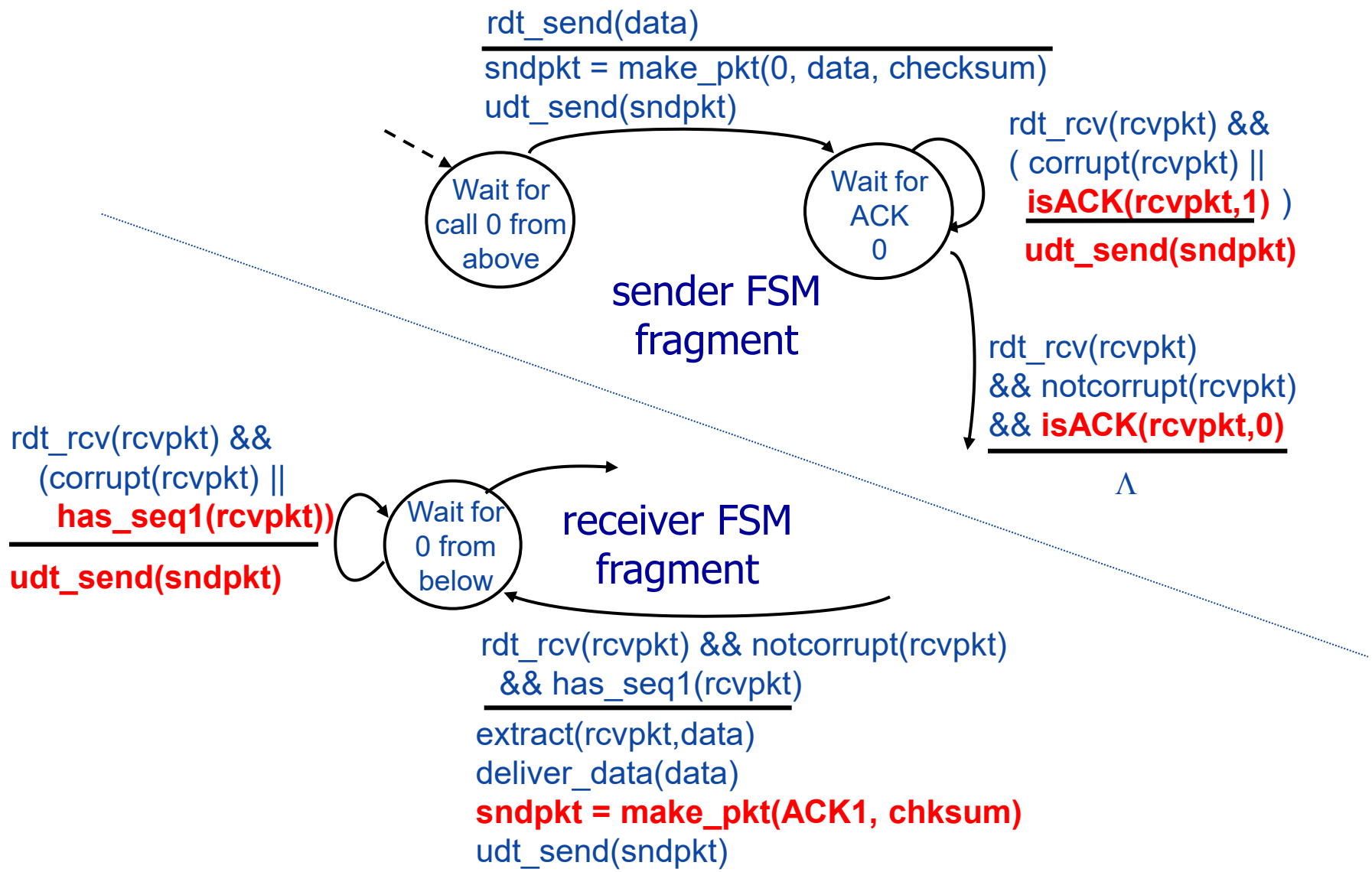
- Chức năng giống như rdt2.1, chỉ dùng các ACK
- Thay cho NAK, bên nhận gửi ACK cho gói cuối cùng được nhận thành công
  - Bên nhận phải ghi rõ số thứ tự của gói vừa được ACK
- Khi nhận ACK trùng lặp, bên gửi truyền lại gói chưa được xác nhận; tác dụng này tương tự khi nhận NAK

rdt2.2: một giao thức không cần NAK





rdt2.2: các phần bên nhận và gửi



Truyền dữ liệu tin cậy

RDT1.0	• Kênh truyền tin cậy hoàn toàn
RDT2.0	• Vấn đề: Kênh truyền có lỗi • Giải quyết: ACK và NAK
RDT2.1	• Vấn đề: ACK/NAK lỗi • Giải quyết: Đánh số gói tin {0,1}
RDT2.2	• Không cần NAK
RDT3.0	• Vấn đề: Mất gói tin

rdt3.0: các kênh với lỗi và mất mát

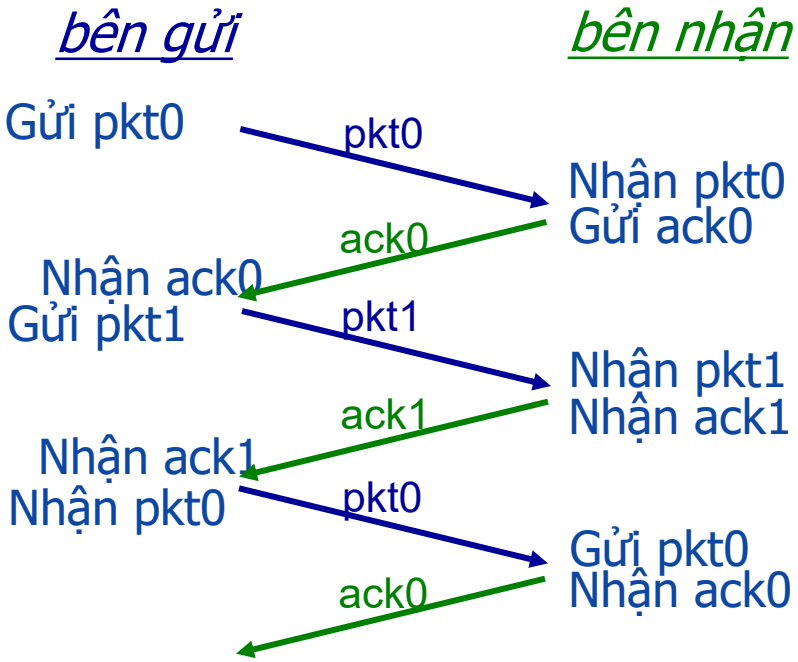
Cách tiếp cận: bên gửi chờ ACK trong khoảng thời gian "hợp lý"

- Truyền lại nếu không nhận được ACK trong khoảng thời gian này
- Nếu gói (hoặc ACK) chỉ trễ (không mất):
  - Việc truyền lại sẽ gây trùng, nhưng số thứ tự đã xử lý trường hợp này
  - Bên nhận phải xác định số thứ tự của gói vừa gửi ACK
- Cần sử dụng bộ định thì (timer)

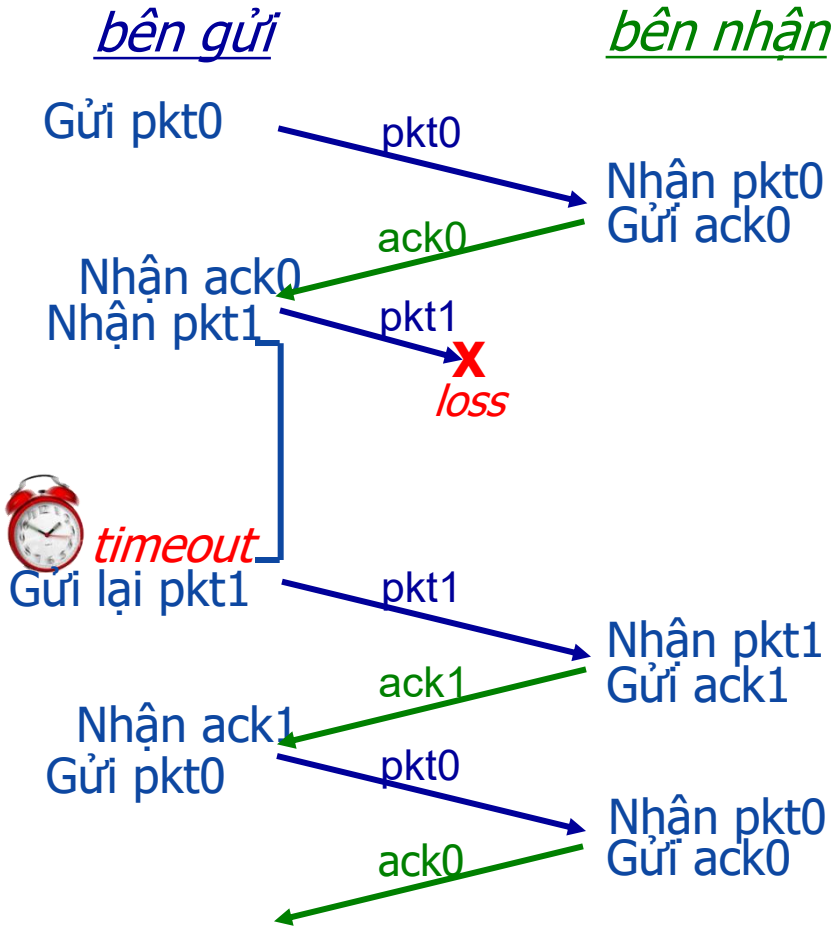
Giả định mới: kênh truyền cũng có thể làm mất gói dữ liệu lẫn ACK

- checksum, số thứ tự, các ACK, việc truyền lại sẽ hỗ trợ… nhưng không đủ

Hoạt động của rdt3.0

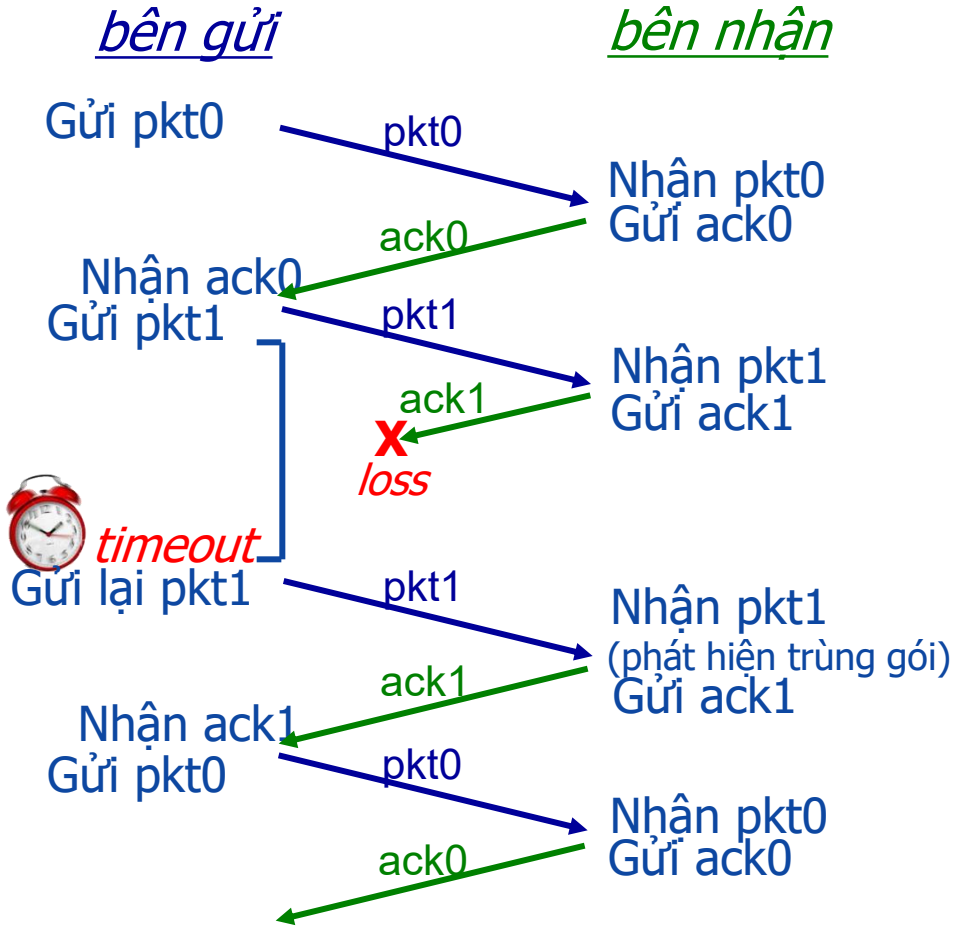


(a) Không mất mát

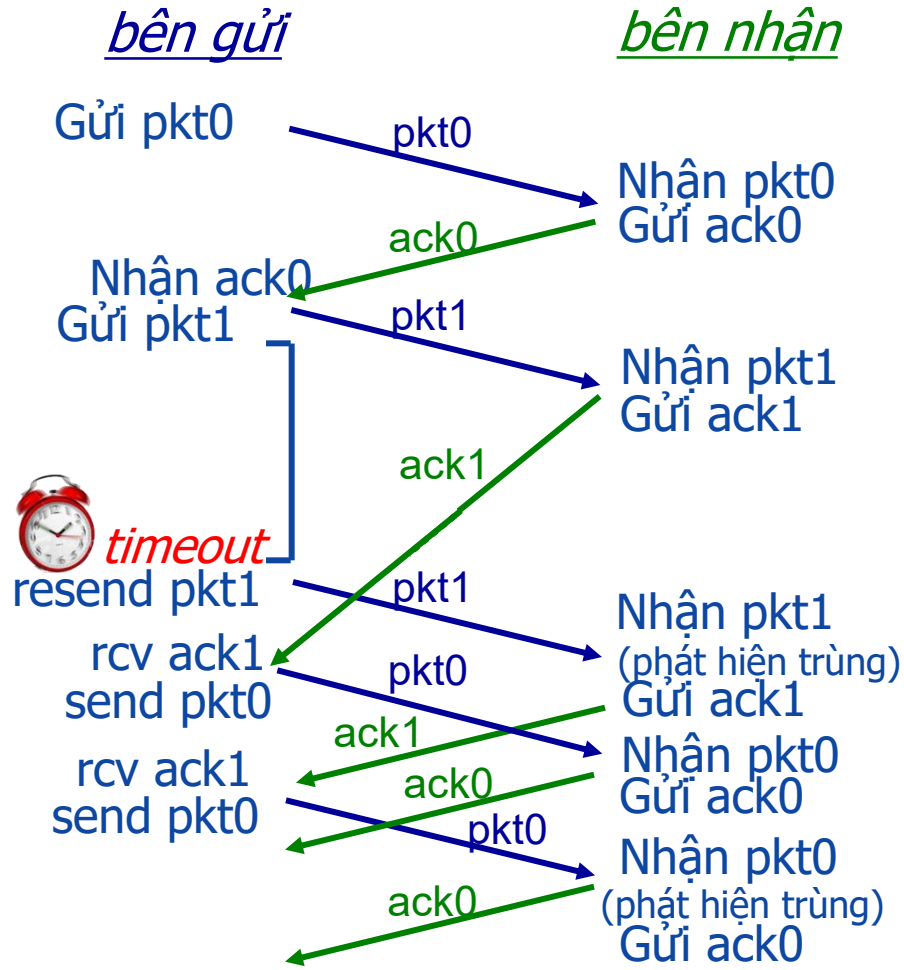


(b) Mất gói

Hoạt động của rdt3.0



(c) Mất ACK



(d) Thời gian chờ quá ngắn / delayed ACK

Tổng kết về Truyền dữ liệu tin cậy

RDT1.0	• Kênh truyền tin cậy hoàn toàn
RDT2.0	• Vấn đề: Kênh truyền có lỗi • Giải quyết: ACK và NAK
RDT2.1	• Vấn đề: ACK/NAK lỗi • Giải quyết: Đánh số gói tin {0,1}
RDT2.2	• Không cần NAK
RDT3.0	• Vấn đề: Mất gói tin • Giải quyết: Thêm bộ định thì (timer)

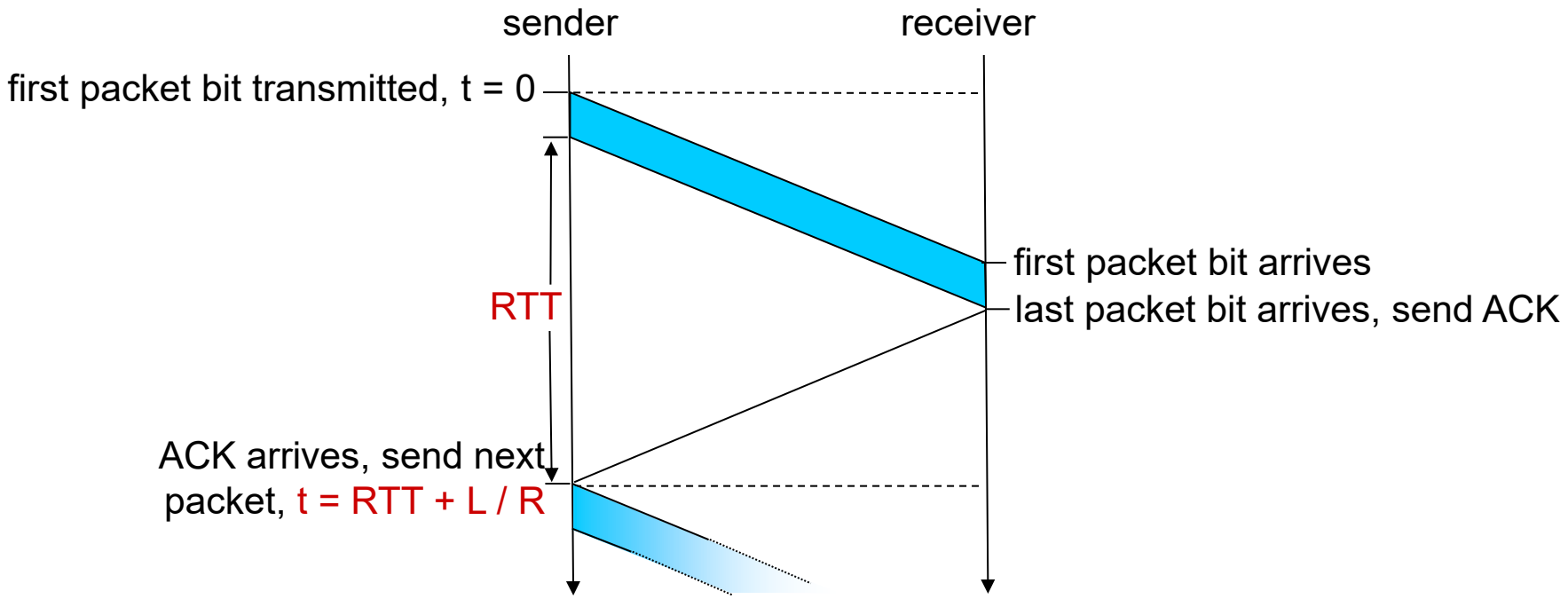
Hiệu suất của rdt3.0 (stop-and-wait)

- U_{sender}: *utilization* – tỉ lệ thời gian truyền của bên gửi
- Ví dụ: 1 Gbps link, truyền một gói tin 8000 bit trên kênh truyền có độ trễ lan truyền 15 ms

  - Thời gian truyền gói tin đến kênh truyền:

$$D_{trans} = \frac{L}{R} = \frac{8000\ bits}{10^9\ bits/sec} = 8\ microseconds$$

rdt3.0: hoạt động stop-and-wait

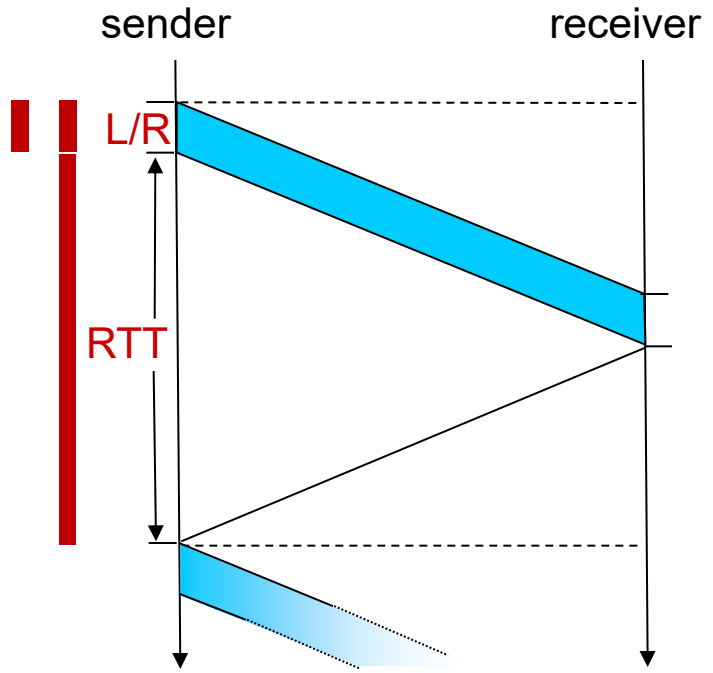


rdt3.0: hoạt động stop-and-wait

$$U_{sender} = \frac{L / R}{RTT + L / R}$$

$$= \frac{.008}{30.008}$$

$$= 0.00027$$

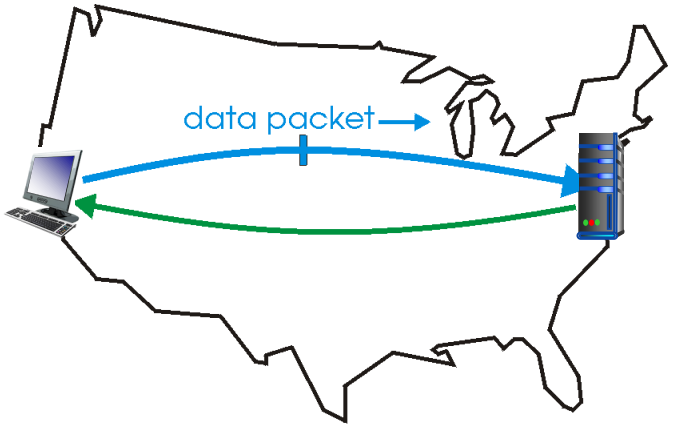


- Hiệu suất giao thức rdt 3.0 kém
- Giao thức stop-and-wait làm hạn chế hiệu suất của đường truyền

rdt3.0: hoạt động giao thức "pipelined"

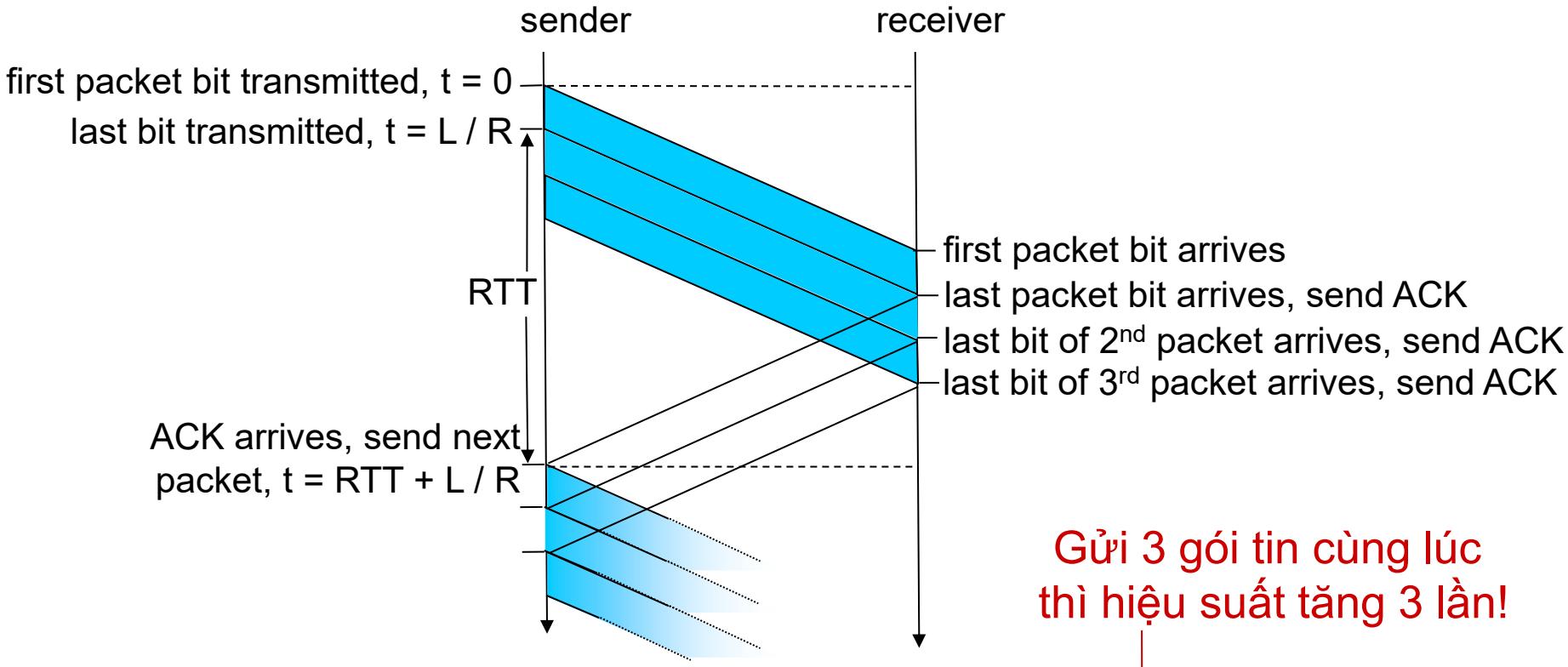
pipelining: cho phép bên gửi truyền nhiều gói liên tiếp mà không phải chờ ACK của từng gói

- Dãy số thứ tự phải được tăng lên
- Đưa vào bộ đệm tại bên gửi hoặc bên nhận



(a) a stop-and-wait protocol in operation

Pipelining: tăng hiệu suất

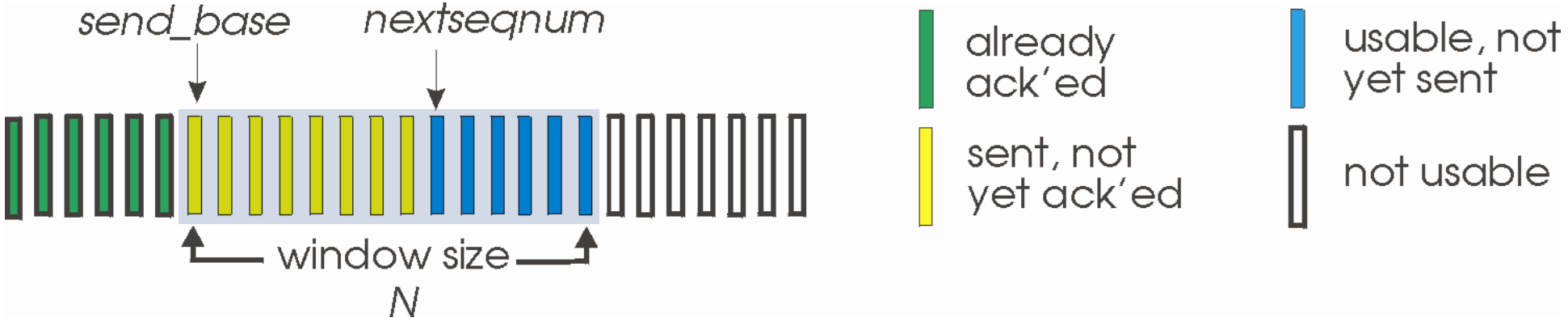


$$U_{sender} = \frac{3L/R}{RTT + L/R} = \frac{.0024}{30.008} = 0.00081$$

Gửi 3 gói tin cùng lúc
thì hiệu suất tăng 3 lần!

Go-Back-N: bên gửi

- Bên gửi: gửi tối đa N packets liên tiếp mà không cần chờ ACK ("window")
  - k-bit seq # in pkt header

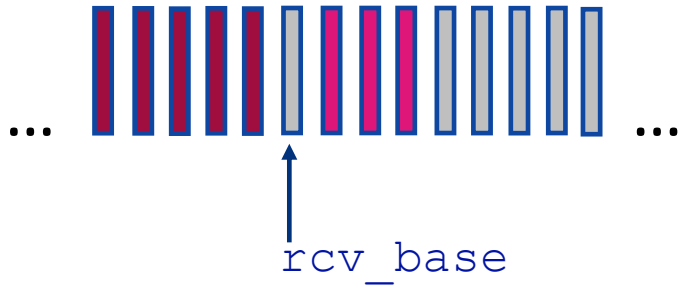


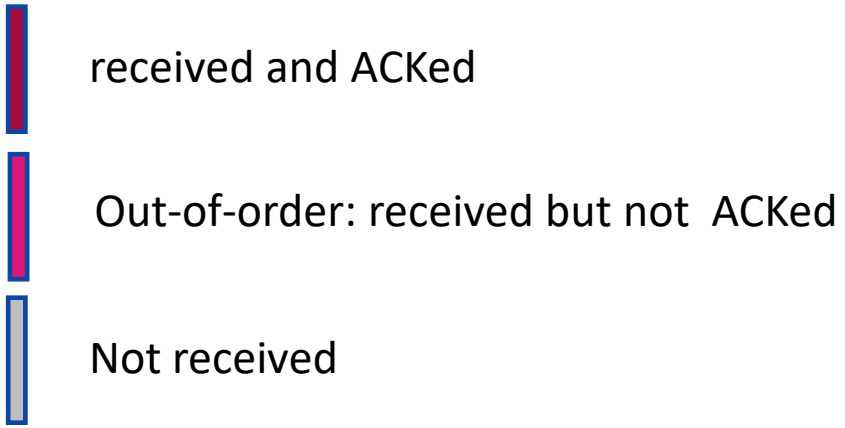
- Khi nhận ACK(n) => ACK cho tất cả các packet đã gửi tính tới gói tin có seq là n (*cumulative ACK* – ACK tích lũy)
- Khởi động bộ định thời cho gói chưa được ACK có số thứ tự nhỏ nhất
- *Khi gói n hết thời gian chờ, gửi lại gói n và tất cả các gói sau nó đang nằm trong cửa sổ.*

Go-Back-N: bên nhận

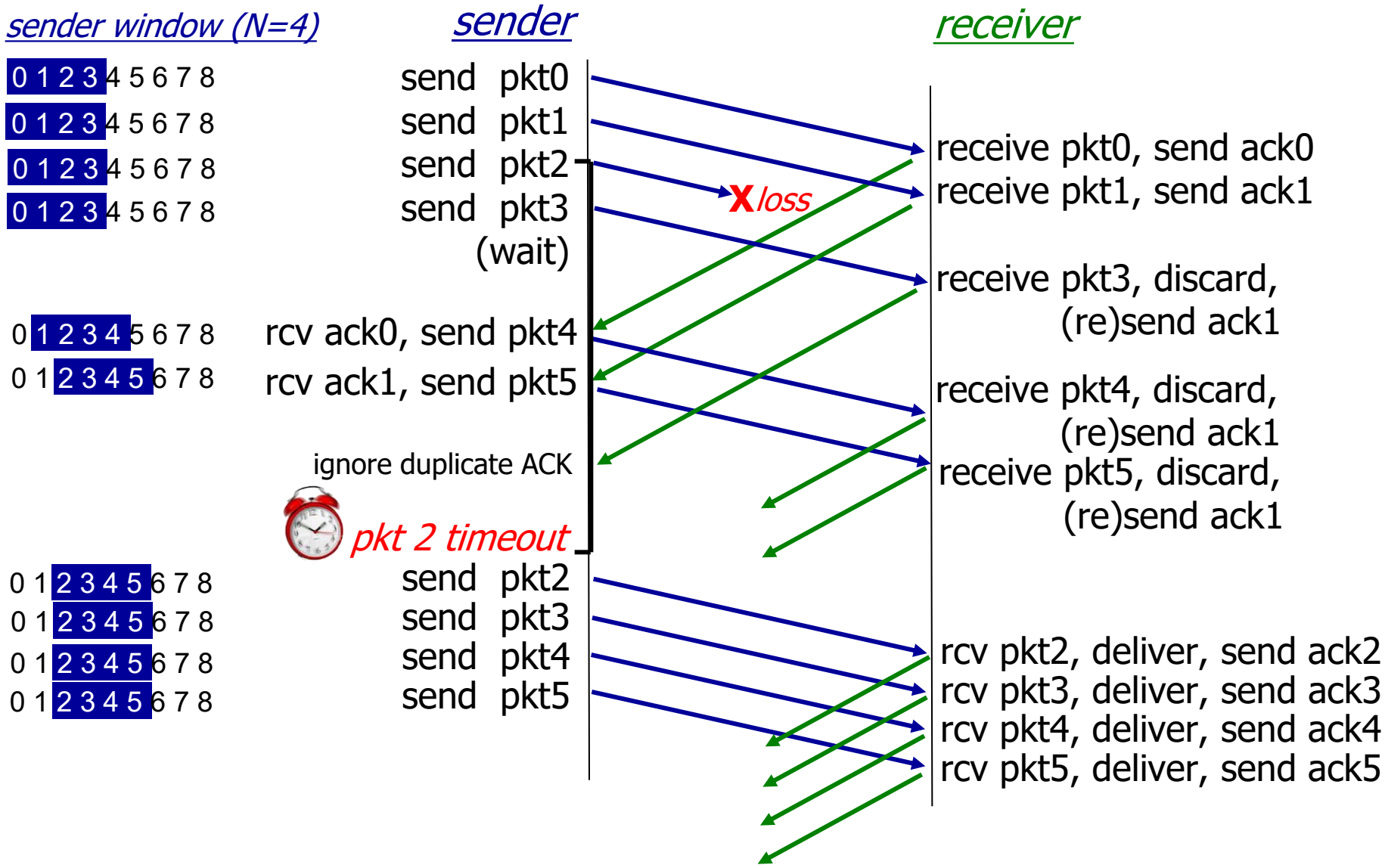
- Nhận packet đúng thứ tự: gửi ACK tương ứng với packet đúng thứ tự có STT cao nhất
- Nhận packet không đúng thứ tự:
  - Có thể hủy hoặc đưa vào buffer: tùy vào cách triển khai
  - Gửi lại ACK tương ứng với packet đúng thứ tự có số TT cao nhất

Receiver view of sequence number space:





Go-Back-N: các hành động



Go-Back-N in action

Insert Web Page

This app allows you to insert secure web pages starting with https:// into the slide deck. Non-secure web pages are not supported for security reasons.

Please enter the URL below.

https:// media.pearsoncmg.com/aw/ecs_kurose_compnetwork_7/cw/content/interactiveanimations/go-back-n-protocol/index.html

Note: Many popular websites allow secure access. Please click on the preview button to ensure the web page is accessible.

Web Viewer Terms | Privacy & Cookies Preview

Selective repeat

- Bên nhận gửi ACK cho từng packet nhận đúng và lưu vào bộ đệm các gói đến sai thứ tự
- Bên gửi bật cơ chế timeout/gửi lại các packet không được ACK
- Cơ chế window tại bên gửi
  - Cho phép gửi liên tiếp N gói có số thứ tự tăng dần
  - Giới hạn số gói đã gửi nhưng chưa được ACKs

Selective repeat

Insert Web Page

This app allows you to insert secure web pages starting with https:// into the slide deck. Non-secure web pages are not supported for security reasons.

Please enter the URL below.

https:// media.pearsoncmg.com/aw/ecs_kurose_compnetwork_7/cw/content/interactiveanimations/selective-repeat-protocol/index.html

Note: Many popular websites allow secure access. Please click on the preview button to ensure the web page is accessible.

Web Viewer Terms | Privacy & Cookies Preview

Selective repeat: bên gửi và bên nhận

Bên gửi

Dữ liệu từ tầng trên:

- Nếu vẫn còn số TT trong window thì gửi gói tin

timeout(*n*):

- Gửi lại packet, thiết lập lại timer

ACK(*n*) trong [sendbase,sendbase+N-1]:

- Đánh dấu packet *n* được nhận
- Nếu n là nhỏ nhất trong các gói tin không được ACK, dịch chuyển window lên vị trí kế tiếp

Bên nhận

Gói tin trong [rcvbase, rcvbase+N-1]

- Gửi ACK(*n*)
- Không đúng thứ tự: buffer
- Đúng thứ tự: chuyển tiếp (cả các packet được lưu trong buffer), rồi dịch chuyển window lên vị trí kế tiếp

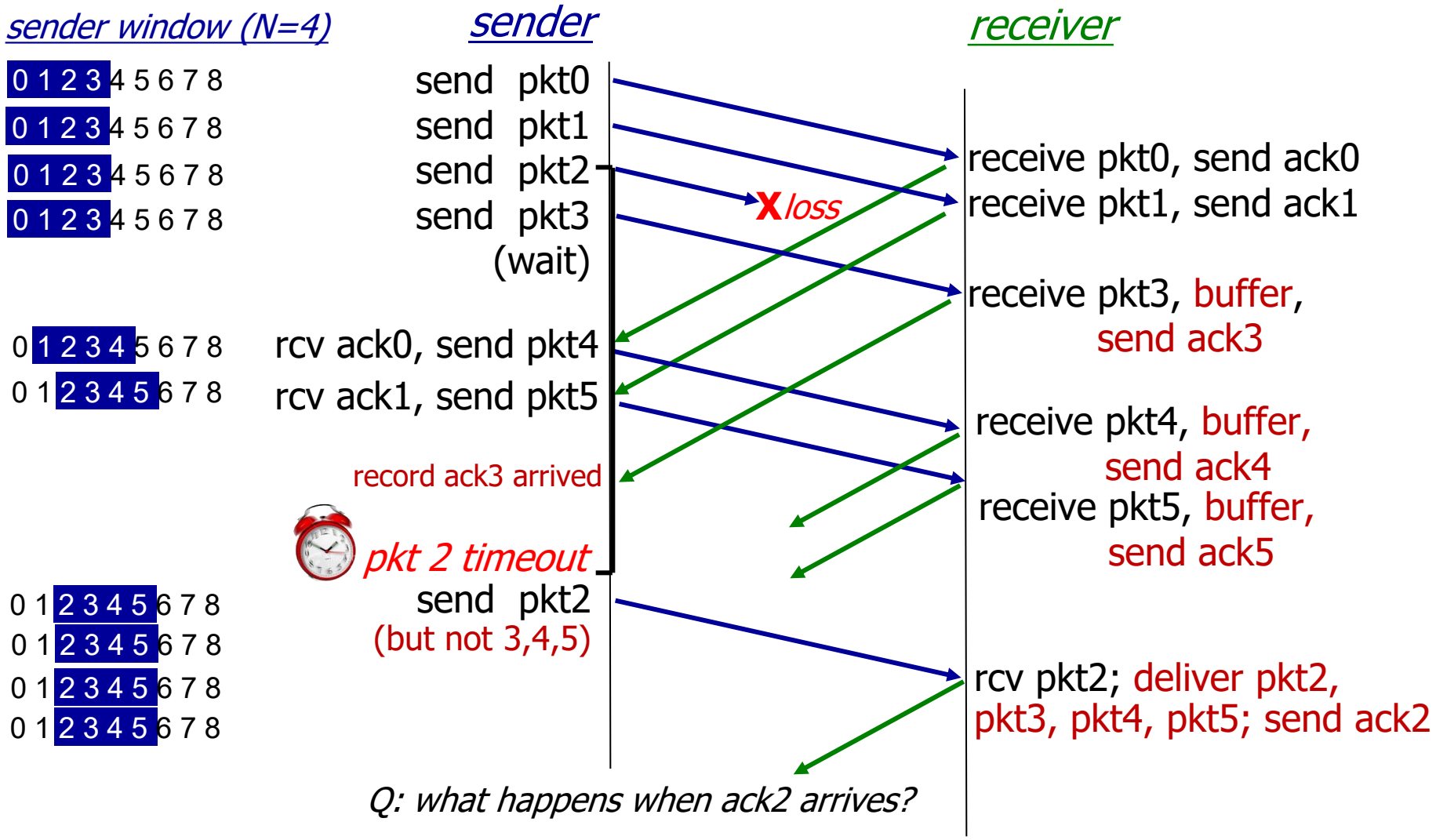
Gói tin trong [rcvbase-N,rcvbase-1]

- ACK(*n*)

Khác:

- Loại bỏ

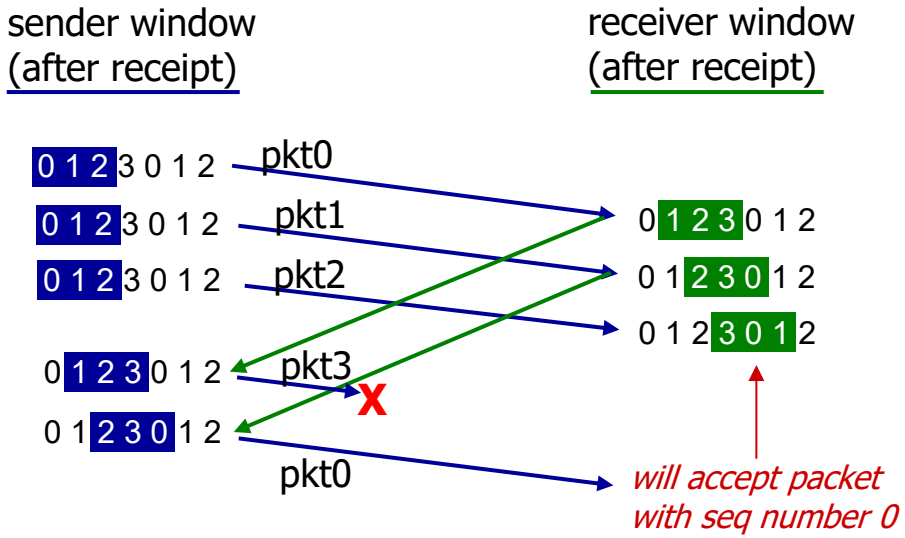
Selective Repeat: các hành động



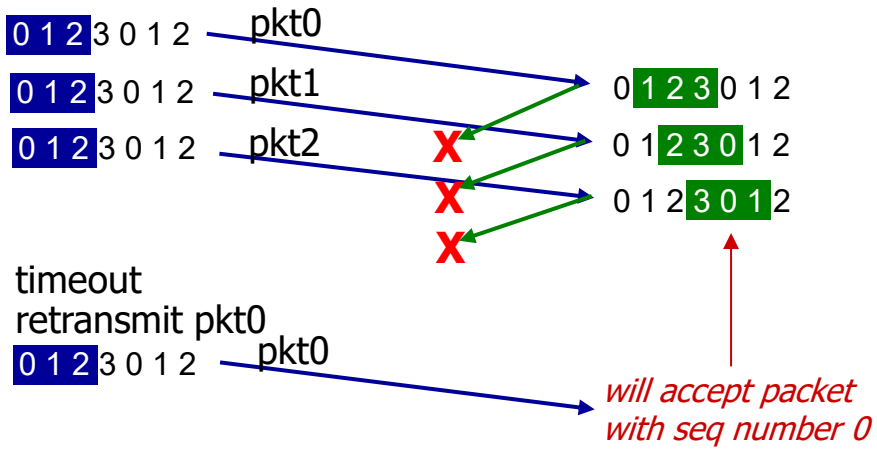
Selective repeat: vấn đề

example:

- seq #s: 0, 1, 2, 3 (base 4 counting)
- window size=3



(a) no problem



(b) oops!

Nội dung

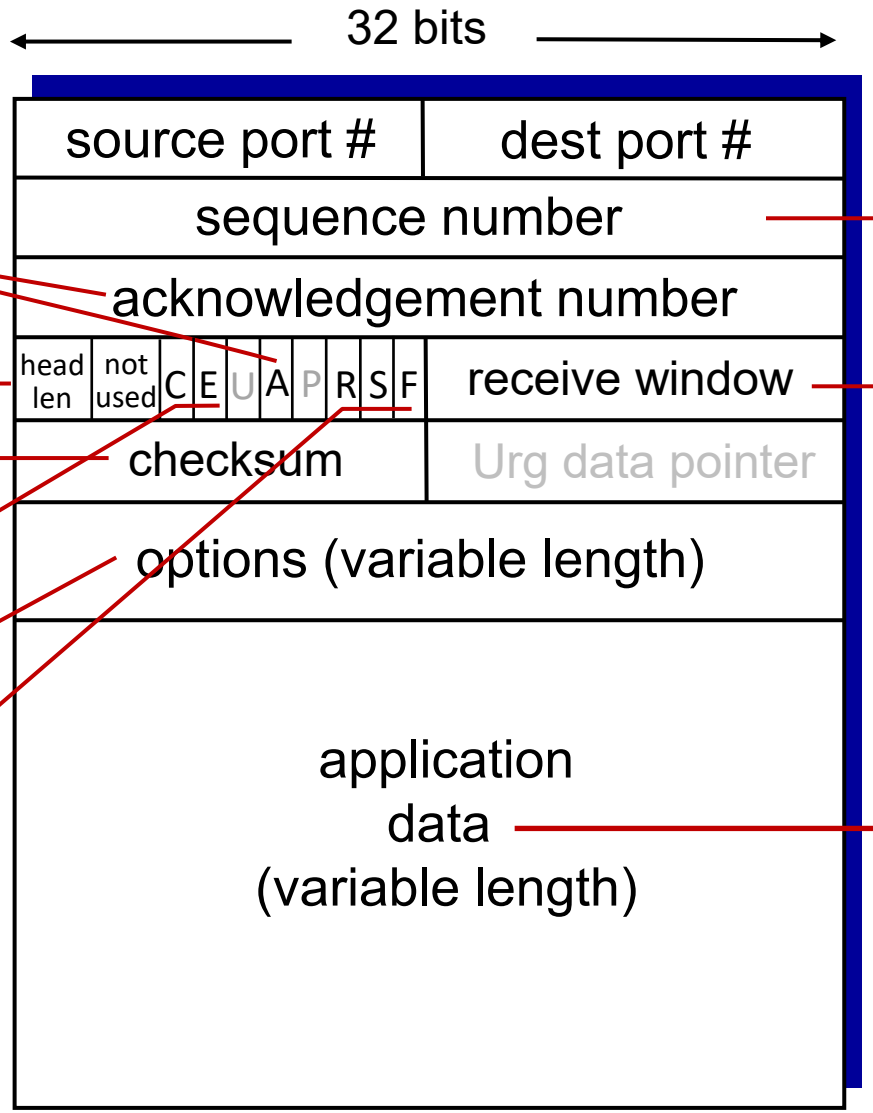
- Các dịch vụ tầng vận chuyển
- Multiplexing and demultiplexing
- UDP
- Nguyên lý truyền tin cậy
- TCP
- TCP - Điều khiển tắc nghẽn
- Sự phát triển của các tính năng của tầng vận chuyển



TCP: tổng quan RFCs: 793,1122, 2018, 5681, 7323

- point-to-point:
  - 1 bên gửi và 1 bên nhận
- Tin cậy, dữ liệu đúng thứ tự theo byte
- "full duplex data":
  - Luồng dữ liệu 2 chiều trong cùng một kết nối
  - MSS: maximum segment size (độ dài tối đa của 1 segment)
- ACKs tích lũy
- Cơ chế pipelining:
  - Điều khiển tắc nghẽn và điều khiển luồng với "window size"
- Hướng kết nối:
  - handshaking (trao đổi các thông điệp thiết lập và đóng kết nối)
- "flow controlled":
  - Bên gửi điều chỉnh lượng dữ liệu truyền để không làm tràn bộ đệm của bên nhận

Cấu trúc của TCP segment



ACK: seq # of next expected byte; A bit: this is an ACK

length (of TCP header)

Internet checksum

C, E: congestion notification

TCP options

RST, SYN, FIN: connection management

segment seq #: counting bytes of data into bytestream (not segments!)

flow control: # bytes receiver willing to accept

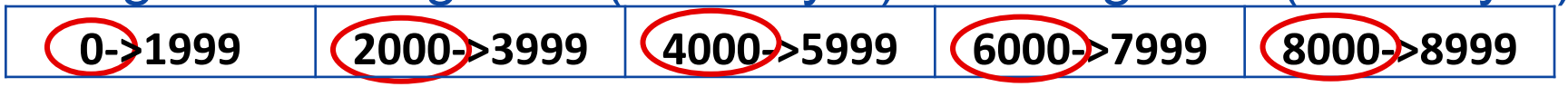
data sent by application into TCP socket

TCP: Reliable data transfer

- Truyền theo thứ tự: Sequence number, ACK
- Truyền lại:
  - Segment lỗi: bị loại bỏ sau khi kiểm tra checksum
  - Segment bị mất: timeout

TCP sequence numbers, ACKs: ví dụ

- Thông điệp tầng ứng dụng: 9000 bytes (byte #0 -> byte #8999)
- Mỗi segment chứa tối đa 2000 bytes
- => 5 segment: 4 segment (2000 byte) và 1 segment (1000 byte)



Sender - Sequence number

Seq = 0, length = 2000B

You need byte from 2000, I'll send you from #2000

S = 2000, length = 2000B

S = 4000, length = 2000B

S = 6000, length = 2000B

S = 8000, length = 1000B

Receiver ACK

ACK = 2000

I receive until #1999, I want byte from #2000

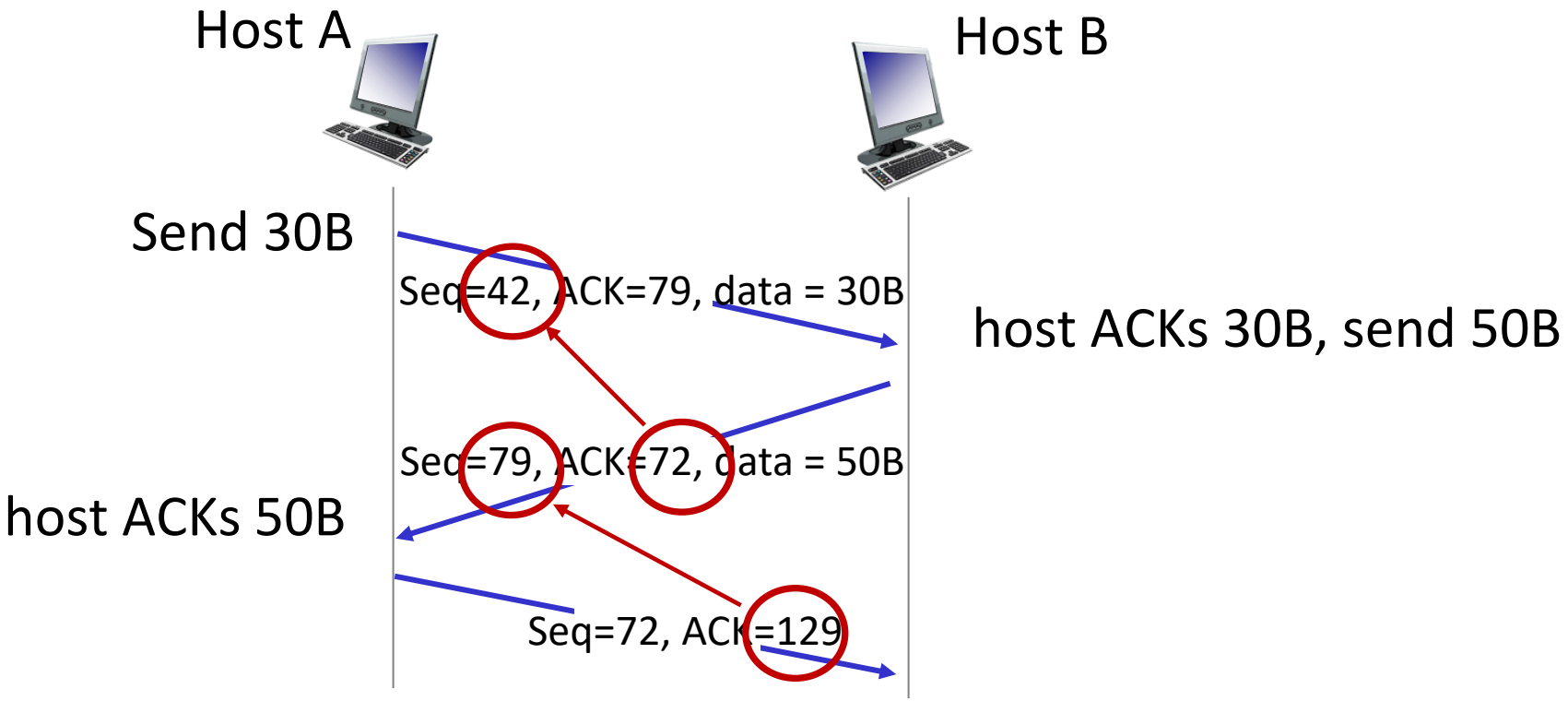
ACK = 4000

ACK = 6000

ACK = 8000

ACK = 9000

TCP sequence numbers, ACKs



TCP round trip time, timeout

Q: TCP xác định thời gian timeout thế nào?

- Phải lâu hơn RTT, nhưng RTT có thể thay đổi giữa các lần truyền!
- *Nếu quá ngắn:* gửi lại không cần thiết
- *Nếu quá lâu:* chậm phản ứng với việc mất segment

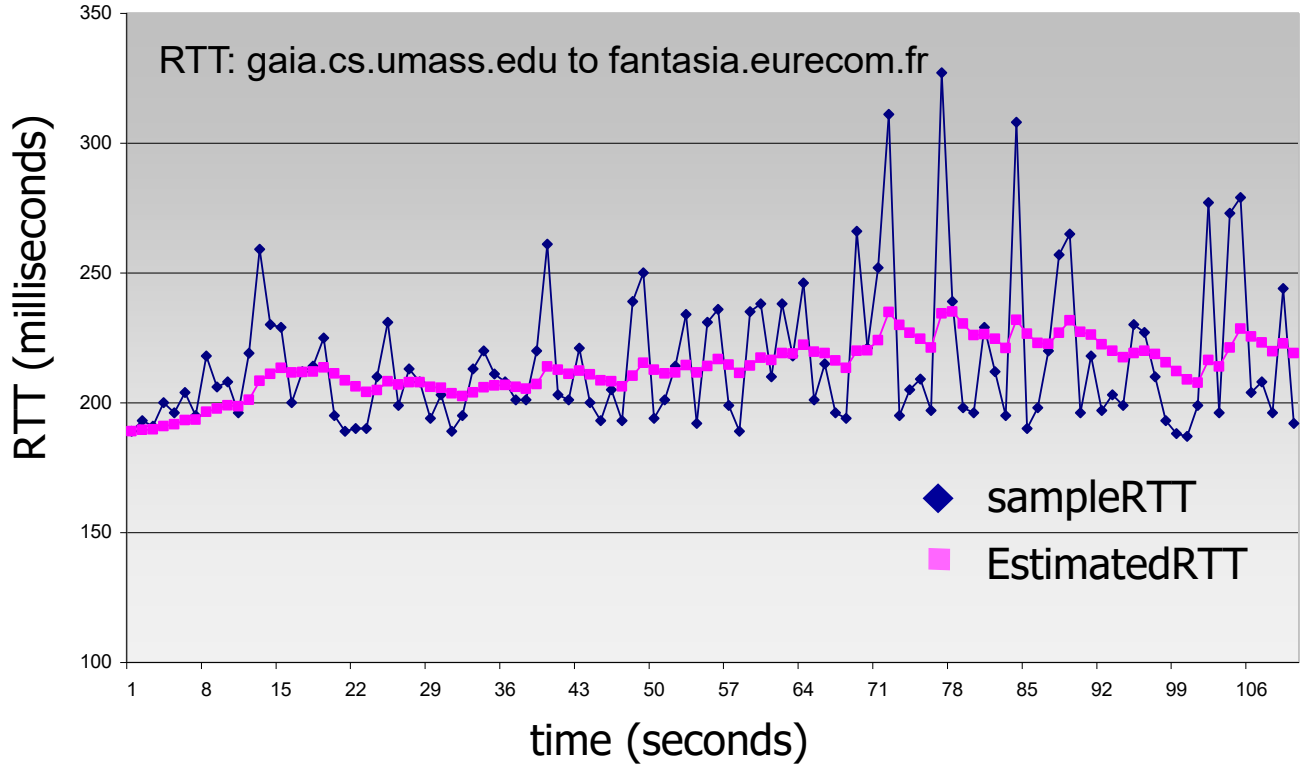
Q: Làm thế nào để dự đoán RTT?

- `SampleRTT:` đo thời gian truyền segment cho đến khi nhận được ACK
  - Không sử dụng các segment được truyền lại để tính SampleRTT
  - `SampleRTT` sẽ khác nhau với mỗi lần đo
  - Tính trung bình nhiều lần đo chứ không chỉ dựa vào lần đo hiện tại

TCP round trip time, timeout

EstimatedRTT = (1- α)*EstimatedRTT + α*SampleRTT

- Cơ chế EWMA: exponential weighted moving average
- Ảnh hưởng của các lần đo trước sẽ giảm dần theo hệ số α
- Giá trị tiêu biểu: α = 0.125



TCP round trip time, timeout

- Khoảng timeout: **EstimatedRTT** cộng với "safety margin" – biên độ an toàn

```
TimeoutInterval = EstimatedRTT + 4*DevRTT
```



estimated RTT

"safety margin"

- **DevRTT**: EWMA of **SampleRTT** deviation from **EstimatedRTT**:

```
DevRTT = (1-β)*DevRTT + β*|SampleRTT-EstimatedRTT|
```

(typically, β = 0.25)

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

TCP Sender (tinh giản)

Sự kiện: Nhận dữ liệu từ tầng ứng dụng

- Tạo segment với STT seq #
- seq #: STT của byte đầu tiên trong segment
- Bắt đầu tính thời gian với segment cũ nhất

Sự kiện: timeout

- Gửi lại segment bị timeout
- Bắt đầu tính lại thời gian

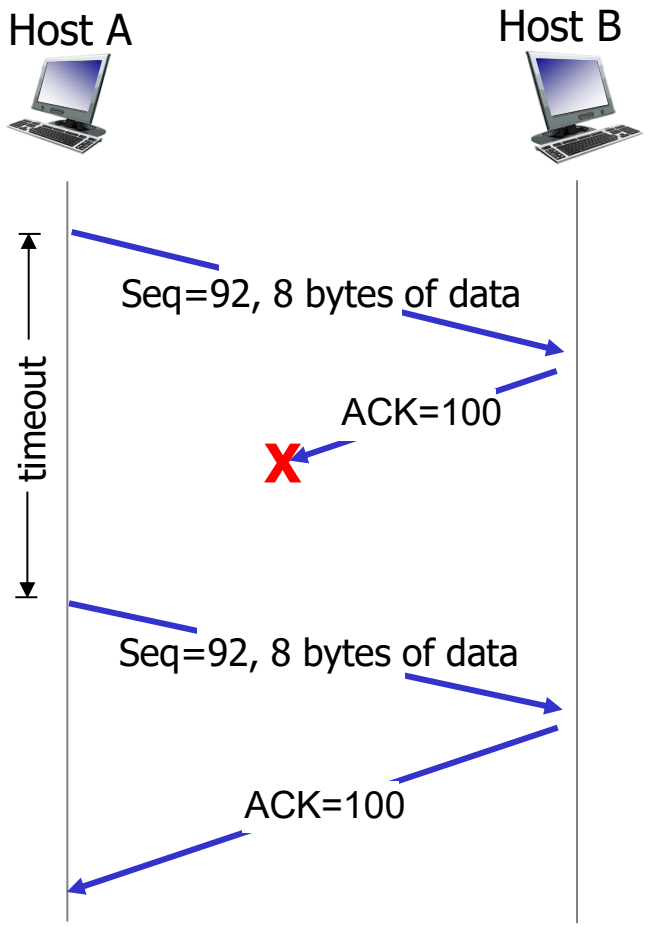
Sự kiện: nhận được ACK

- Nếu ACK xác nhận cho các segment chưa được ACK
  - Cập nhật trạng thái
  - Tính lại thời gian với các segment chưa được ACK

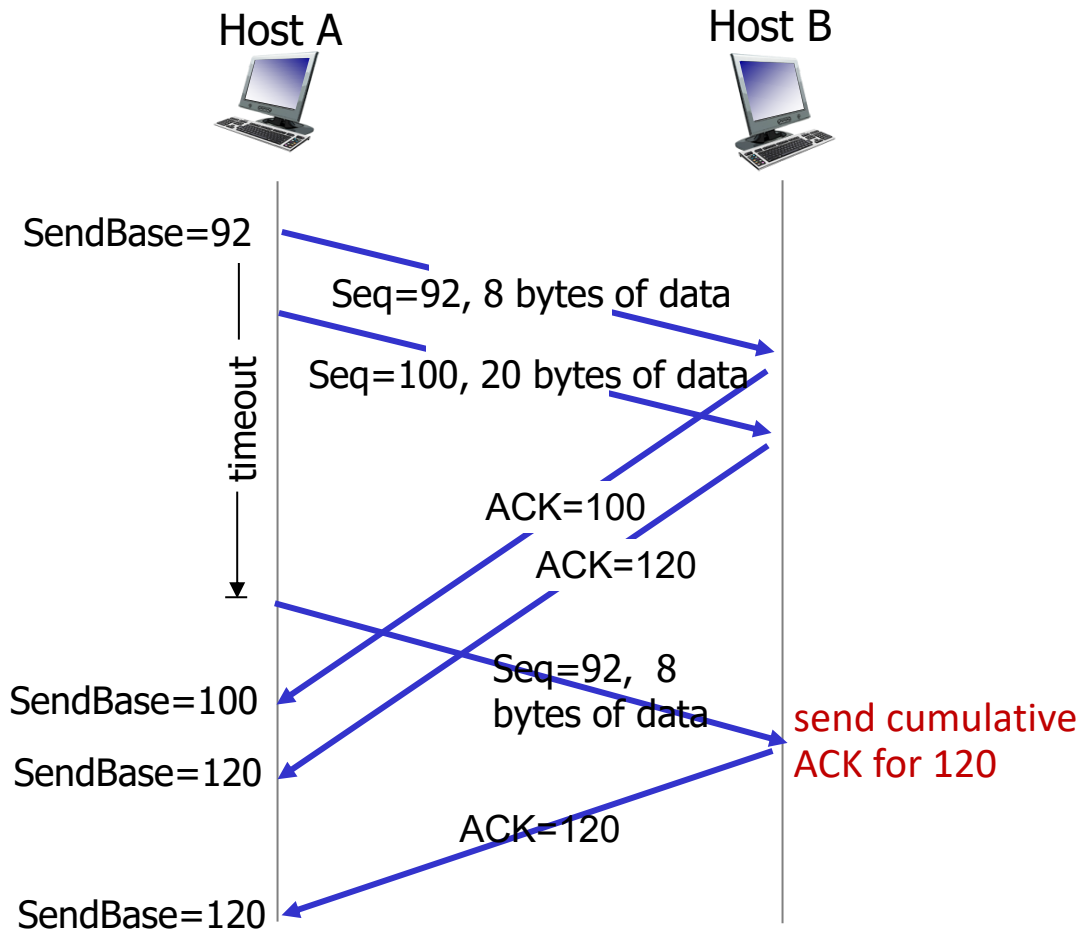
TCP Receiver: ACK generation [RFC 5681]

Sự kiện tại bên nhận	*Hành động*
Nhận được segment đúng với STT đang chờ. Tất cả segment trước đó đã được ACK.	Chờ segment kế tiếp trong 500 ms, nếu không nhận được thì sẽ ACK.
Nhận được segment đúng với STT đang chờ, một segment chưa được ACK.	Gửi ACK tích lũy để ACK cho 2 segment.
Nhận được segment không đúng thứ tự (STT cao hơn).	Gửi ACK trùng ngay lập tức, để chỉ cho bên gửi biết đang chờ segment nào.
Nhận được segment trong khoảng bị trống (giữa STT đang chờ và STT nhận được trước đó).	ACK ngay lập tức cho segment có thứ tự thấp nhất trong khoảng bị trống

TCP: các trường hợp truyền lại

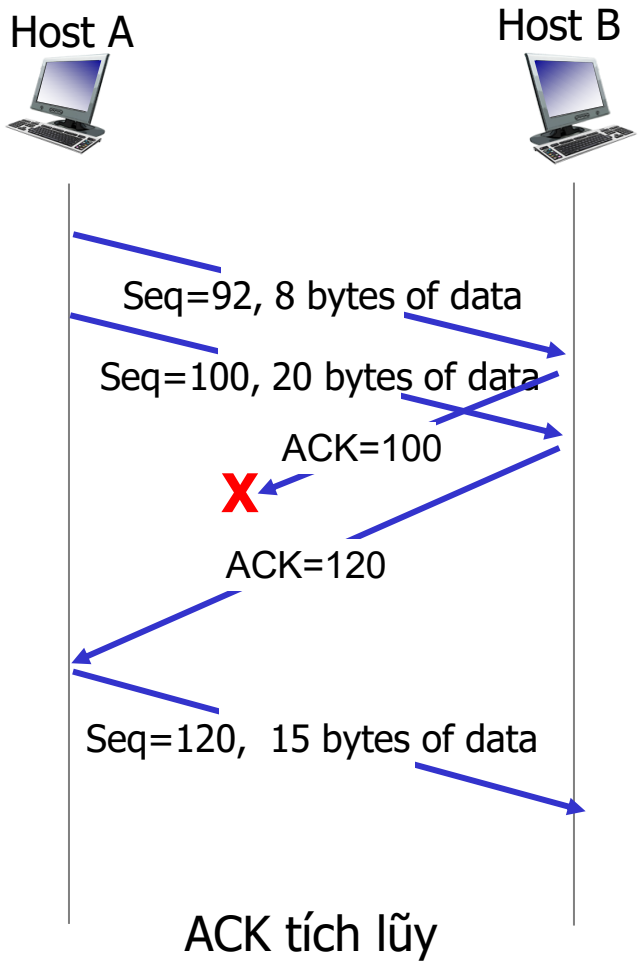


Mất ACK



timeout

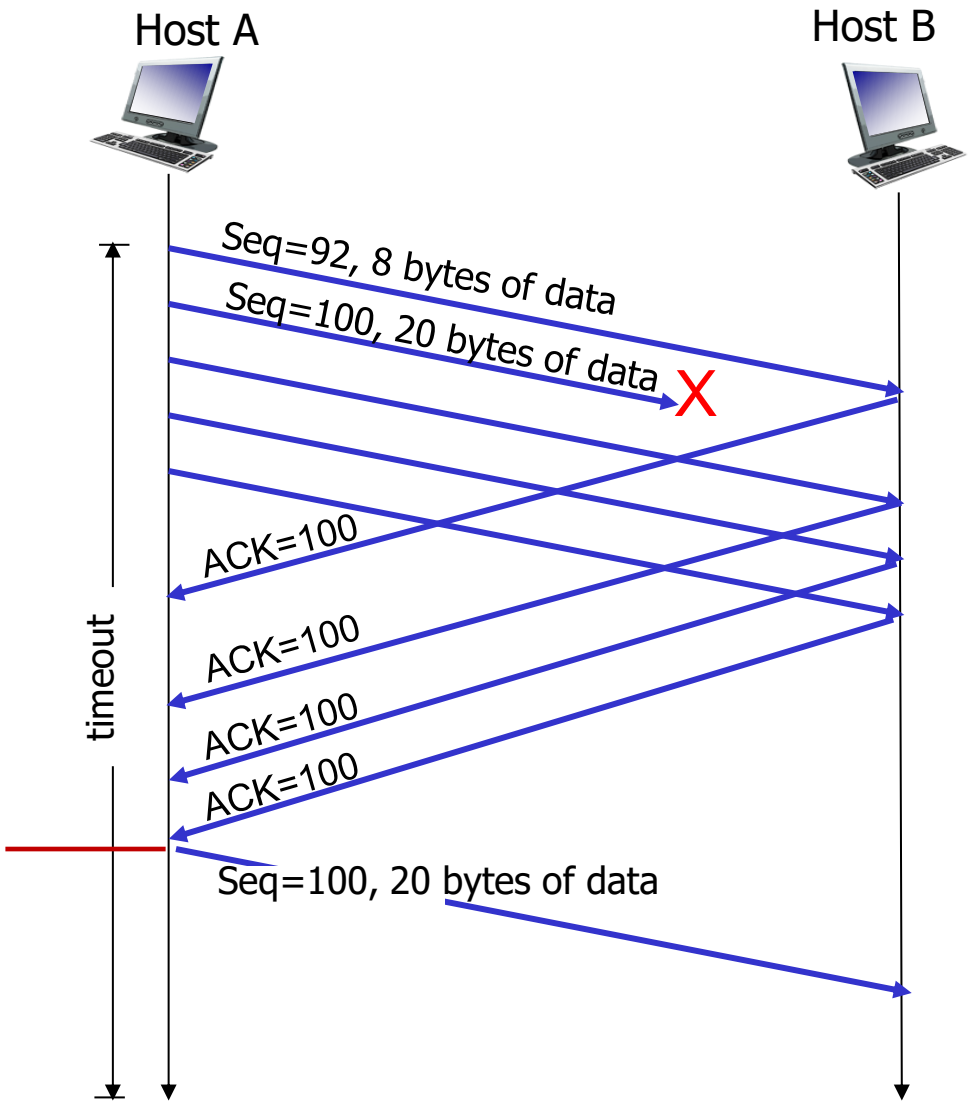
TCP: các trường hợp truyền lại



TCP: truyền lại nhanh

TCP: truyền lại nhanh

Khi bên gửi nhận được 3 ACK yêu cầu cùng segment, bên gửi gửi lại segment được yêu cầu mà không cần chờ timeout.

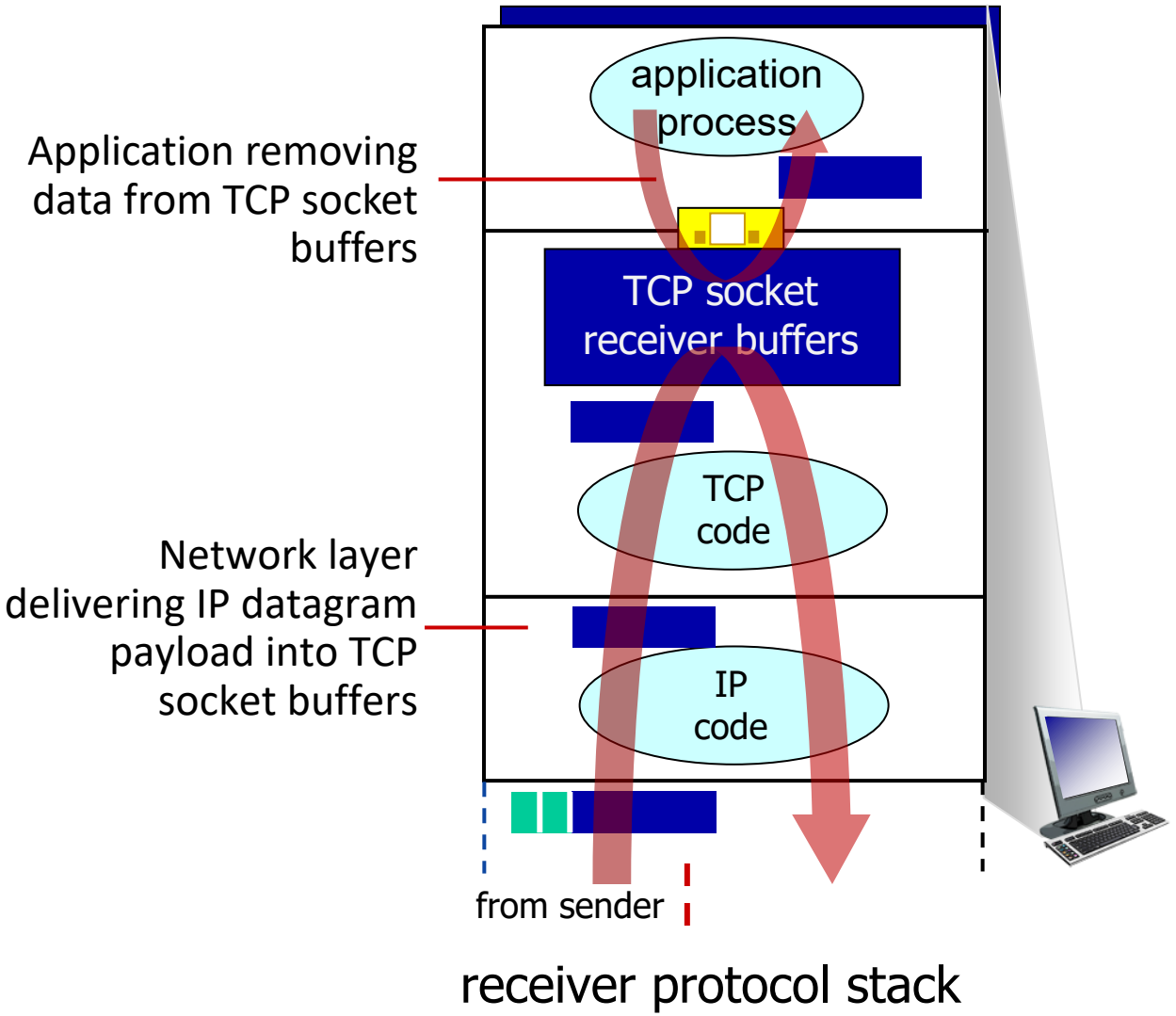


Việc nhận 3 ACK trùng lặp cho biết đã nhận được ít nhất 3 segment nằm sau segment đang bị thiếu – có khả năng bị mất segment này.
=> Truyền lại!

TCP: flow control – điều khiển luồng

Q: Điều gì xảy ra nếu tầng mạng truyền dữ liệu nhanh hơn tầng ứng dụng xóa dữ liệu khỏi bộ đệm của socket?

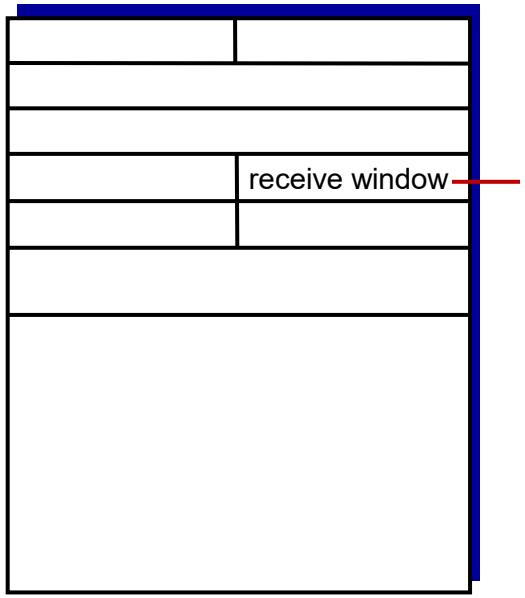




receiver protocol stack

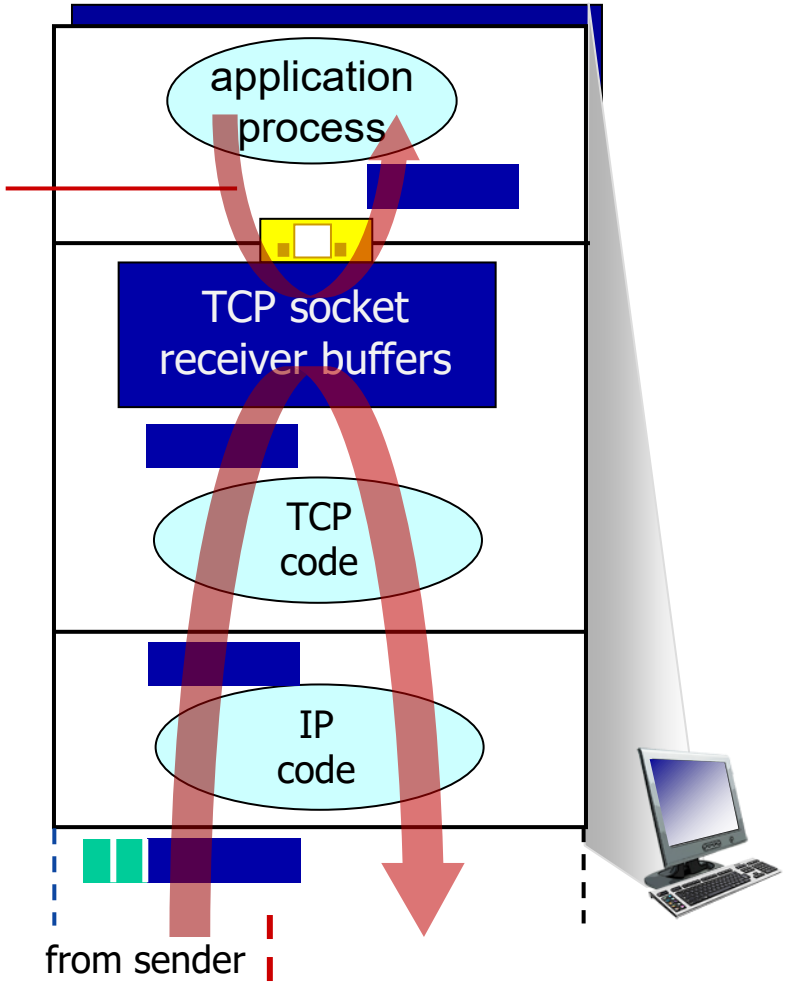
TCP: điều khiển luồng

Q: Điều gì xảy ra nếu tầng mạng truyền dữ liệu nhanh hơn tầng ứng dụng xóa dữ liệu khỏi bộ đệm của socket?



flow control: # bytes receiver willing to accept

Application removing data from TCP socket buffers



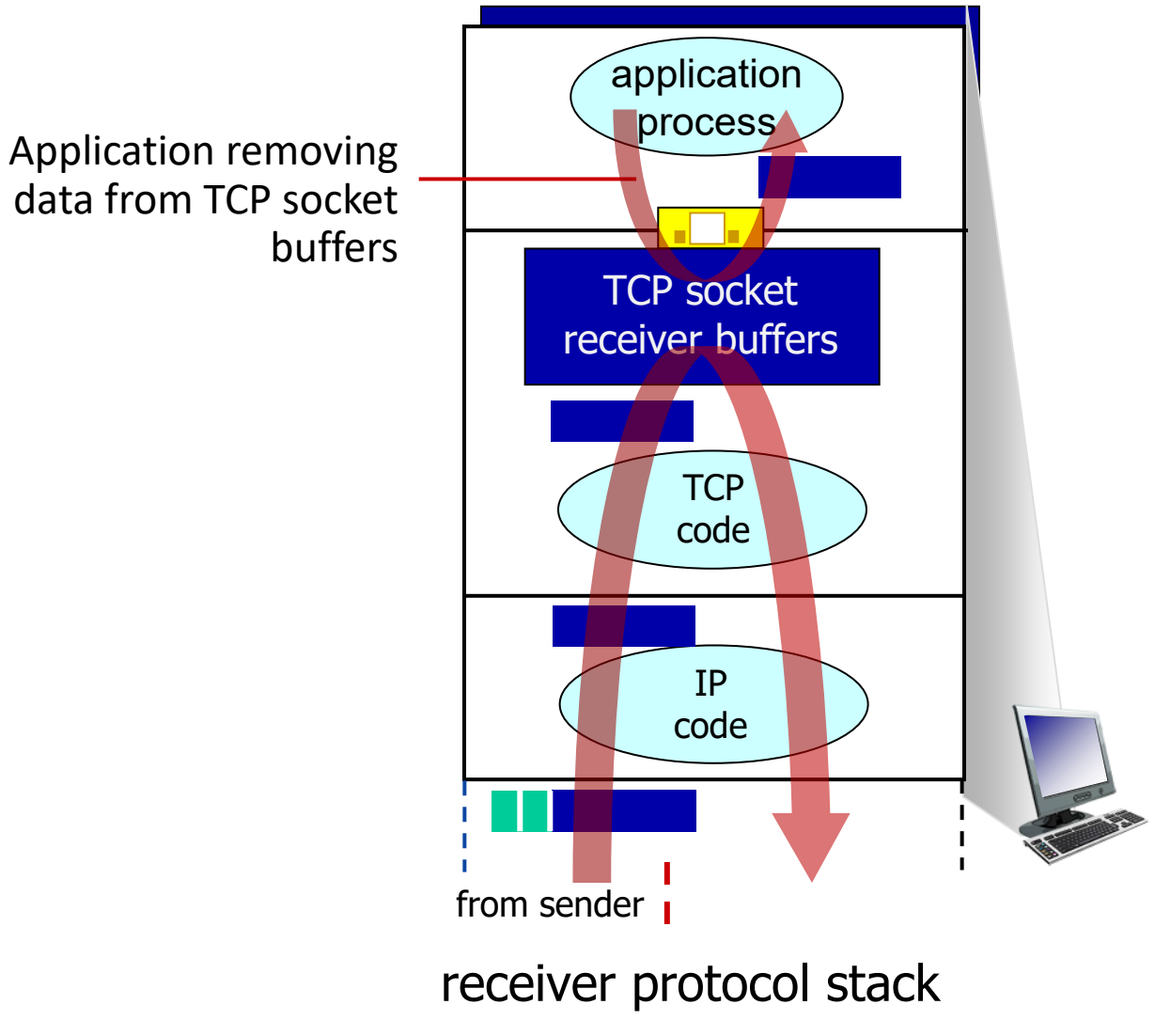
receiver protocol stack

TCP: điều khiển luồng

Q: Điều gì xảy ra nếu tầng mạng truyền dữ liệu nhanh hơn tầng ứng dụng xóa dữ liệu khỏi bộ đệm của socket?

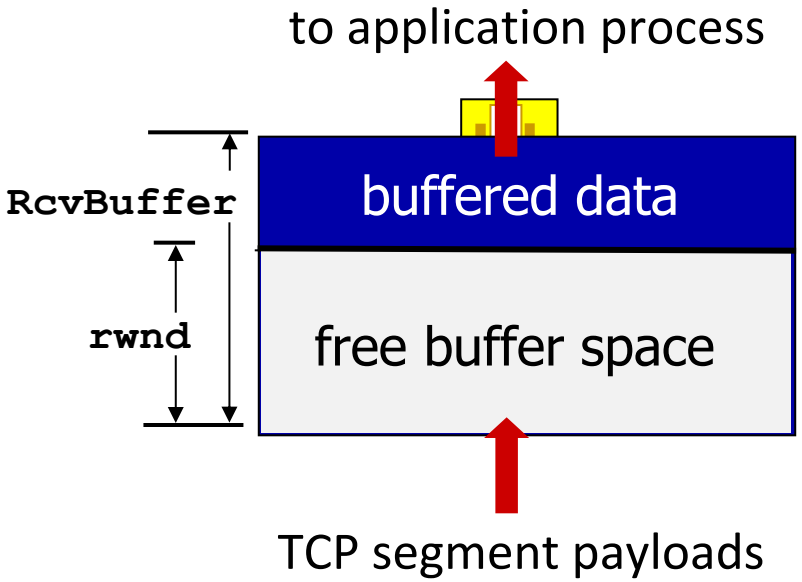
flow control

bên nhận kiêm soát bên gửi, vì vậy bên gửi sẽ không làm tràn bộ đệm của bên nhận bằng cách truyền quá nhiều, quá nhanh



TCP: điều khiển luồng

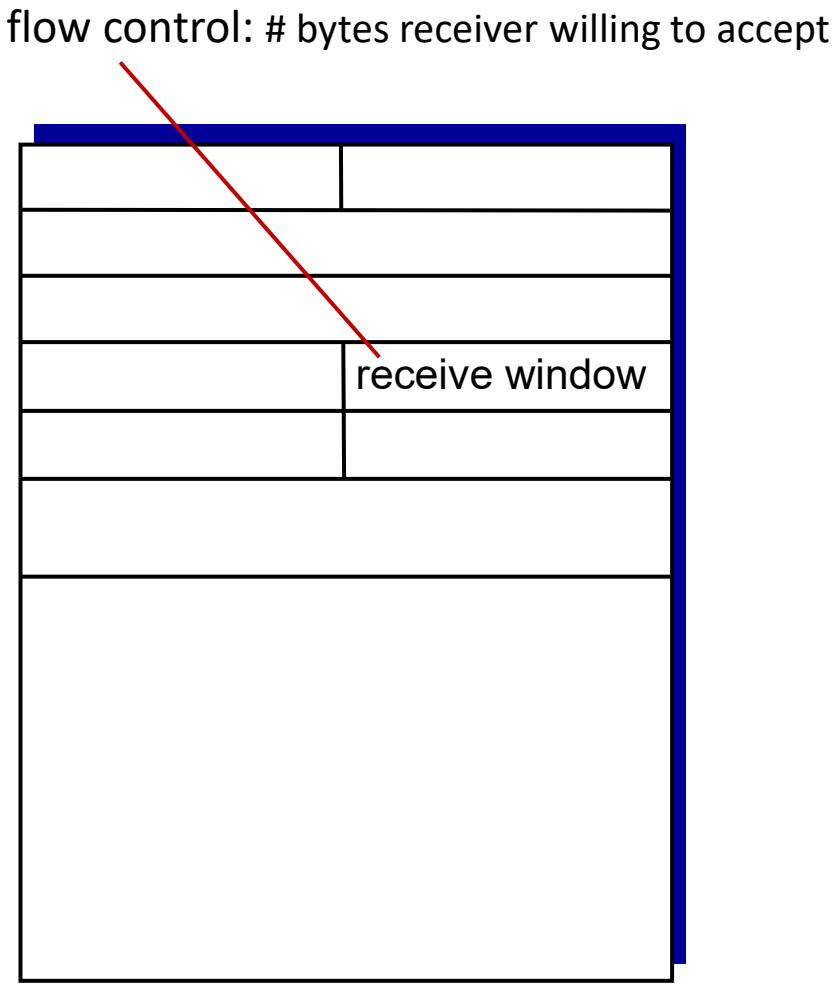
- Bên nhận thông báo dung lượng bộ đệm còn trống bằng trường rwnd trong TCP header
- Bên gửi giới hạn lượng dữ liệu được gửi sao cho không vượt quá rwnd do bên nhận quảng bá



TCP receiver-side buffering

TCP: điều khiển luồng

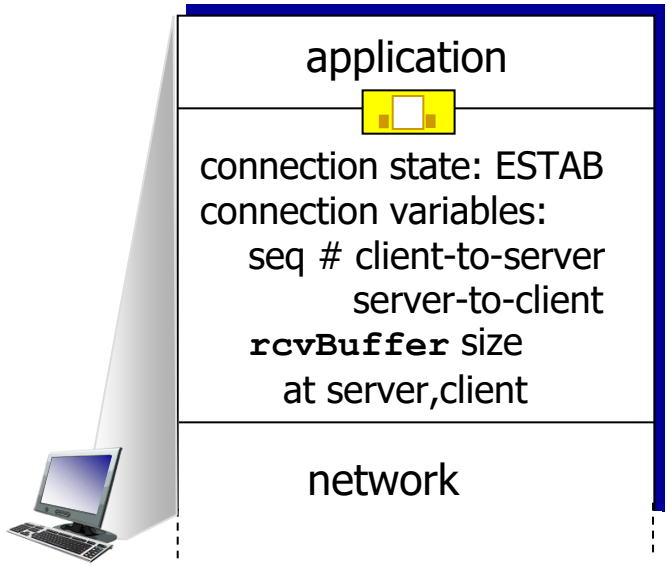
- Bên gửi thông báo bộ đệm trống bằng thông tin rwnd trong TCP header
- Bên gửi giới hạn lượng dữ liệu được gửi khi nhận được rwnd để đảm bảo bên nhận không bị quá tải

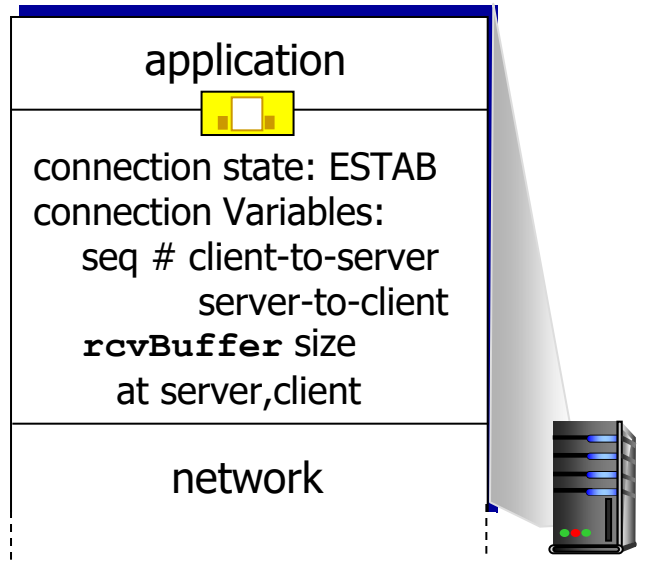


TCP segment format

TCP: quản lý kết nối

- trước khi trao đổi dữ liệu, bên gửi/bên nhận "handshake – bắt tay":
  - đồng ý thiết lập kết nối (mỗi bên đều biết bên kia sẵn sàng thiết lập kết nối)
  - đồng ý về các thông số kết nối (ví dụ: bắt đầu từ seq #s)






```
Socket clientSocket =
  newSocket("hostname","port number");
```

```
Socket connectionSocket =
  welcomeSocket.accept();
```

TCP 3-way handshake: bắt tay 3 bước

Client state

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

LISTEN

```
clientSocket.connect((serverName,serverPort))
```

choose init seq num, x
send TCP SYN msg

SYNSENT

SYNbit=1, Seq=x

choose init seq num, y
send TCP SYN-ACK
msg, acking SYN

SYNbit=1, Seq=y
ACKbit=1; ACKnum=x+1

received SYNACK(x)
indicates server is live;
send ACK for SYN-ACK;
this segment may contain
client-to-server data

ESTAB

ACKbit=1, ACKnum=y+1

received ACK(y)
indicates client is live

Server state

```
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
connectionSocket, addr = serverSocket.accept()
```

LISTEN

SYN RCVD

ESTAB

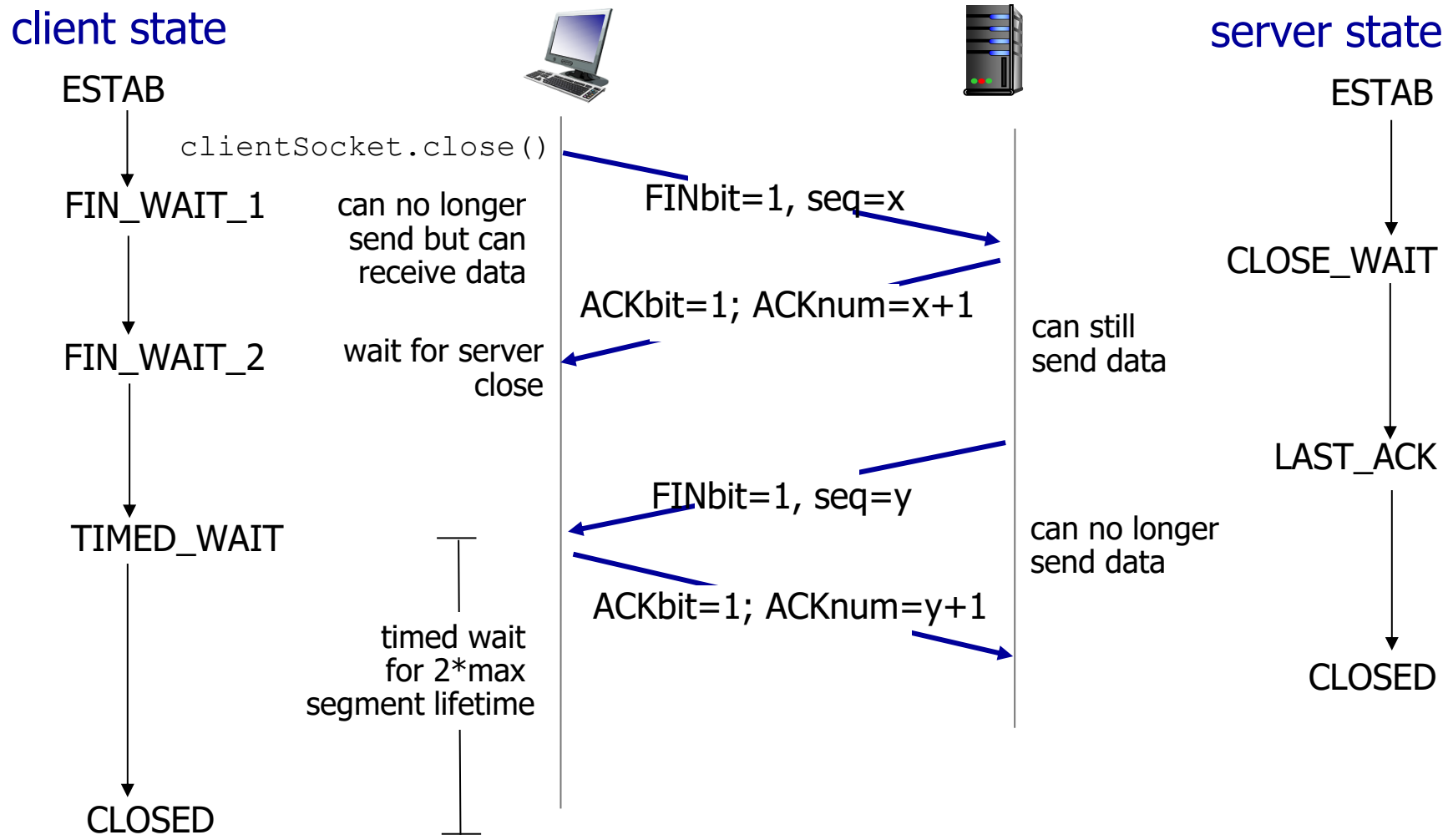
Ví dụ thực tế



Đóng kết nối

- Client và server lần lượt đóng chiều truyền của mình trong kết nối TCP
  - Gửi TCP segment có FIN bit = 1
- Phản hồi khi nhận segment có FIN với ACK
- Khi nhận FIN, ACK có thể kết hợp với FIN của nó
  - Có thể xử lý các trao đổi FIN đồng thời

Đóng kết nối


client state
ESTAB
clientSocket.close()
FIN_WAIT_1
can no longer send but can receive data
FINbit=1, seq=x
ACKbit=1; ACKnum=x+1
FIN_WAIT_2
wait for server close
can still send data
FINbit=1, seq=y
TIMED_WAIT
can no longer send data
ACKbit=1; ACKnum=y+1
timed wait for 2*max segment lifetime
CLOSED
server state
ESTAB
CLOSE_WAIT
LAST_ACK
CLOSED

Nội dung

- Các dịch vụ tầng vận chuyển
- Multiplexing and demultiplexing
- UDP
- Nguyên lý truyền tin cậy
- TCP
- TCP - Điều khiển tắc nghẽn
- Sự phát triển của các tính năng của tầng vận chuyển



Quản lý tắc nghẽn của TCP: AIMD

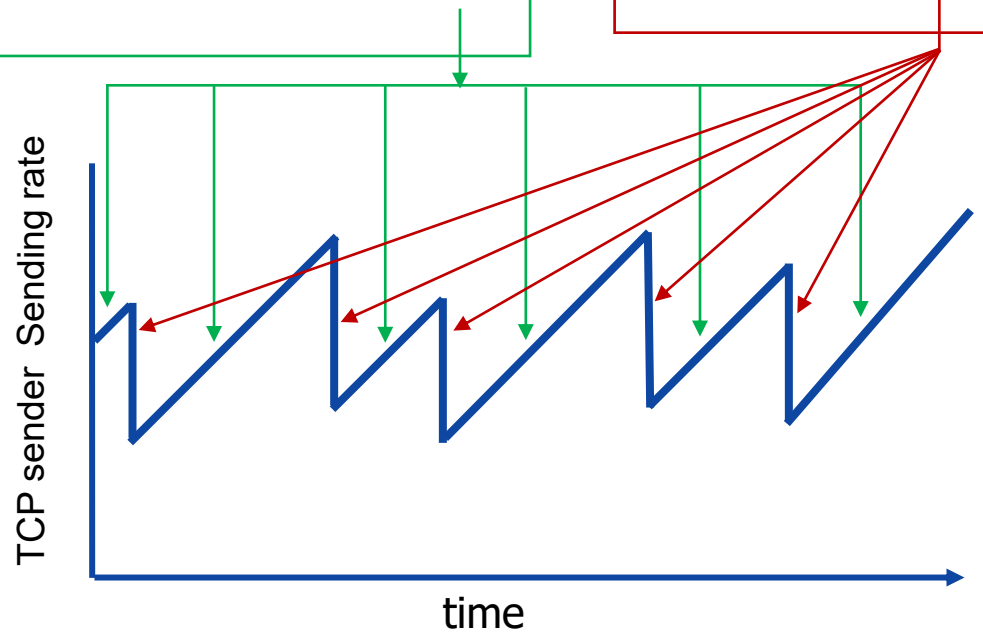
- Cách tiếp cận: bên gửi tăng dần tốc độ truyền cho đến khi phát hiện mất gói tin do tắc nghẽn (congestion), sau đó giảm tốc độ truyền dựa trên sự kiện mất gói.

Tăng tuyến tính

Tăng tốc độ gửi lên 1 MSS cho đến khi phát hiện sự mất mát

Giảm theo hệ số nhân

Giảm tốc độ gửi 1 nửa khi mất mát xảy ra



AIMD : *thăm dò băng thông*

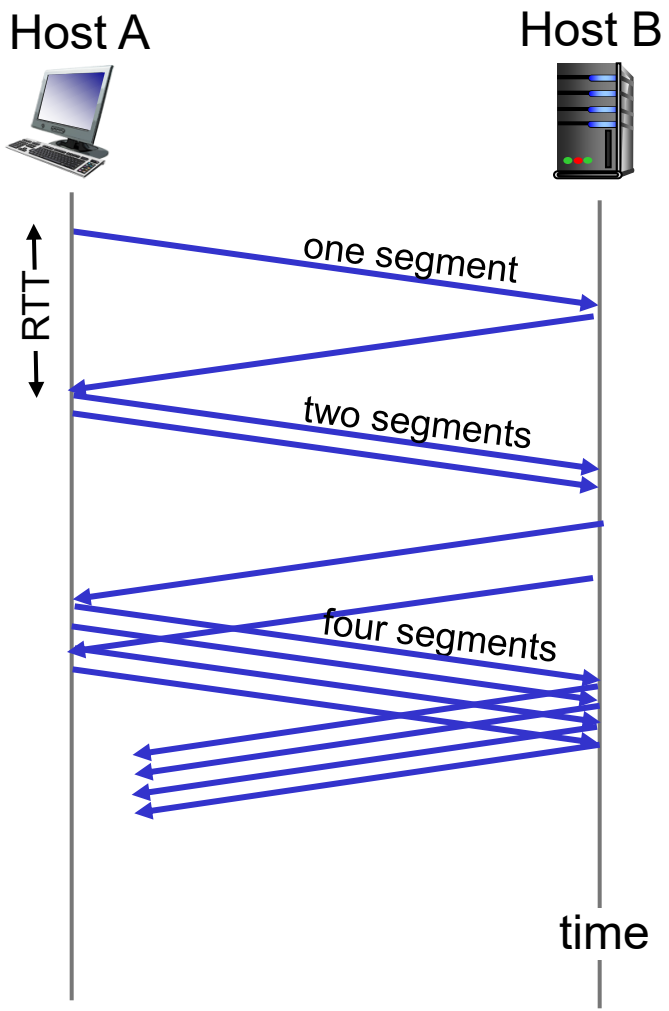
TCP: Điều khiển tắc nghẽn

- Ký hiệu
  - cwnd (congestion windows – cửa sổ tắc nghẽn): số segment được gửi trong cùng một window
  - Ssthresh (slow start threshold): ngưỡng kết thúc giai đoạn slow start và chuyển sang congestion avoidance

TCP slow start

- Khi bắt đầu, tăng tốc độ theo cấp số nhân cho đến khi xảy ra sự kiện mất đầu tiên:
  - Ban đầu **cwnd** = 1 MSS
  - Nhân đôi **cwnd** sau mỗi RTT
  - Việc tang được thực hiện bằng cách tăng **cwnd** cho mỗi ACK nhận được

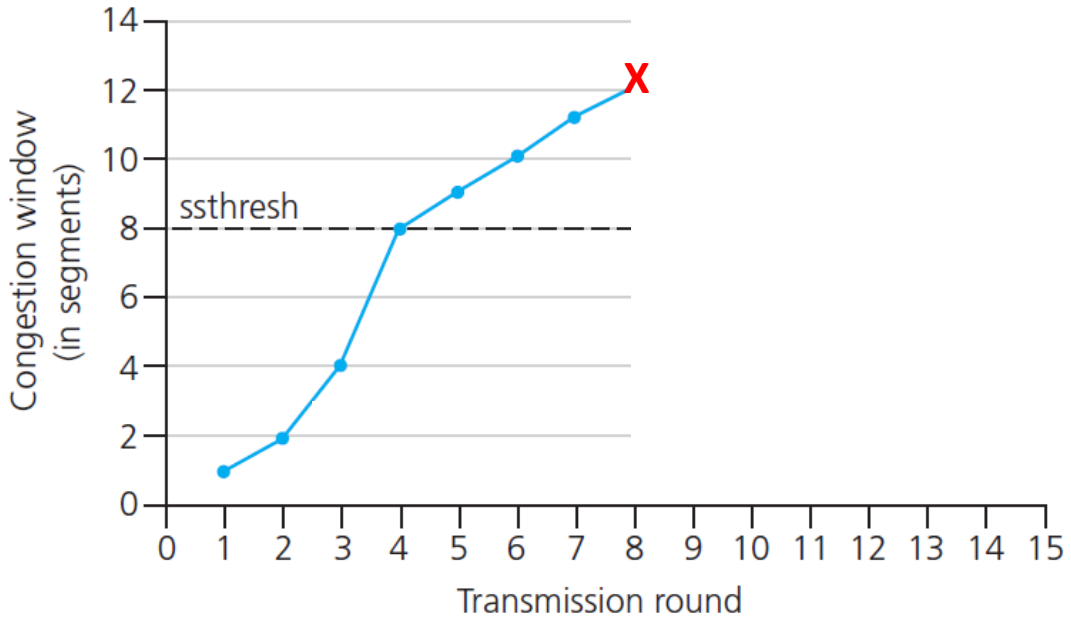
Tóm tắt: tốc độ ban đầu chậm, nhưng tăng nhanh theo cấp số nhân



TCP: từ slow start đến congestion avoidance

- Q: Khi nào TCP chuyển từ tăng theo cấp số nhân sang tăng tuyến tính?

- A:
  - **Ssthresh:** ngưỡng kết thúc Slowstart
  - Khi phát hiện mất gói, **ssthresh** được đặt bằng 1/2 giá trị của **cwnd** trước khi xảy ra mất gói



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

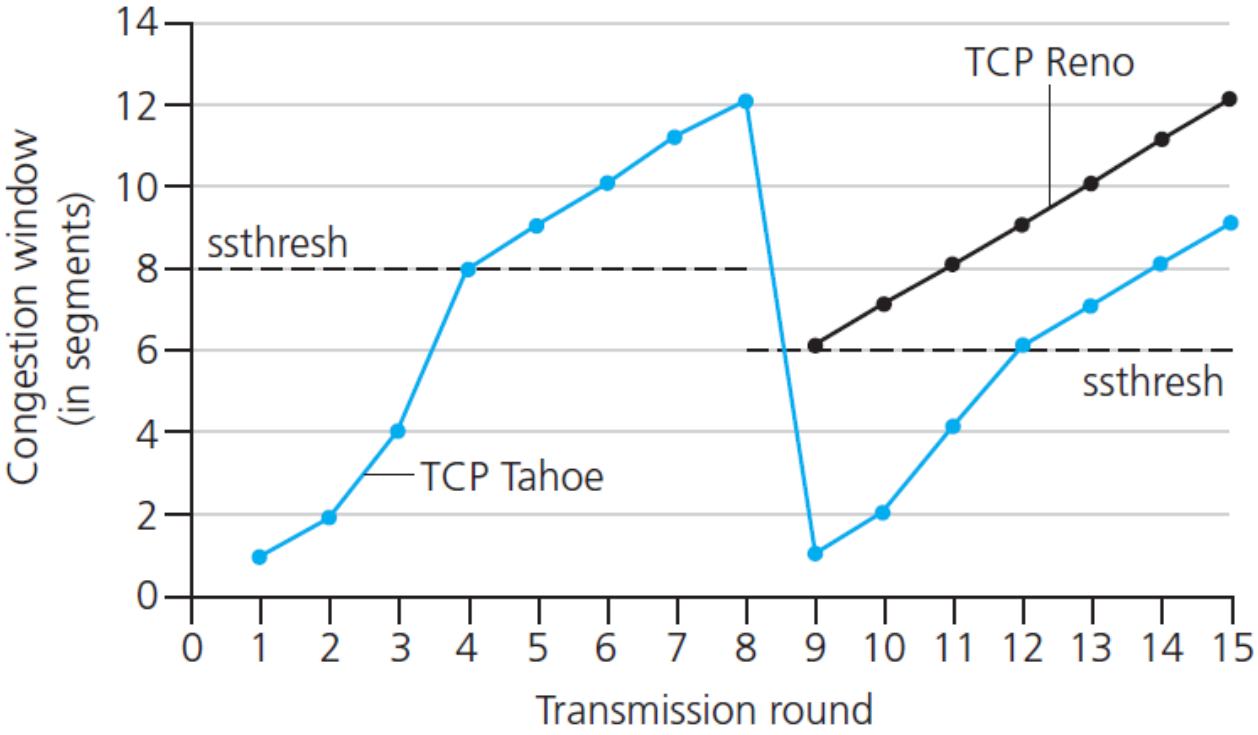
TCP: phát hiện, phản ứng khi mất gói

- Mất gói được chỉ ra bởi timeout:
  - **cwnd** (congestion window) được thiết lập 1 MSS;
  - Sau đó kích thước cửa sổ sẽ tăng theo cấp số nhân (như trong slow start) đến ngưỡng, sau đó sẽ tăng tuyến tính
- Mất gói được xác định bởi 3 ACK trùng nhau: TCP RENO
  - Các ACK trùng lặp chỉ ra khả năng truyền của mạng
  - **cwnd** bị cắt một nửa sau đó tăng theo tuyến tính
- TCP Tahoe luôn luôn đặt **cwnd** bằng 1 khi phát hiện mất gói (dù là timeout hoặc 3 ACK trùng lặp)

TCP: chuyển từ slow start qua CA (congestion avoidance)

Hỏi: khi nào tăng cấp số nhân nên chuyển qua tuyến tính?

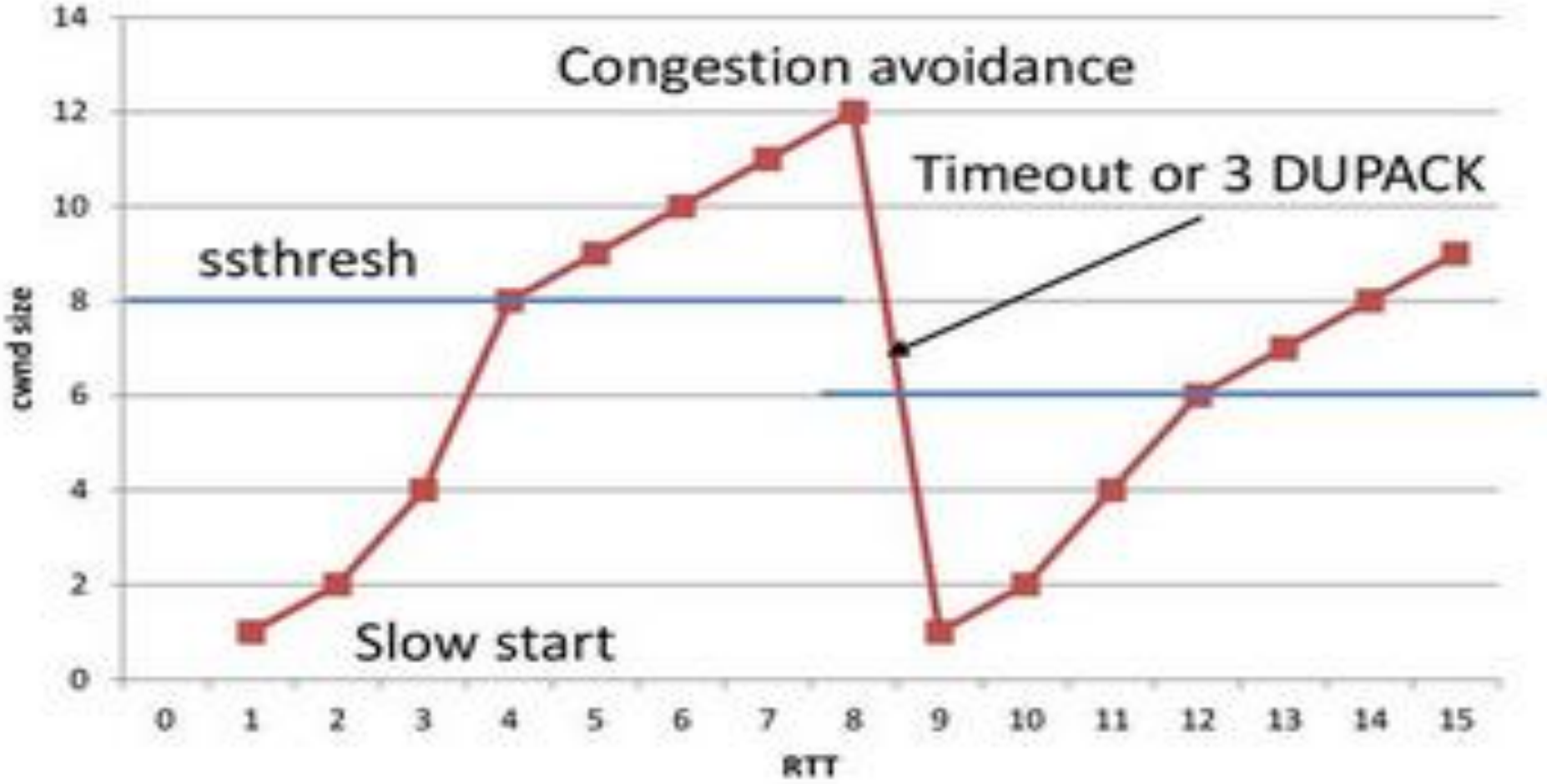
Trả lời: khi **cwnd** đạt đến ssthresh, TCP chuyển từ slow start sang congestion avoidance.



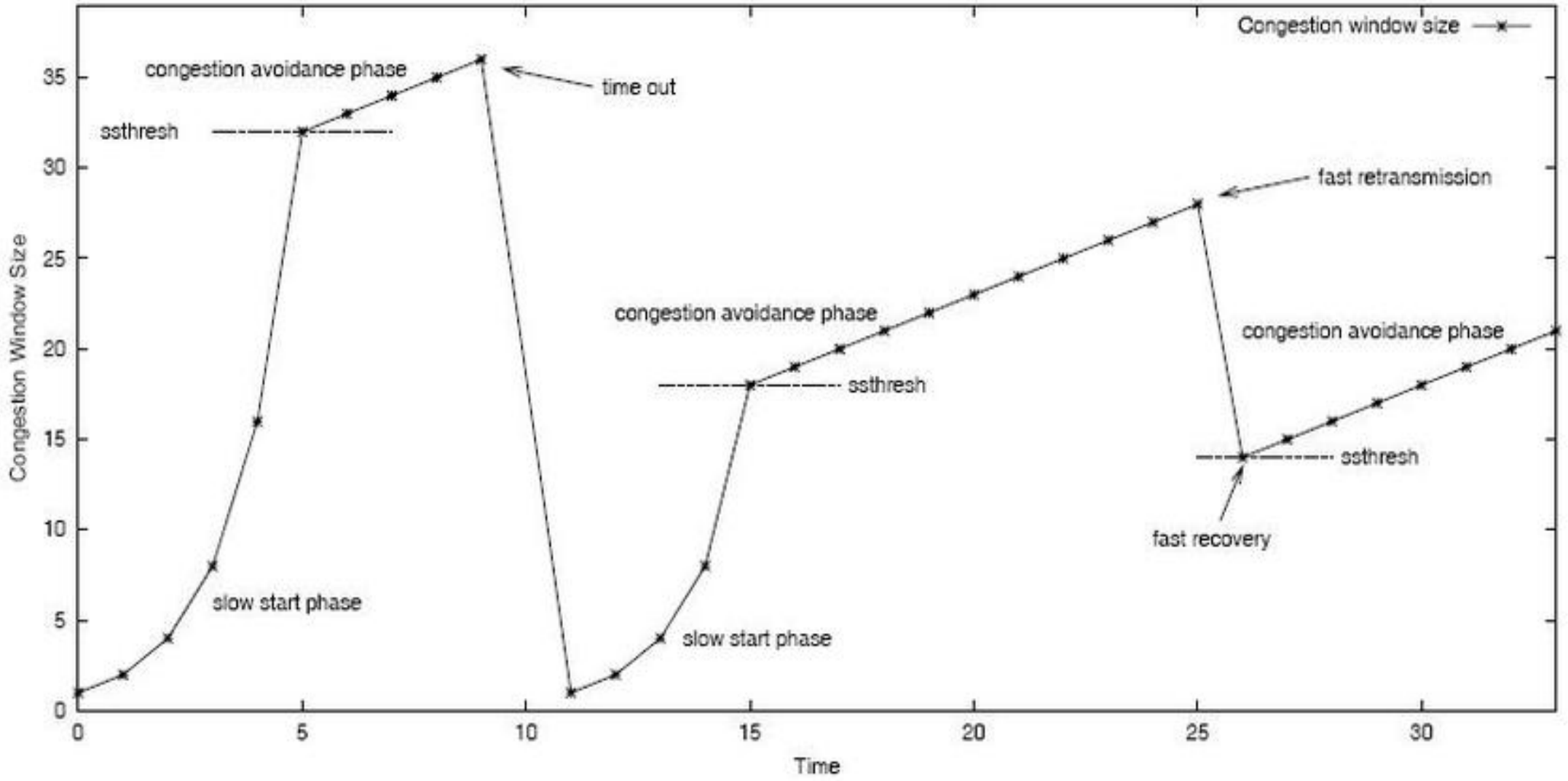
Thực hiện:

- **ssthresh** (slow start threshold) thay đổi
- Khi mất gói, **ssthresh** được đặt bằng 1/2 của **cwnd** ngay trước khi mất gói

Các pha trong TCP Tahoe



Các pha trong TCP Reno



TCP: Điều khiển tắc nghẽn – Các giai đoạn

Slowstart

cwnd(n) = cwnd(n-1)x2

Congestion Avoidance

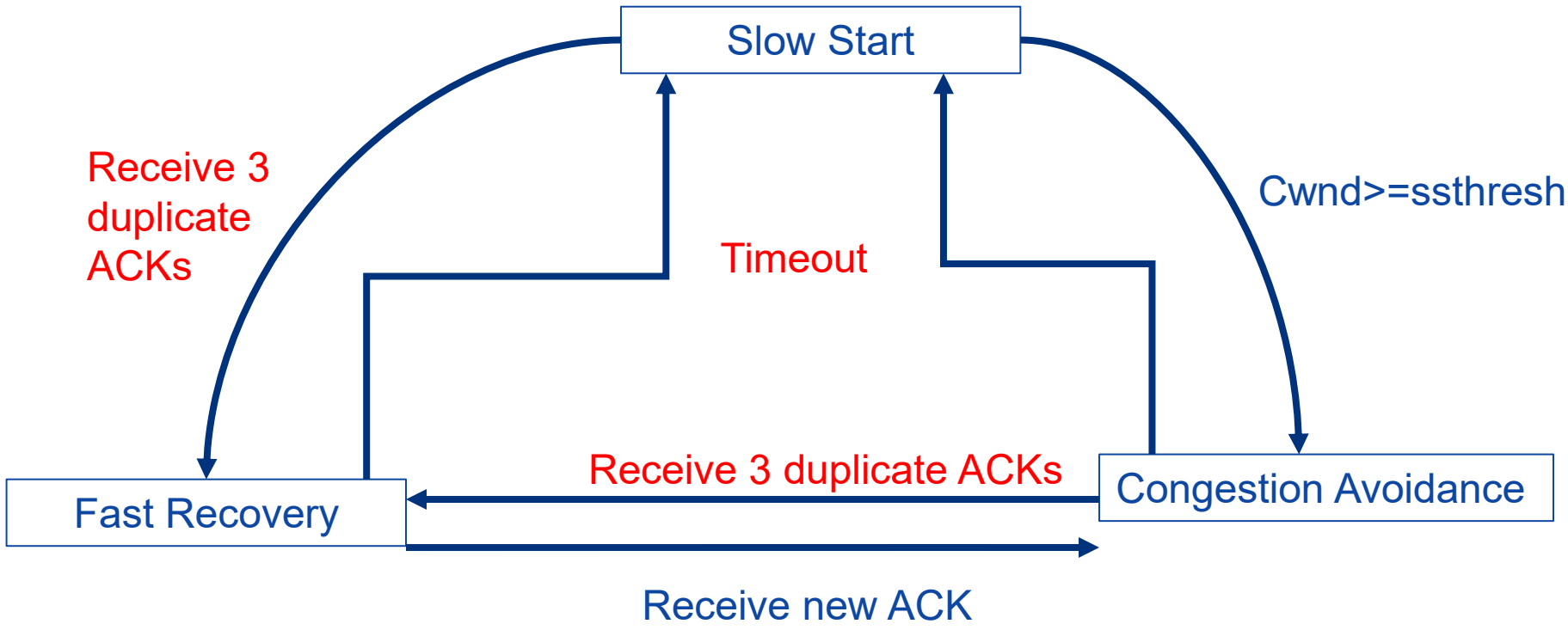
cwnd(n) = cwnd(n-1)+1

Fast Recovery

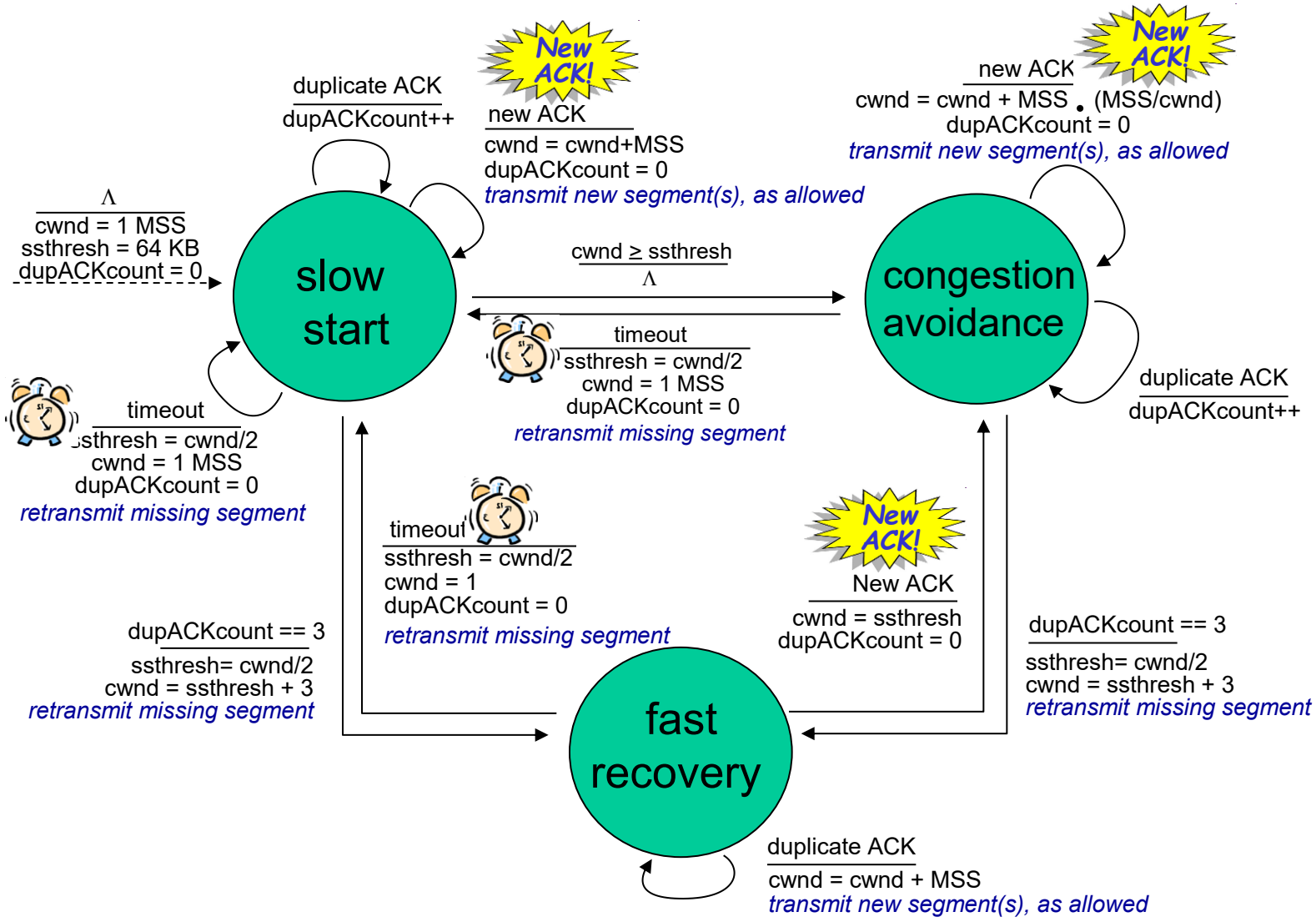
3ACK: cwnd(n)= cwnd(n-1)/2+3

Timeout: cwnd(n)=1

TCP: Điều khiển tắc nghẽn – Chuyển giai đoạn



Tổng kết


slow start
congestion avoidance
fast recovery
Λ
cwnd = 1 MSS
ssthresh = 64 KB
dupACKcount = 0
duplicate ACK
dupACKcount++
New ACK!
new ACK
cwnd = cwnd+MSS
dupACKcount = 0
transmit new segment(s), as allowed
cwnd ≥ ssthresh
Λ
timeout
ssthresh = cwnd/2
cwnd = 1 MSS
dupACKcount = 0
retransmit missing segment
new ACK
cwnd = cwnd + MSS . (MSS/cwnd)
dupACKcount = 0
transmit new segment(s), as allowed
duplicate ACK
dupACKcount++
timeout
ssthresh = cwnd/2
cwnd = 1 MSS
dupACKcount = 0
retransmit missing segment
timeout
ssthresh = cwnd/2
cwnd = 1
dupACKcount = 0
retransmit missing segment
New ACK
cwnd = ssthresh
dupACKcount = 0
dupACKcount == 3
ssthresh= cwnd/2
cwnd = ssthresh + 3
retransmit missing segment
dupACKcount == 3
ssthresh= cwnd/2
cwnd = ssthresh + 3
retransmit missing segment
duplicate ACK
cwnd = cwnd + MSS
transmit new segment(s), as allowed

Nội dung

- Các dịch vụ tầng vận chuyển
- Multiplexing and demultiplexing
- UDP
- Nguyên lý truyền tin cậy
- TCP
- TCP - Điều khiển tắc nghẽn
- Sự phát triển của các tính năng của tầng vận chuyển



Evolving transport-layer functionality



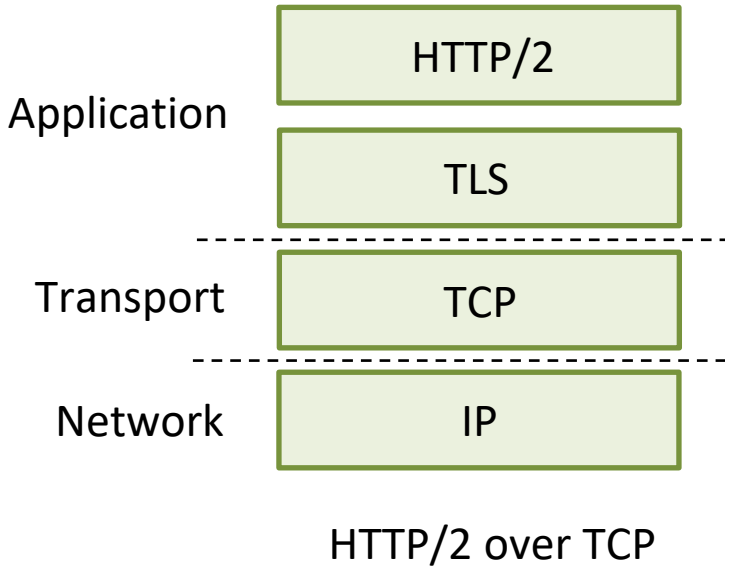
- TCP, UDP: principal transport protocols for 40 years
- different “flavors” of TCP developed, for specific scenarios:

Scenario	Challenges
Long, fat pipes (large data transfers)	Many packets “in flight”; loss shuts down pipeline
Wireless networks	Loss due to noisy wireless links, mobility; TCP treats this as congestion loss
Long-delay links	Extremely long RTTs
Data center networks	Latency sensitive
Background traffic flows	Low priority, “background” TCP flows

- moving transport–layer functions to application layer, on top of UDP
  - HTTP/3: QUIC

QUIC: Quick UDP Internet Connections

- application-layer protocol, on top of UDP
  - increase performance of HTTP
  - deployed on many Google servers, apps (Chrome, mobile YouTube app)



HTTP/2 over TCP

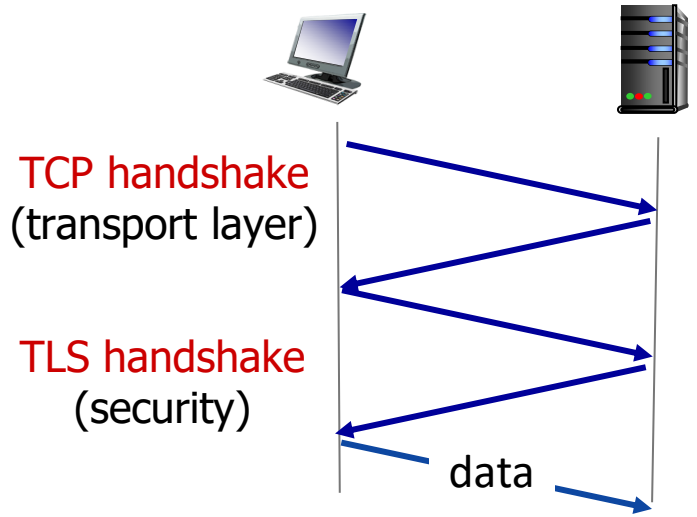
QUIC: Quick UDP Internet Connections



- multiple application-level "streams" multiplexed over single QUIC connection
  - separate reliable data transfer, security
  - common congestion control
    - **error and congestion control:** "Readers familiar with TCP's loss detection and congestion control will find algorithms here that parallel well-known TCP ones." [from QUIC specification]
    - **connection establishment:** reliability, congestion control, authentication, encryption, state established in one RTT

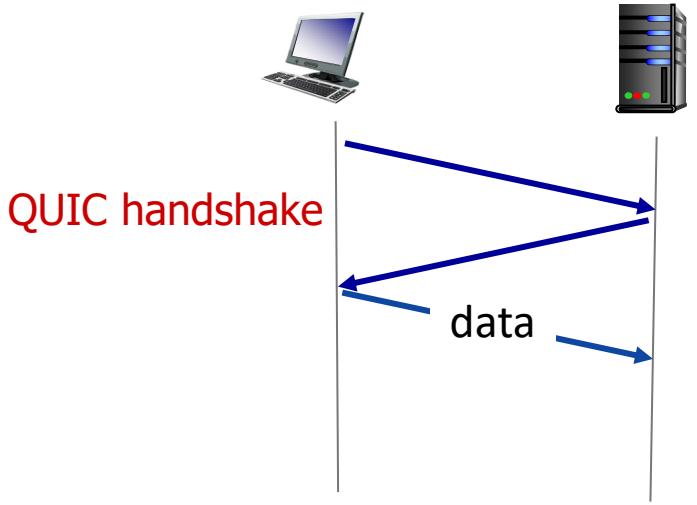
adopts approaches we've studied in this chapter for connection establishment, error control, congestion control

QUIC: Connection establishment



TCP (reliability, congestion control state) + TLS (authentication, crypto state)

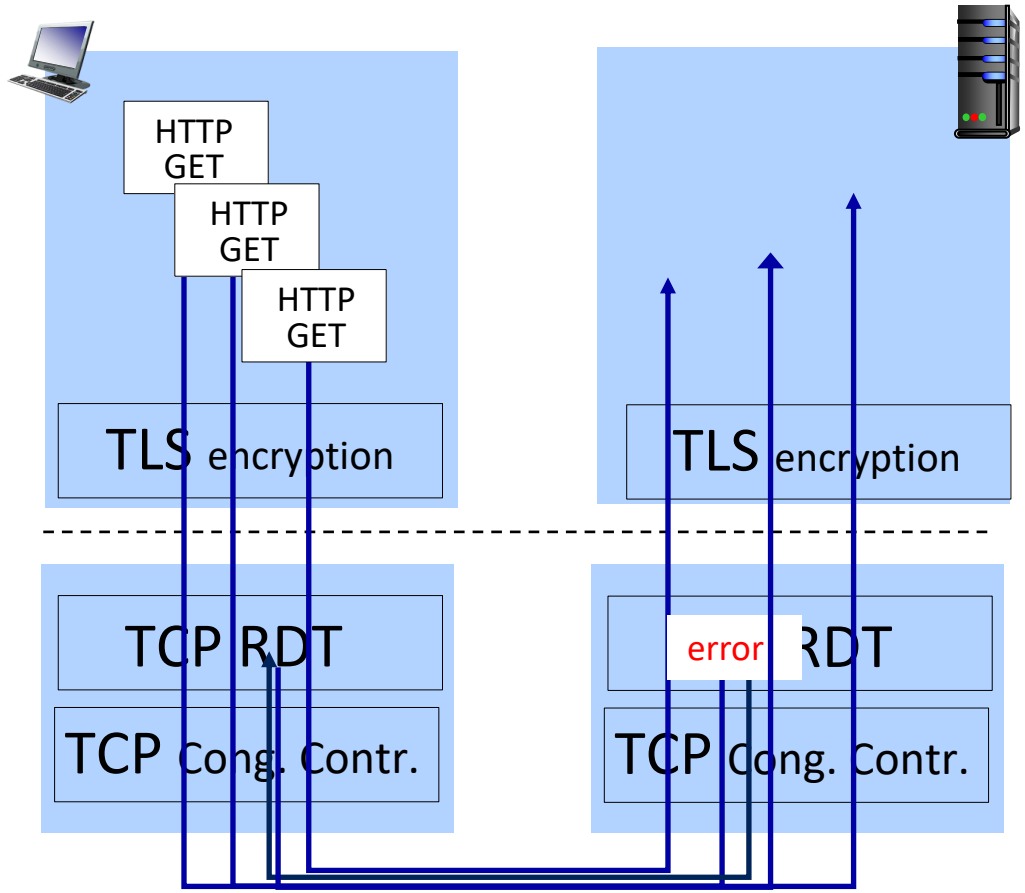
- 2 serial handshakes



QUIC: reliability, congestion control, authentication, crypto state

- 1 handshake

QUIC: streams: parallelism, no HOL blocking



(a) HTTP 1.1

Chương 3: Tổng kết

- Các nguyên lý của tầng vận chuyển:
  - multiplexing, demultiplexing
  - Truyền tin cậy
  - Điều khiển luồng
  - Điều khiển tắc nghẽn
  - UDP
  - TCP

Tiếp theo:

- Tầng mạng (Network Layer)